408 详解 · 2011 年选择题 · 升级版

本册每题先给出原卷题干与选项全文（转录自公开渠道真题，来源与核对见《2011-升级日志》），再按 6 节讲解；讲解重心放在方法、推理、易混点上。

>

说明：Q27 的 A、B、C 各自序列与部分数字在源材料中未齐，本书只按排除逻辑讲，并标注部分核对；Q9 的陈述Ⅰ逐字表述、Q12 的 A、B、C 与三个指标的字母对应未齐，按已知逻辑与定义讲，不编造原文。

升级版说明：每题标题下新增"**题干与选项：**"节（原卷题干与选项全文）；6 节之后新增 7 类增补小节（【图像】【定位】【易混对比】【坑位】【套路】【类比】【口诀】，无适用内容的小节省略）；重点句按全卷统一配色：红＝关键结论、蓝＝定位与套路关键动作、橙＝坑位警告、绿＝口诀；【口诀】附考频标注（该知识点在 2009–2022 年解析中的出现次数＋高/中/低频档位，高频红标）。

第 1 题 · 程序最高频语句的时间复杂度（正确答案：A）

题干与选项：

设 n 是描述问题规模的非负整数，下面程序片段的时间复杂度是( )

x=2;

while(x<n/2)

x=2*x;

- A. O(log2n)
- B. O(n)
- C. O(nlog2n)
- D. O(n^2)

入手点： 最高频语句是 x＝2*x，x 每次翻倍；数一数 x 从 1 翻到 n 要翻多少次。

专有名词从零解释：

- 先想“最高频语句”：程序里执行次数最多的那条语句，整个程序的工作量就看它。
- 再想“时间复杂度”：衡量工作量随问题规模怎么涨的粗略尺子，只看涨势，不抠常数。
- 再想“对数级”：翻倍涨的工作量，规模翻一倍，工作量只多 1 次。

考查知识点：

- 先想 x 的变化：x 从 1 开始，每次乘 2，执行 T(n) 次之后，x 变成 2 的 T(n) 次方；题目给的条件推出 2 的 T(n)+1 次方不超过 n/2，解出来 T(n)＝log2(n/2)-1＝log2 n-2。
- 再想量级：log2 n-2 里起决定作用的是 log2 n，常数 -2 不影响涨势，所以是 O(log2 n)，就是 A。
- 拿生活例子验算：n＝1024 时，x 从 1 翻倍到 512 要翻 9 次（2 的 9 次方是 512），再翻一次就超了；n 变成 2048 时，也只多翻 1 次——规模翻倍，工作量只加 1，这就是对数级。
- 边界情况：n 很小时，log2 n-2 可能是负数，但时间复杂度只看 n 很大的时候的涨势，不看小 n 的具体数值。
- 易混对子：不要把“每次乘 2”当成“工作量翻倍”——每次乘 2 的是 x 的值，工作量是翻的次数；不要把 log2 n-2 里的 -2 当回事——常数项在量级里直接丢掉。

正确选项为什么对： 执行次数是 log2 n-2，量级是 O(log2 n)，就是 A，选 A。

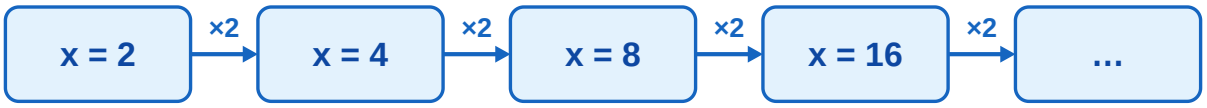
错误选项为什么错＋举例：

- 量级算成线性级：把“x 每次翻倍”想成了“x 每次加 1”。教训：翻倍增长对应的是对数次数，不是线性次数。
- 量级算成平方级：把循环次数和每次的工作量乘起来了。教训：这道题只有一层循环，工作量就是循环次数本身。

笔记本一句话： 第 1 题＋最高频语句：x 每次翻倍，执行次数是 log2 n-2，量级 O(log2 n)。

【图像】

Q1 x=2; while(x<n/2) x=2*x;



每轮 x 翻一番：第 k 轮后 x = 2^(k+1)（指数增长）

退出条件：2^(k+1) ≥ n/2 ⇒ k ≈ log2 n

循环体为常数时间 ⇒ 总时间 O(log2 n)

注：n/2 只改变常数项，不改变 log 量级

时间复杂度 O(log2 n)
答案：A

x 从 2 开始每次翻倍、循环约 log2n 次后达到 n/2 的示意图

【定位】 数据结构 · 第1章 绪论 · 时间复杂度分析

【易混对比】

- "x 翻倍"和"x 每次加 1"的区别：翻倍是指数增长，循环次数取对数；加 1 是线性推进，循环次数是 n 量级。
- "O(log2n)"和"O(n)"的区别：log 来自"每次乘 2"的增长方式；O(n) 要求每轮只推进常数步或把 n 个元素各处理一遍。

循环约 log2n 次，时间复杂度 O(log2n)，选 A。

【坑位】

别被 while 条件里的 n/2 带偏：n/2 只改变常数项，量级仍然是 log。

误选 B（O(n)）：把 x=2*x 看成了 x=x+1；教训：乘 2 是指数增长，循环次数是指数 k，不是线性步数。误选 D（$O(n^2)$）：臆想了嵌套循环，本题只有一层循环。误选 C（O(nlog2n)）：把"循环 n 次、每次 log"硬套进来，循环次数本身只有 log2n。

【套路】

- 第一步，写出 x 的变化规律：x 初值 2，每轮 x=2*x，第 k 轮后 x=2^(k+1)。
- 第二步，用退出条件解出 k：2^(k+1)⩾n/2 时退出，k≈log2n。
- 第三步，写出量级：循环体是常数时间，总量级 O(log2n)，对上选项 A。

【类比】

像细菌分裂：1 个变 2 个、2 个变 4 个，要涨到 n/2 个只需要分裂 log2n 次。x 就是细菌数，循环一次分裂一次。术语贴回去：分裂次数是"循环次数"，每次分裂是"常数时间的循环体"，总步数是"时间复杂度 O(log2n)"。

【口诀】 变量翻倍每次乘，循环次数取对数；涨到 n 半就退出，log2n 选 A 定。

【考频】 「时间复杂度」在 2009–2022 年解析中出现 12 次 · 高频

第 2 题 · 栈的出栈序列种数（正确答案：B）

题干与选项：

元素 a，b，c，d，e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是( )

- A. 3
- B. 4
- C. 5
- D. 6

入手点： d 第一个出来，说明 d 出来之前 c、b、a 还在栈里、e 还没进栈；之后 e 可以在任何时候进栈出栈，数一数 e 能插进几个空位。

专有名词从零解释：

- 先想“栈”：像一摞盘子，只能从上面放、只能从上面拿，后放的先拿，这叫后进先出。
- 再想“出栈序列”：元素离开栈的先后顺序；栈里的相对顺序是死的，新元素可以在任何时候插队进来。

考查知识点：

- 先想 d 出来之前：入栈顺序是 a、b、c、d、e；d 第一个出来，说明 d 出来时 c、b、a 还在栈里（从上到下是 c、b、a），e 还没进栈。
- 再想剩下的顺序：c、b、a 只能按 c、b、a 的顺序出来，不能乱；e 可以在任何时候进栈、马上出来，所以 e 能插进 4 个空位：c 前面、c 和 b 之间、b 和 a 之间、a 后面。
- 再想一共几种：4 个空位就是 4 种序列，按逻辑推导是 d、e、c、b、a / d、c、e、b、a / d、c、b、e、a / d、c、b、a、e（注：这是按逻辑推导出来的 4 种，不是选项原文抄），就是 B。
- 拿生活例子验算：四个人已经在排队等着出来（c、b、a 只能按这个顺序），e 是个后来的人，可以插到队伍的任何一个位置——前面、中间两个空当、最后面，一共 4 个位置。
- 边界情况：如果 e 在 d 之前进栈，d 就不可能是第一个出来的；所以 d 第一个出来这个条件，直接定死了 e 还没进栈。
- 易混对子：不要把 c、b、a 的顺序排错——栈里的元素只能按 c、b、a 出来，不能是 a、b、c；不要漏了 e 插在最后面那种——a 后面也是一个空位。

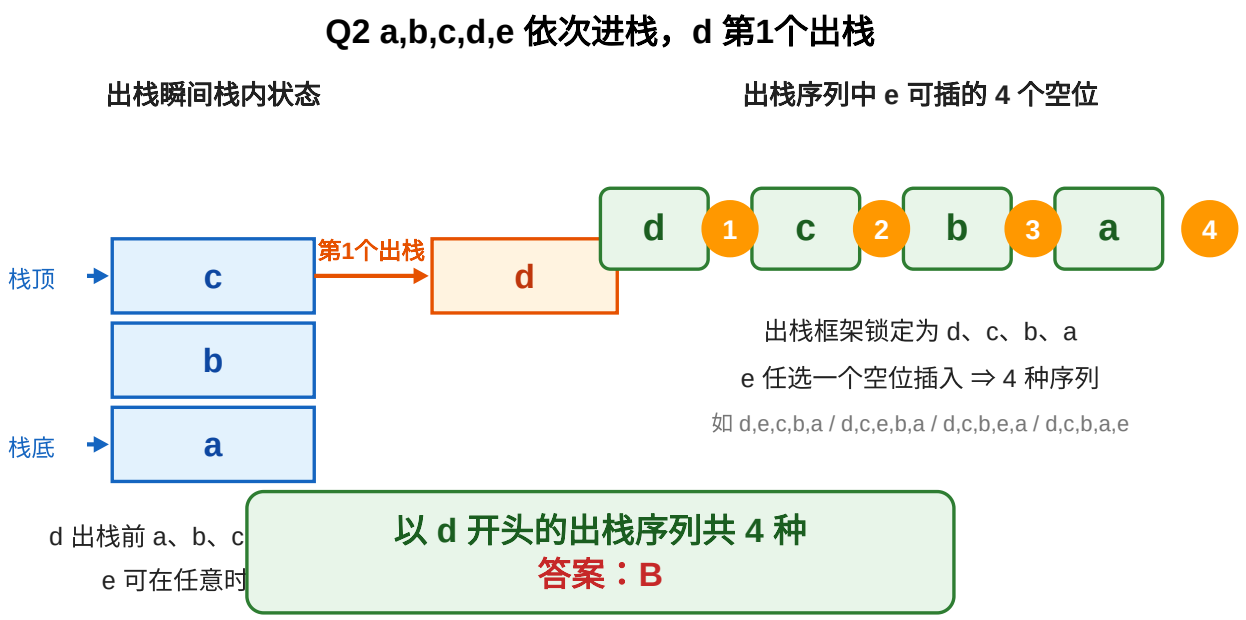
正确选项为什么对： e 有 4 个空位可插，一共 4 种，就是 B，选 B。

错误选项为什么错＋举例：

- 种数算少了：漏了 e 插在某个位置，比如漏了 a 后面。教训：空位要从头数到尾，c 前、c 与 b 间、b 与 a 间、a 后，一共 4 个。
- 种数算多了：把 c、b、a 的顺序也排起来了。教训：栈里的相对顺序是死的，不能重排，只有 e 的位置是活的。

笔记本一句话： 第 2 题＋栈出栈种数：d 先出，c、b、a 顺序已定，e 可插 4 个空位，共 4 种。

【图像】



a、b、c、d 入栈后 d 先出栈、e 可插入出栈序列 4 个空位的示意图

【定位】 数据结构 · 第3章 栈和队列 · 栈的出栈序列

【易混对比】

- "d 第 1 个出栈"和"d 最后出栈"的区别：d 第 1 个出栈时 a、b、c 必须已在栈中压着，e 还没进栈、可以自由安排插入位置；d 最后出栈则 e 的位置受限。
- "本题的 4 种"和"卡特兰数"的区别：全部出栈序列的总数用卡特兰数；本题固定了首个出栈元素，退化成"e 插 4 个空位"的计数。

d 第 1 个出栈锁定出栈框架为 d、c、b、a，e 任选 4 个空位之一插入，共 4 种，选 B。

【坑位】

d 第 1 个出栈不代表 a、b、c 先出栈：它们还在栈底压着，顺序锁定为 c、b、a，e 是唯一能"插队"的。

误选 A（3 种）：漏数了"e 也可以最后一个出栈"这个空位。误选 C（5 种）：把空位数成了 5——d 已出，框架是 d、c、b、a，e 只能插在 c 前、b 前、a 前、a 后这 4 个空位。

【套路】

- 第一步，定进栈框架：d 第 1 个出栈，则 d 出栈前 a、b、c 必须已入栈（e 可在任意时刻入栈）。
- 第二步，定出栈框架：d 出栈后，栈中从顶到底为 c、b、a，出栈顺序锁定为 d、c、b、a。
- 第三步，数 e 的空位：e 在这个出栈序列里可插 4 个空位，共 4 种，对上选项 B。

【类比】

像排队买票：d 是第 1 个买到票离开的，a、b、c 在他身后按 c、b、a 的顺序排好（动不了），e 是后来的人，可以插到队伍的 4 个空位中任意一个。术语贴回去：队伍是"出栈序列"，插空位是"e 的入栈出栈时机"，4 个空位是"4 种序列"。

【口诀】 d 先出锁框架，c b a 顺序定；e 来插队找空位，四个空位 B 选定。

【考频】 「出栈序列」在 2009–2022 年解析中出现 24 次 · 高频

第 3 题 · 循环队列队头队尾初值（正确答案：B）

题干与选项：

已知循环队列存储在一维数组 A[0..n-1]中，且队列非空时 front 和 rear 分别指向队头和队尾元素。若初始时队列为空，且要求第 1 个进入队列的元素存储在 A[0]处，则初始时 front 和 rear 的值分别是( )

- A. 0，0
- B. 0，n-1
- C. n-1，0
- D. n-1，n-1

入手点： 先想第 1 个元素存进 A[0] 时 front 和 rear 各指哪，再想入队操作 (rear+1)%n 对初值有什么要求。

专有名词从零解释：

- 先想“循环队列”：数组首尾接成一个圈来存队列，队尾到了数组末尾就绕回开头。
- 再想“front”：指向队头元素（最先出的那个）；再想“rear”：指向队尾元素（最后存进去的那个）；本题入队时先把 rear 往后挪一位 ((rear+1)%n)，再把新元素存进 rear 指的地方。
- 再想“队空和队满的区分”：光看 front 和 rear 的值，队空和队满可能长得一模一样，必须靠初值和规则把它们区分开。

考查知识点：

- 先想第 1 个元素存进 A[0]：存之前 rear 的初值是 n-1；入队时 rear 先挪到 (n-1+1)%n＝0，再把元素存进 A[0]；存完之后 front 还是 0（队头就是它），rear 指着 0（刚存进去的那个）。
- 再想入队操作：入队要做 (rear+1)%n；如果第 1 个元素存进 A[0] 之后 rear 指向 0，倒推回去，rear 的初值就得是 n-1（因为 (n-1+1)%n＝0）。
- 再想 front 的初值：插入只改 rear，不改 front；非空的时候 front 指向队头，所以 front 初值是 0。
- 再想组合：front＝0、rear＝n-1，就是 B。
- 拿生活例子验算：一排 n 个座位围成圈，第一个人坐 0 号座；队头牌子指着 0 号，队尾牌子本来指着 n-1 号（最后一个座），往后挪一位正好绕回 0 号——第 1 个人坐进去，两个牌子的位置就对上了。
- 边界情况：n＝1 时，n-1＝0，front 和 rear 初值都是 0，队空和队满确实分不开，需要另外用计数或标志位区分；这是小 n 的特殊情况，不影响本题的初值。
- 易混对子：不要把 front 和 rear 的初值都设成 0——两个指针初值相等，队空和队满就长一个样，分不开了；不要把 rear 初值设成 0——那样第 1 个元素就存不进 A[0] 了。

正确选项为什么对： front 初值 0、rear 初值 n-1，第 1 个元素正好存进 A[0]，就是 B，选 B。

错误选项为什么错＋举例：

- A（0,0）：front 和 rear 初值相等，队空和队满无法区分。教训：初值相等是循环队列的大忌。
- C（n-1,0）：front 初值是 n-1，不是 0；非空时 front 应该指向队头 0 号位置。教训：front 初值看队头，队头在 0 号，front 就是 0。
- D（n-1,n-1）：front 和 rear 初值又相等了，同样分不清队空和队满。教训：只要两个初值相等，区分问题就还在。

笔记本一句话： 第 3 题＋循环队列初值：第 1 个元素进 A[0]，front＝0、rear＝n-1，初值不能相等。

【图像】

![循环队列空队时 front=0、rear=n-1，首个元素入队后 rear 走到 0 落入 A[0] 的示意图](2011-diagrams/q03-circular-queue.svg)

【定位】 数据结构 · 第3章 栈和队列 · 循环队列

【易混对比】

- "rear 指向队尾元素"和"rear 指向队尾下一个空位"的区别：本题题干是前者，入队时 rear 先走一步再放元素；两种约定下初值和判空判满公式完全不同，做题先看题干用哪种。
- "B(0,n-1)"和"A(0,0)"的区别：front=rear 时分不清队空还是队满，A、D 都栽在这里；B 的 rear=n-1 让首个元素入队时恰好走到 A[0]。

首个元素要落在 A[0]，入队时 rear 必须从 n-1 走到 0，故初值 front=0、rear=n-1，选 B。

【坑位】

先看清题干的 rear 约定再动手：约定一变，初值和公式全变，本题是"rear 指向队尾元素"。

误选 A（0,0）：front=rear 无法区分队空队满。误选 D（n-1,n-1）：同样 front=rear，分不清空满。误选 C（n-1,0）：front=n-1≠0，队头指针指错了位置。

【套路】

- 第一步，确认 rear 约定：本题 rear 指向队尾元素，入队 = rear 先走一步再放元素。
- 第二步，倒推 rear 初值：首个元素要落在 A[0]，入队前 rear 必须在 n-1（走一步到 0）。
- 第三步，定 front 初值：front 指向队头元素，首个元素在 A[0]，front 初值为 0。对上选项 B。

【类比】

像环形跑道旁的发令员：rear 是"上一个队员站的位置"，要让第 1 个队员站上 0 号跑道，发令员开场必须站在 n-1 号跑道旁，鸣枪后向前走一步正好到 0。术语贴回去：发令员是"rear 指针"，0 号跑道是"A[0]"，鸣枪走一步是"入队时 rear=(rear+1)%n"。

【口诀】 front 为零 rear n-1，首个入队落 A[0]；rear 指向队尾元素，B 选项就是它。

【考频】 「循环队列」在 2009–2022 年解析中出现 2 次 · 低频

第 4 题 · 完全二叉树叶子结点数（正确答案：C）

题干与选项：

若一棵完全二叉树有 768 个结点，则该二叉树中叶结点的个数是( )

- A. 257
- B. 258
- C. 384
- D. 385

入手点： 叶子数＝总结点数－分支结点数；完全二叉树里最后一个分支结点的序号是总结点数的一半取整。

专有名词从零解释：

- 先想“完全二叉树”：除了最后一层，其他层都排满；最后一层的结点都靠左排，中间不留空位。
- 再想“叶子结点”：没有孩子的结点；再想“分支结点”：至少有一个孩子的结点。
- 再想“结点编号”：完全二叉树按从上到下、从左到右给结点编 1、2、3……号，编号和父子关系一一对应。

考查知识点：

- 先想分支结点数：总结点 768，最后一个分支结点的序号是 768/2 取整＝384，也就是说 1 到 384 号都是分支结点。
- 再想叶子数：叶子数＝768-384＝384，就是 C。
- 再想另一条路验证：设叶子数 n0、度为 1 的结点数 n1、度为 2 的结点数 n2，有 n＝n0+n1+n2 且 n0＝n2+1；完全二叉树结点总数是偶数时 n1＝1，代入得 768＝2n0-1+1，n0＝384。
- 再想按层验证：叶子只在最下面两层；最后一层有 768-511＝257 个结点（全是叶子），倒数第二层有 256 个结点，其中 129 个是最后一层结点的父亲，剩下 256-129＝127 个是叶子；257+127＝384。
- 拿生活例子验算：768 个人排成完全二叉树队形，前 384 个人后面都至少站着一个人（分支结点），后 384 个人后面没人（叶子），两拨人加起来正好 768。
- 边界情况：总结点是奇数时 n1＝0，叶子数＝(总结点+1)/2；是偶数时 n1＝1，叶子数＝总结点/2；本题 768 是偶数，直接除以 2 就是 384。
- 易混对子：不要把最后一个分支结点的序号算错——768/2＝384，不是 383 或 385；不要把叶子数当成总结点的一半减 1——偶数个结点的完全二叉树，叶子数正好是总结点的一半。

正确选项为什么对： 叶子数＝768-384＝384，就是 C，选 C。

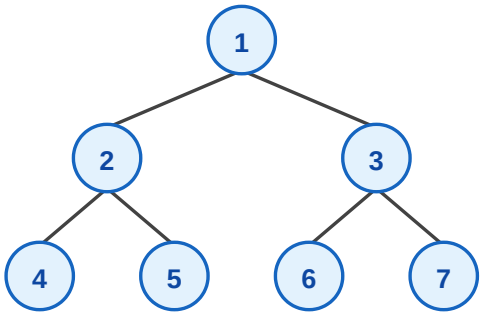
错误选项为什么错＋举例：

- 叶子数算成 383 或 385：最后一个分支结点序号算错了一位。教训：768/2 取整就是 384，叶子数＝768-384。
- 叶子数算成总结点的一半减 1：把奇数情况的公式套到偶数上了。教训：先看总结点是奇是偶，再选公式。

笔记本一句话： 第 4 题＋完全二叉树叶子：叶子数＝总结点－最后一个分支结点序号，768-384＝384。

【图像】

Q4 完全二叉树：768 个结点，叶子结点数？



层序编号 1…768

编号 1…384：分支结点（2i ≤ 768）

编号 385…768：叶子结点

n = n0 + n1 + n2

n0 = n2 + 1

n=768 为偶数 ⇒ n1 = 1

n0 = (768+1-1)/2 = 384

叶子结点数 = 384
答案：C

768 个结点的完全二叉树按层序编号、前 384 为分支结点、后 384 为叶子结点的示意图

【定位】 数据结构 · 第5章 树与二叉树 · 完全二叉树的性质

【易混对比】

- "n0=⌈n/2⌉"和"满二叉树叶子 (n+1)/2"的区别：后者只在总结点数是 2^k-1 时成立；完全二叉树一律用 ⌈n/2⌉，别套满二叉树的公式。
- "n 为偶数"和"n 为奇数"的区别：偶数时度为 1 的结点有 1 个（n1=1），奇数时 n1=0；列方程 n=n0+n1+n2 时 n1 不能瞎填。

n=768 为偶数，n1=1，n0=(768+1-1)/2=384，选 C。

【坑位】

先判 n 的奇偶再定 n1：768 是偶数，n1=1；奇偶判错，整道题全错。

误选 D（385）：把 n 当奇数用了 (n+1)/2，或把 n1 当成了 0。误选 B（258）：套了别的公式的数；教训：完全二叉树叶子数只认 ⌈n/2⌉，别的不认。

【套路】

- 第一步，列方程：n=n0+n1+n2，n0=n2+1。
- 第二步，定 n1：n=768 为偶数，最后一个分支结点只有左孩子，n1=1。
- 第三步，解 n0：n0=(768+1-1)/2=384，对上选项 C。

【类比】

像 768 个座位按层坐满：前 384 个人膝盖上都坐着孩子（分支结点），后 384 个人膝盖上没人，就是"终端"（叶子）。术语贴回去：膝盖上有没有孩子是"度数"，终端是"叶子结点"，⌈n/2⌉ 是"后一半的人数"。

【口诀】 完全二叉数叶子，总结点除二向上取；偶数 n1 是一别忘，三八四里 C 选项。

【考频】 「完全二叉树」在 2009–2022 年解析中出现 11 次 · 高频

第 5 题 · 由前序、后序序列判断不可能的中序序列（正确答案：C）

题干与选项：

若一棵二叉树的前序遍历序列和后序遍历序列分别为 1，2，3，4 和 4，3，2，1，则该二叉树的中序遍历序列不会是( )

- A. 1，2，3，4
- B. 2，3，4，1
- C. 3，2，4，1
- D. 4，3，2，1

入手点： 前序和后序刚好相反，说明每个结点只有一个孩子，整棵树是一条单链；单链里任一子树的中序中，子树根一定在端点上。

专有名词从零解释：

- 先想“前序”：先根、再左子树、再右子树；再想“后序”：先左子树、再右子树、再根；再想“中序”：先左子树、再根、再右子树。
- 再想“单链”：每个结点只有一个孩子，整棵树像一条链子一样抻下来，没有分叉。
- 再想“子树根在端点”：在单链里，一个子树的中序序列中，子树的根要么在最前面（子树全是右孩子链），要么在最后面（子树全是左孩子链），不可能夹在中间。

考查知识点：

- 先想前序和后序为什么相反：前序是 NLR（根左右），后序是 LRN（左右根）；两个序列刚好相反，说明每个结点不可能同时有左右孩子——只要有一个结点有两个孩子，前序和后序就不可能完全反过来；所以每个结点只有一个孩子，树高为 4，整棵树是单链。
- 再想四个中序序列：A 是 1,2,3,4；B 是 2,3,4,1；C 是 3,2,4,1；D 是 4,3,2,1。
- 再想为什么 A、B、D 都可能：A 是全右孩子链（根 1 在最前面）；D 是全左孩子链（根 1 在最后面）；B 是 2、3、4 这条链全是右孩子、再挂在 1 的左孩子上（2 的子树中序是 2,3,4，2 在端点，整棵树中序是 2,3,4,1）。
- 再想为什么 C 不可能：C 是 3,2,4,1；看 2 的子树，2 的子树里有 3、2、4 三个结点，中序是 3,2,4，2 被夹在中间——这要求 2 同时有左孩子 3 和右孩子 4，也就是 2 有两个孩子，和单链矛盾；所以 C 不可能是这棵树的中序序列，就是 C。
- 拿生活例子验算：一条链子从上往下是 1、2、3、4，每个结点只有一个孩子；中序遍历就是沿着链子“左进右出”地读，任何一个子链的头一定出现在子链读出顺序的两头，不可能被夹在中间。
- 边界情况：单链里整棵树的根一定在整条中序的端点上（右孩子链时在最前面，左孩子链时在最后面）；所以不能只看 1 在不在首尾——四个选项的 1 都在端点上，区分要看每个子树根是不是在它自己子树中序的端点上。
- 易混对子：不要写成“整个序列里 2 必须在首或尾”——A 的 2 在第 2 位却是可能的，因为 A 里 2 的子树是 2,3,4，2 在它自己子树中序的端点上；关键是子树根在子树中序里的位置，不是整条序列里的位置。

正确选项为什么对： C 在 2 的子树里把 2 放在中间，要求 2 有两个孩子，和单链矛盾，所以 C 不可能，就是 C，选 C。

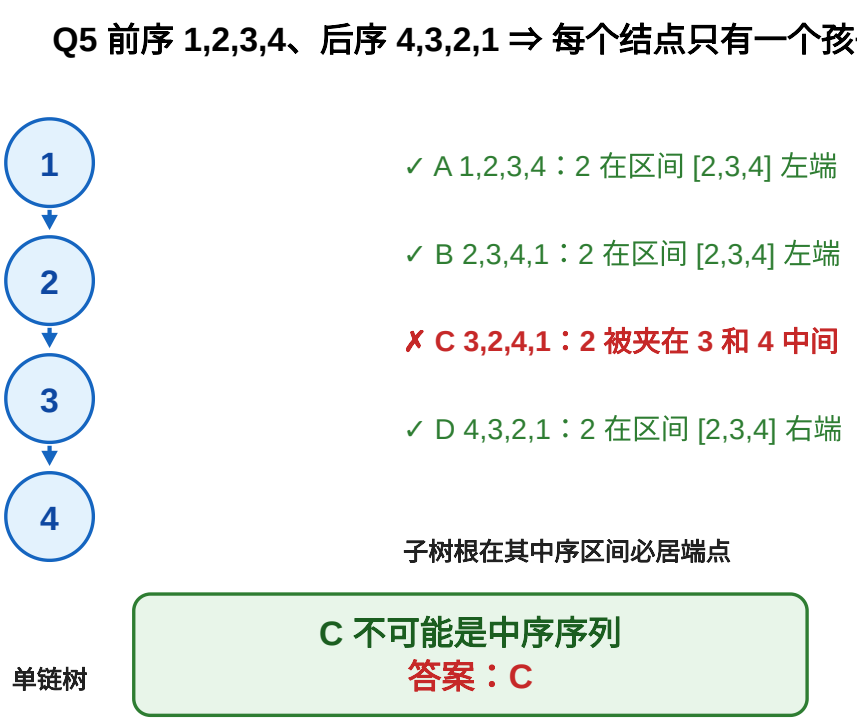
错误选项为什么错＋举例：

- A（1,2,3,4）：全右孩子链的中序，完全可能。教训：单链可以全往右长，中序就是 1、2、3、4。

- B（2,3,4,1）：2、3、4 全右孩子链挂在 1 的左孩子上，2 在它子树中序的端点上，完全可能。教训：看子树根在子树中序里的位置，不看整条序列。
- D（4,3,2,1）：全左孩子链的中序，完全可能。教训：单链可以全往左长，中序就是倒着的 4、3、2、1。

笔记本一句话： 第 5 题＋前序后序相反：树是单链，子树根必在子树中序端点，C 把 2 夹中间，不可能。

【图像】



前序 1,2,3,4、后序 4,3,2,1 对应的单链二叉树，以及 C 选项把子树根 2 放在中间的错误示意图

【定位】 数据结构 · 第5章 树与二叉树 · 二叉树的遍历

【易混对比】

- "子树根在端点"和"根在整个序列的首尾"的区别：约束只落在每个子树自己的中序区间里；A 的 2 在整个序列第 2 位，但它在自己子树区间 [2,3,4] 的左端点，合法。
- "前序+后序"和"前序+中序"的区别：前序+中序能唯一确定一棵树；前序+后序在单孩子结点处分不清左右，只能做排除法。

单链树里每个子树根在其中序区间必居端点，C 把 2 夹在 3 和 4 中间，不可能，选 C。

【坑位】

约束是"子树根在它自己子树的中序区间端点"，不是"在整个序列的首尾"：盯住区间，别盯全序列。

误选 A：把"2 在第 2 位"误判成"2 不在端点"；教训：端点是相对于子树区间 [2,3,4] 而言的，2 在左端点，合法。
误判思路"前序后序能唯一确定树"：单孩子结点的左右分不清，一组前序后序对应多种中序，只能排除。

【套路】

- 第一步，判树形：前序 1,2,3,4、后序 4,3,2,1，每个结点都只有一个孩子，是条单链。
- 第二步，取关键子树：2 的子树结点是 {2,3,4}，其根 2 在中序里必须在该区间的左端或右端。

- 第三步，逐个排除：A（2 在左端）对、B（2 在左端）对、D（2 在右端）对、C（3,2,4 把 2 夹在中间）错，选 C。

【类比】

像一串糖葫芦（单链）：每颗山楂是它下面那串的"根"，从中间剖开看（中序），根要么在剖下来的这段的最左、要么最右，不可能被包在中间。C 把 2 包在 3 和 4 中间，等于把签从中间折断了还说它是整串。术语贴回去：糖葫芦串是"单链二叉树"，剖面是"中序序列"，签不能断是"子树根必居端点"。

【口诀】 前序后序单链树，中序根在端点处；C 把 2 夹在当中，不可能里把它除。

【考频】 「二叉树遍历」在 2009–2022 年解析中出现 24 次 · 高频

第 6 题 · 树转为二叉树后无右孩子结点数（正确答案：D）

题干与选项：

已知一棵有 2011 个结点的树，其叶结点个数为 116，该树对应的二叉树中无右孩子的结点的个数是( )

- A. 115
- B. 116
- C. 1895
- D. 1896

入手点： 孩子兄弟表示法里，一个结点有右孩子，当且仅当它有右边的兄弟；没有右孩子的，就是每组兄弟里的最右孩子，再加上根。

专有名词从零解释：

- 先想“孩子兄弟表示法”：把树转成二叉树的方法——一个结点的第一个孩子变成它的左孩子，紧挨着的下一个兄弟变成它的右孩子。
- 再想“右孩子”：二叉树里某个结点的右指针指着的结点；按这个转法，右孩子就是原来的右边兄弟。
- 再想“分支结点”：至少有一个孩子的结点；再想“叶子结点”：没有孩子的结点。

考查知识点：

- 先想分支结点数：总结点 2011，叶子 116，分支结点＝2011-116＝1895。
- 再想没有右孩子的是哪些：每组兄弟里最右的那个孩子没有右边兄弟，转成二叉树后就没有右孩子；每组兄弟恰好出一个最右孩子，分支结点有 1895 个，所以最右孩子有 1895 个；再加上根（根没有父亲、也没有兄弟），一共 1895+1＝1896，就是 D。
- 再想另一条路验证：非根结点一共 2010 个，其中最右孩子 1895 个，所以有右边兄弟（也就是有右孩子）的结点是 2010-1895＝115 个；没有右孩子的＝2011-115＝1896。
- 拿生活例子验算：一家有 1895 个家长，每个家长的孩子排成一排，只有排最后的那个孩子右边没人；再加上全家最上面那个家长（根），右边没人的正好是 1895+1＝1896 个。
- 边界情况：根一定没有右孩子（根没有兄弟），这 1 个不能漏；只有一个结点的树，根既是叶子又没有右孩子，答案是 1，公式“分支结点数+1＝0+1＝1”同样成立。
- 易混对子：不要把叶子数 116 当成没有右孩子的结点数——叶子没有孩子，但叶子转成二叉树后可能有右孩子（只要它有右边兄弟）；不要漏了根——根没有兄弟，一定没有右孩子。

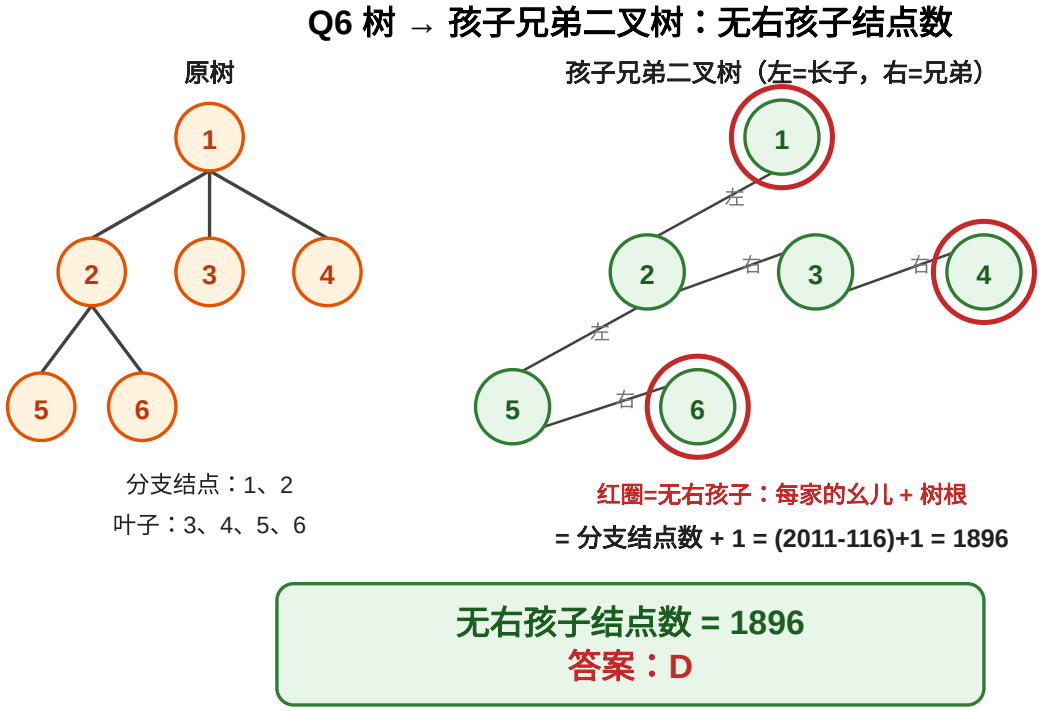
正确选项为什么对： 没有右孩子的＝最右孩子 1895 个＋根 1 个＝1896，就是 D，选 D。

错误选项为什么错＋举例：

- 算成 116：把叶子数当成了答案。教训：叶子和“没有右孩子”是两回事，叶子有右边兄弟时转成二叉树后就有右孩子。
- 算成 1895：漏了根。教训：根没有兄弟，一定没有右孩子，这 1 个要加上。
- 算成 2011-116＝1895 的其他变体：同样是漏根或把分支结点数算错。教训：分支结点＝2011-116＝1895，再加根才是 1896。

笔记本一句话： 第 6 题＋树转二叉树无右孩子：最右孩子＋根，1895+1＝1896。

【图像】



普通树转孩子兄弟二叉树、无右孩子结点等于分支结点数加一的示意图

【定位】 数据结构 · 第5章 树与二叉树 · 树与二叉树的转换

【易混对比】

- "无右孩子结点"和"叶子结点"的区别：二叉树里无右孩子包括叶子和只有左孩子的分支结点；原树的叶子转过去都无右孩子，但无右孩子的还有别人。
- "分支结点数"和"总结点数-1"的区别：分支结点=非叶子结点=2011-116=1895，别拿总结点直接算。

无右孩子结点数=分支结点数+1=(2011-116)+1=1896，选 D。

【坑位】

"无右孩子"不是"无孩子"：只有左孩子的分支结点也算，别只数叶子。

误选 C（1895）：忘了加 1；教训：树根在二叉树里没有右兄弟，无右孩子结点比"有兄弟的结点"多一个。误选 B（116）：把无右孩子当成了叶子。

【套路】

- 第一步，算分支结点：2011-116=1895 个非叶子结点。
- 第二步，套公式：孩子兄弟表示法里，无右孩子结点数=分支结点数+1。
- 第三步，得数：1895+1=1896，对上选项 D。

【类比】

像家族里"没有弟弟妹妹"的人：孩子兄弟表示法里，右孩子就是"下一个弟弟妹妹"。所以"无右孩子"=每个家庭的幺儿，再加上整棵树的根（根没有兄弟）。幺儿的数量=有孩子的家庭数（每个当父母的家庭恰出一个幺儿）=分支结点数 1895，加树根 1 个，共 1896。术语贴回去：幺儿是"无右孩子结点"，当父母的是"分支结点"。

【口诀】 树转二叉右孩子，兄弟排行下一位；无右孩子幺儿加树根，分支加一 D 落定。

【考频】 「树转二叉树」在 2009–2022 年解析中出现 28 次 · 高频

第 7 题 · 哪条查找序列不是二叉排序树查找序列（正确答案：A）

题干与选项：

对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的序列是( )

- A. 95，22，91，24，94，71
- B. 92，20，91，34，88，35
- C. 21，89，77，29，36，38
- D. 12，25，71，68，33，34

入手点： 二叉排序树里，左子树所有结点都比根小，右子树所有结点都比根大；沿着查找路径，每一步都要符合这个大小关系。

专有名词从零解释：

- 先想“二叉排序树”：左子树所有结点的值都比根小，右子树所有结点的值都比根大，子树也一样。
- 再想“查找序列”：从根出发找一个值，每一步按大小往左或往右走，走过的结点排成的序列。

考查知识点：

- 先想四个序列：A 是 95、22、91、24、94、71；B 是 92、30、91、34、88、35；C 是 21、89、77、29、36、38；D 是 12、25、71、68、33、34。
- 再想 A 为什么不行：从 95 出发，22 比 95 小，往左走到 22；91 比 22 大，往右走到 91；24 比 91 小，往左走到 24；94 比 24 大，往右走到 94——94 出现在 91 的左子树路径上，但 94 比 91 大，左子树里的结点都应该比 91 小，矛盾；所以 A 不是二叉排序树的查找过程，就是 A。
- 再想 B、C、D 为什么都行：每一步的大小关系都符合左小右大的规矩，没有哪一步在左子树路径上出现比祖先大的值，也没有在右子树路径上出现比祖先小的值。
- 拿生活例子验算：在一棵按大小排好的树里找人，往左走就只能碰到更小的人，往右走就只能碰到更大的人；A 的路径里，往左走到 91 之后又碰到了更大的 94，相当于往左走了之后又往回大了，规矩破了。
- 边界情况：查找序列的最后一个值不一定是树里已有的值（可能是查找失败停在某个位置），但本题的判断只看路径上的大小关系，不看最后一个值在不在树里。
- 易混对子：不要只看相邻两个值的大小——要看每个值和它路径上所有祖先的大小关系；不要以为“序列不是单调的”就是错的——查找路径本来就可以一会儿往左一会儿往右，关键是每一步相对祖先的方向对不对。

正确选项为什么对： A 在 91 的左子树路径上出现大于 91 的 94，违反左小右大，就是 A，选 A。

错误选项为什么错＋举例：

- B、C、D 的路径：每一步都符合左小右大，没有矛盾。教训：一条一条按祖先关系验，不要凭感觉。

笔记本一句话： 第 7 题＋二叉排序树查找：左小右大，A 在 91 左子树路径上出现 94，不可能。

【图像】

Q7 二叉排序树查找路径：95,22,91,24,94,71



每个转弯都给后面划界：左转定上界、右转定下界，界累积

（路径在 94 处已断裂，后面的 71 无需再看）

A 不可能是查找路径
答案：A

二叉排序树查找路径 95,22,91,24,94 中 94 大于 91 违反左子树上界的示意图

【定位】 数据结构 · 第7章 查找 · 二叉排序树

【易混对比】

- "查找路径"和"插入顺序"的区别：查找路径是沿比较方向走的单链，每个结点给后续值划定上下界；插入顺序只决定最后树形。
- "94>24 对"和"94>91 错"的区别：94 相对 24 是右子树方向没问题，但 24 在 91 的左子树里，91 给 24 的整个左子树定了上界 91，94 越界。

走到 91 后左转到 24，后续值必须小于 91，94 越界，故 A 不可能是查找路径。

【坑位】

每个转弯都给后面划一道"界"：左转定上界、右转定下界，界是累积的，后面的值要同时满足路上所有的界。

误判 B/C/D 不可能：逐个验界，B 全程满足（35 落在 (34,88) 里）；教训：别只看相邻两个数比大小，要看它相对路上所有祖先的位置。

【套路】

- 第一步，模拟走位：95（左转，上界 95）→22（右转，下界 22）→91（左转，上界 91）→24（右转，下界 24）。
- 第二步，验界：94 要同时满足 24<94<91，不满足，路径断裂。
- 第三步，下结论：A 不可能；再验 B、C、D 均合法，选 A。

【类比】

像玩"猜数字"：95 说"小了"，22 说"大了"，91 说"小了"——范围锁死在 (22,91)；24 说"大了"，范围变成 (24,91)；再报 94？出界了，主持人掀桌。术语贴回去：报数是"查找路径上的关键字"，"大了/小了"是"左转/右转"，锁死的范围是"上下界"。

【口诀】 查找路径界累积，左转上界右转下；九四越过九一线，A 序列里它先垮。

【考频】 「二叉排序树」在 2009–2022 年解析中出现 13 次 · 高频

第 8 题 · 图论三句话的正误组合（正确答案：C）

题干与选项：

下列关于图的叙述中，正确的是( )

Ⅰ. 回路是简单路径

Ⅱ. 存储稀疏图，用邻接矩阵比邻接表更省空间

Ⅲ. 若有向图中存在拓扑序列，则该图不存在回路

- A. 仅 Ⅱ
- B. 仅 Ⅰ、Ⅱ
- C. 仅Ⅲ
- D. 仅 Ⅰ、Ⅲ

入手点： 一句一句验：回路和简单路径的关系、稀疏图该用什么存、有回路的有向图有没有拓扑序列。

专有名词从零解释：

- 先想“回路”：起点和终点是同一个顶点的路径。
- 再想“简单路径”：除了起点和终点，中间顶点都不重复的路径；回路首尾相同，顶点重复了，所以回路不是简单路径。
- 再想“稀疏图”：边很少的图；再想“邻接矩阵”和“邻接表”：矩阵是 $n\times n$ 的表，不管边多边少都要占 n^2 的地方；邻接表只给每条边留地方，占 $n+e$ 的地方。
- 再想“拓扑序列”：把有向图的顶点排成一排，使得每条边都从前面的顶点指向后面的顶点；有回路的图排不出来。

考查知识点：

- 先想陈述 Ⅰ（错）：首尾相同的路径叫回路；回路首尾顶点重复，不符合简单路径“顶点不重复”的要求，所以回路必然不是简单路径；说“回路是简单路径”那句是错的。
- 再想陈述 Ⅱ（错）：稀疏图边很少，用邻接矩阵要占 $O(n^2)$ 的空间，大量位置是空的，浪费；邻接表只要 $O(n+e)$，应该选邻接表；说“稀疏图适合用邻接矩阵”那句是错的。
- 再想陈述Ⅲ（对）：有回路的有向图不存在拓扑序列；如果拓扑输出结束后还剩下顶点、且剩下的顶点都有前驱，说明只得到了部分顶点的拓扑序列，图中确实存在回路；这句是对的。
- 再想组合：只有Ⅲ正确，A 是仅 Ⅱ、B 是仅 Ⅰ、Ⅱ、C 是仅Ⅲ、D 是仅 Ⅰ、Ⅲ，就是 C。
- 拿生活例子验算：绕一圈回到起点的路（回路）不可能是“不走回头路”的路（简单路径）；边很少的图用大方阵存，等于租了个大仓库只放几件货，浪费；有环的任务依赖关系排不出先后顺序，排到一半卡住，说明确实有环。
- 边界情况：无回路的有向图一定有拓扑序列；自环（自己指向自己）也算回路，同样排不出拓扑序列。
- 易混对子：不要把“回路”和“简单回路”混在一起——简单回路要求除了首尾相同、中间顶点不重复，和本题说的“回路必然不是简单路径”是两回事；不要以为边少就适合矩阵——边越少，矩阵浪费得越厉害。

正确选项为什么对： 只有Ⅲ正确，组合是仅Ⅲ，就是 C，选 C。

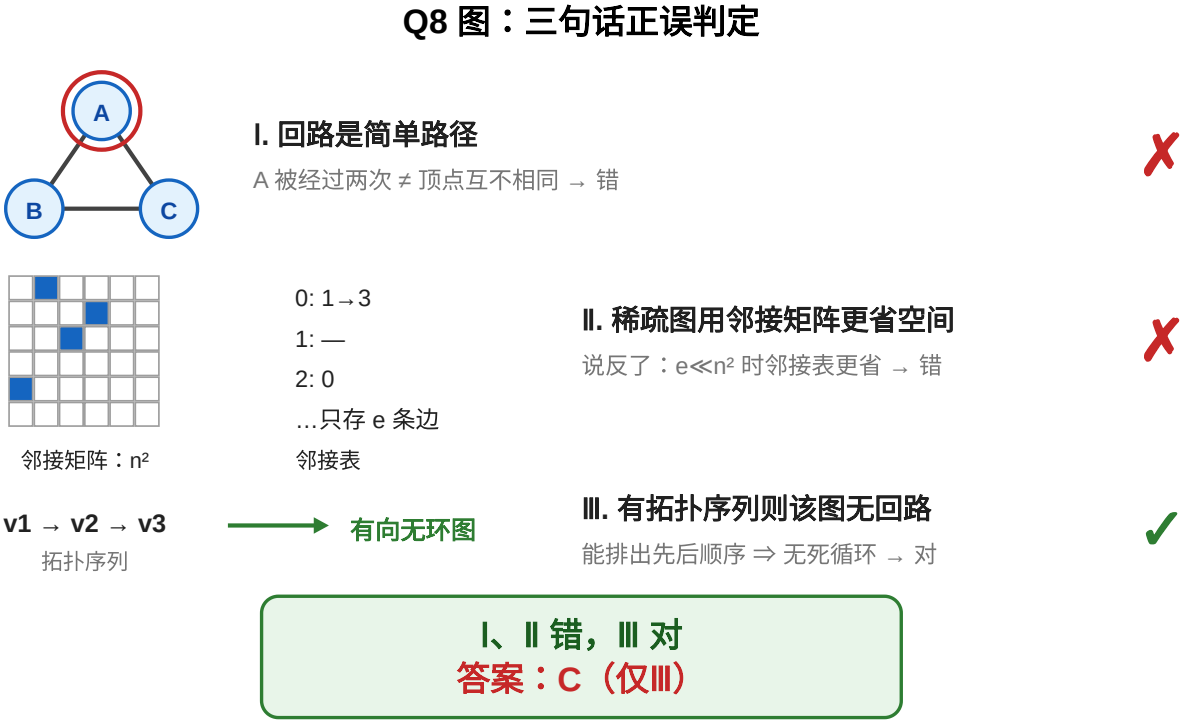
错误选项为什么错＋举例：

- A（仅 Ⅱ）：Ⅱ是错的——稀疏图用矩阵浪费空间，应该用邻接表。教训：边少选表，边密才考虑矩阵。

- B（仅 I 、 II）： I 和 II 都是错的。教训：回路不是简单路径，稀疏图不适合矩阵。
- D（仅 I 、III）： I 是错的，III虽然对，但组合里多了 I 。教训：组合题要一句一句验，多一句错的，整个组合就错。

笔记本一句话： 第 8 题＋图三句话：回路不是简单路径，稀疏图用邻接表，有回路无拓扑序列，只有III对。

【图像】



图论三句话正误判定：回路不是简单路径、稀疏图邻接表更省、拓扑序列对应无回路有向图的示意图

【定位】 数据结构 · 第6章 图 · 图的基本概念与存储

【易混对比】

- "回路"和"简单回路"的区别：简单路径要求除首尾外顶点互不相同；回路首尾顶点相同，已经破了"全不相同"， I 把二者混为一谈。
- "邻接矩阵"和"邻接表"在稀疏图下的区别：矩阵 n^2 个格子不管有没有边；表只存 e 条边，稀疏时 e 远小于 n^2， II 刚好说反。

I 、 II 错，III 对（有拓扑序列则必为有向无环图），选 C。

【坑位】

III 的逆命题也成立，但别顺手当已知用：无回路的有向图一定有拓扑序列；本题只考"有拓扑序列则无回路"这个方向。

误选 D（仅 I 、III）：把"回路是简单路径"当对；教训：简单路径定义里"顶点互不相同"，回路首尾顶点相同，直接出局。误选 A（仅 II ）：把"稀疏图用矩阵更省"当对；教训：矩阵是稠密图的菜，稀疏图用表。

【类比】

I 像说"环形跑道是直道"：跑一圈回到起点，起点被踩了两次，直道要求每一步都是新地方。 II 像给只有几辆车的小村子修 n^2 个车位（矩阵），不如按实际车数修车位（表）。III 像课程先修关系：能排出一个谁先谁后的完整顺

序，说明课程之间没有"互为先修"的死循环。术语贴回去：环形跑道是"回路"，车位是"存储空间"，先修顺序是"拓扑序列"。

【口诀】 回路首尾重合非简单，稀疏存图用表省空间；拓扑一出无回路，C 选项里III 单显。

【考频】 「图」在 2009–2022 年解析中出现 11 次 · 高频

第 9 题 · 散列表查找效率三句话的正误组合（正确答案：D）

题干与选项：

为提高散列（Hash）表的查找效率，可以采取的正确措施是( )

Ⅰ. 增大装填（载）因子

Ⅱ. 设计冲突（碰撞）少的散列函数

Ⅲ. 处理冲突（碰撞）时避免产生聚集（堆积）现象

- A. 仅 Ⅰ
- B. 仅 Ⅱ
- C. 仅 Ⅰ、Ⅱ
- D. 仅 Ⅱ、Ⅲ

入手点： 散列表查得快不快，看三样东西：散列函数好不好、冲突处理得好不好、表装得满不满；一句一句对上去。

专有名词从零解释：

- 先想“散列表”：按关键字算出一个位置、直接把记录放过去的表；查找时先算位置，再到那个位置去找。
- 再想“散列函数”：把关键字变成位置编号的函数；好的散列函数算出来的位置分散，不容易撞到一起。
- 再想“冲突”：两个不同的关键字算出同一个位置，这叫冲突；冲突多了，查找就要在同一个位置翻好几个记录，变慢。
- 再想“装填因子”：表里已经放的记录数除以表长；装得越满，越容易冲突。
- 再想“聚集”：好多记录挤在连续的一片位置上，查找时要顺着翻很久；拉链法把冲突的记录串成链表，不会聚集；线性探测法往后一个一个找空位，容易聚集。

考查知识点：

- 先想陈述 Ⅰ（错）：冲突概率和装填因子成正比，表装得越满，越容易冲突；说“装填因子和冲突没关系”或者反着说的那句是错的（注：陈述 Ⅰ 的逐字表述在源材料中未齐，这里只写它错的方向，不编造原文）。
- 再想陈述 Ⅱ（对）：设计产生冲突较少的散列函数，能让记录分散开，查找效率就高；这句是对的。
- 再想陈述Ⅲ（对）：采用合适的处理冲突方法可以避免聚集、提高效率；拉链法把冲突的记录串成链表，不存在聚集；线性探测法容易引起聚集；所以处理方法选得好，效率就高；这句是对的。
- 再想组合：Ⅱ和Ⅲ正确，A 是仅 Ⅰ、B 是仅 Ⅱ、C 是仅 Ⅰ、Ⅱ、D 是仅 Ⅱ、Ⅲ，就是 D。
- 拿生活例子验算：散列表像一排带编号的储物柜，散列函数是“按名字算柜号”的规矩；规矩定得好，人就分散在各个柜子；柜子装得越满，两个人分到同一个柜号的可能越大；柜号撞了之后，是串成一串挂在柜子边上（拉链法），还是往后一个一个找空柜子（线性探测），后者容易挤成一片。
- 边界情况：装填因子很小时，冲突很少，查找接近一次到位；装填因子接近 1 时，冲突多，查找退化成翻链表或顺着找空位，效率大降。
- 易混对子：不要把“散列函数”和“处理冲突的方法”当成一回事——前者是算位置的规矩，后者是撞了之后怎么办；不要把拉链法和线性探测法的聚集情况说反——拉链法不聚集，线性探测法容易聚集。

正确选项为什么对： Ⅱ（散列函数冲突少）和Ⅲ（处理方法避免聚集）都对，组合是仅 Ⅱ、Ⅲ，就是 D，选 D。

错误选项为什么错＋举例：

- A（仅Ⅰ）：Ⅰ是错的——冲突概率和装填因子成正比，装得越满越容易冲突。教训：装填因子是影响冲突的关键因素，不能忽略。
- B（仅Ⅱ）：漏了Ⅲ——合适的处理冲突方法同样能避免聚集、提高效率。教训：三句话要一句一句验，漏一句对的，组合就错。
- C（仅Ⅰ、Ⅱ）：多了错的Ⅰ，漏了对的Ⅲ。教训：组合题里多一句错的、少一句对的，都不对。

笔记本一句话： 第 9 题＋散列表效率：散列函数冲突少、处理方法避聚集、装填越满越易冲突，Ⅱ、Ⅲ对。

【图像】



散列表装填因子高低对比、冲突少的散列函数、链地址法避免聚集的示意图

【定位】 数据结构 · 第7章 查找 · 散列表

【易混对比】

- "增大装填因子"和"减小装填因子"的区别：装填因子=表中记录数/表长，越大表越满、冲突越多、查找越慢；Ⅰ说反了。
- "链地址法"和"开放定址法"在聚集上的区别：拉链法冲突的元素挂在各自链表上，互不干扰，无聚集问题；开放定址法（尤其线性探测）会产生"堆积"，Ⅲ的"避免聚集"拉链法天然满足。

Ⅰ错（装填越满冲突越多），Ⅱ、Ⅲ对，选 D。

【坑位】

本题最大坑：不少解析因Ⅲ的"避免"二字过于绝对而只选Ⅱ（B），但官方答案是 D——考试以官方为准，Ⅲ算对。

误选 B（仅Ⅱ）：被"避免聚集"的绝对化表述吓退；教训：链地址法确实无聚集现象，Ⅲ在官方口径下成立，记住答案 D。误选 A、C（含Ⅰ）：把"装填因子大=表利用率高=效率高"混为一谈；教训：利用率高省的是空间，代价是查找变慢。

【类比】

散列表像停车场：装填因子是"车位占用率"——停得越满，新车越难找空位（Ⅰ 错）；好的散列函数像公平的保安，把车均匀引到各区（Ⅱ 对）；处理冲突时别让车都挤在入口附近排长队（聚集），拉链法像给每个车位配了立体车库，互不挡道（Ⅲ 对）。术语贴回去：车位占用率是"装填因子"，排长队是"聚集（堆积）现象"。

【口诀】 装填因子别增大，冲突少了效率佳；拉链聚集本没有，D 选项 Ⅱ Ⅲ 拿。

【考频】 「散列表」在 2009–2022 年解析中出现 24 次 · 高频

第 10 题 · 快速排序适用的存储结构（正确答案：A）

题干与选项：

为实现快速排序算法，待排序序列宜采用的存储方式是( )

- A. 顺序存储
- B. 散列存储
- C. 链式存储
- D. 索引存储

入手点： 快速排序的过程里既要从后往前找、又要从前往后找；哪种存储结构能让它前后都方便地走？

专有名词从零解释：

- 先想“顺序存储”：数据按顺序排在一片连续的地方，第几个元素直接按下标找到，不用一个一个顺着找。
- 再想“链式存储”：数据散在各处，靠指针一个一个串起来；要找第几个元素，得从头一个一个顺着走。
- 再想“快速排序”：每趟挑一个枢轴，把比它小的放左边、比它大的放右边，枢轴就站在最终位置上；然后对左右两边分别再排。

考查知识点：

- 先想快速排序的查找方向：每趟里既要从后向前找比枢轴小的，也要从前向后找比枢轴大的，两个方向都要走。
- 再想为什么是顺序存储：顺序存储按下标直接访问，前后两个方向都是一步到位；链式存储只能从头顺着走，往后找要从头重走，效率差；所以快速排序宜采用顺序存储，就是 A。
- 再想一般规律：绝大部分内排序只适用于顺序存储，因为内排序经常要随机访问、交换任意两个位置的元素，链式存储做这些都很别扭。
- 拿生活例子验算：快速排序像整理一排书架上的书，左右两边同时往中间比对、换位置；书架是顺序排好的（顺序存储），伸手就能拿到任何一本；如果书是串成一串的（链式存储），要拿中间一本得从头一本一本翻，整理起来就慢了。
- 边界情况：链表版的快速排序不是不存在，但要靠指针操作硬做，效率不如顺序存储；题目问的是“宜采用”，选最合适的那种。
- 易混对子：不要以为“能用链式存储排序”就等于“适合链式存储”——能跑和跑得合适是两回事；不要把快速排序和归并排序的存储要求混在一起——归并排序对链式存储相对友好，快速排序不是。

正确选项为什么对： 快速排序前后两个方向都要查找，顺序存储按下标直接访问最合适，就是 A，选 A。

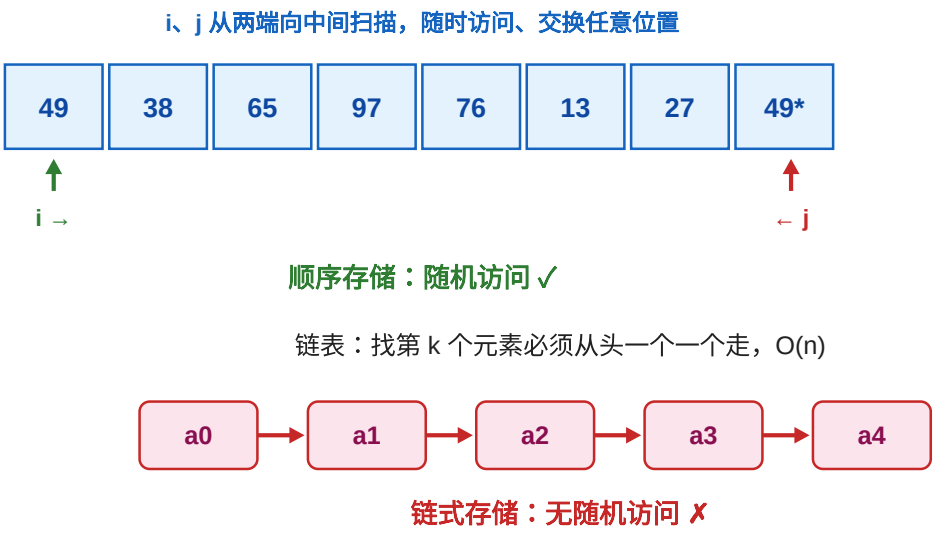
错误选项为什么错＋举例：

- 选链式存储：链式存储只能从头顺着走，往后找要从头重走，快速排序的双向查找做起来别扭。教训：双向随机访问是顺序存储的强项，不是链式存储的。

笔记本一句话： 第 10 题＋快排存储：前后双向查找，顺序存储按下标直达，选顺序存储。

【图像】

Q10 快速排序：待排序序列宜采用的存储方式



快速排序依赖随机访问 → 答案：A（顺序存储）

快速排序划分过程 i、j 指针在数组两端随机访问、链式存储无法直接定位的对比示意图

【定位】 数据结构 · 第8章 排序 · 快速排序

【易混对比】

- "顺序存储"和"链式存储"对快排的区别：快排的划分要 i、j 从两端随机跳、交换任意位置元素；链表找第 k 个元素得从头走，随机访问是硬伤。
- "快排"和"归并"在存储上的区别：归并（链表版）可以顺序走，链式也行；快排非随机访问不可，只能顺序存储。

快速排序依赖随机访问，宜采用顺序存储，选 A。

【坑位】

别被"链表插入删除快"带偏：快排的核心操作是"按下标随机访问和交换"，不是插入删除。

误选 C（链式存储）：只记得链表操作灵活，忘了快排划分时 i、j 指针要反复跳到任意位置；教训：链表定位第 k 个元素是 O(n)，快排的优势全丢。

【类比】

像整理书架：快排的划分是左右两只手同时从两端往中间翻书、随时交换任意两本书——书架（数组）伸手就能够；链表像一串钥匙，要拿到第 k 把必须从第 1 把一根根数过去。术语贴回去：伸手能够是"随机访问"，数钥匙是"顺序访问"。

【口诀】 快排划分双指针，两端跳跃要随机；链表定位步步走，顺序存储 A 牢记。

【考频】 「快速排序」在 2009–2022 年解析中出现 17 次 · 高频

第 11 题 · 堆插入新元素的比较次数（正确答案：B）

题干与选项：

已知序列 25，13，10，12，9 是大根堆，在序列尾部插入新元素 18，将其再调整为大根堆，调整过程中元素之间进行的比较次数是( )

- A. 1
- B. 2
- C. 4
- D. 5

入手点： 堆插入是把新结点先放在最后，再一层一层往上和父亲比；比一次，决定换不换，再往上比。

专有名词从零解释：

- 先想“堆”：一种完全二叉树，父亲和孩子之间有大小规矩（大顶堆是父亲比孩子大，小顶堆是父亲比孩子小），整棵树从上到下按这个规矩排。
- 再想“堆插入”：新结点先放在完全二叉树的最后一个位置，再和父亲比较；如果违反规矩就交换位置，换完再和新的父亲比，直到规矩满足或到了根。

考查知识点：

- 先想第一步：插入结点 18 之后，先和 10 比较；比较结果是需要交换位置，换完之后 18 往上走了一层。
- 再想第二步：18 再和 25 比较；这次不需要交换，插入过程就停下来了。
- 再想一共几次：一共比较 2 次，就是 B。
- 拿生活例子验算：新来的 18 先和门口的 10 比，比赢了往里走一格；再和里面的 25 比，比输了就停在原地——一共比了 2 场。
- 边界情况：如果新结点一路比到根，每一步都交换，比较次数就是树的高度；如果第一步就不交换，比较次数就是 1 次；本题是 2 次，停在中间。
- 易混对子：不要把“交换次数”和“比较次数”混在一起——本题交换 1 次、比较 2 次，题目问的是比较次数；不要以为比较到根才停——只要有一次不交换，马上就停，不用比到根。

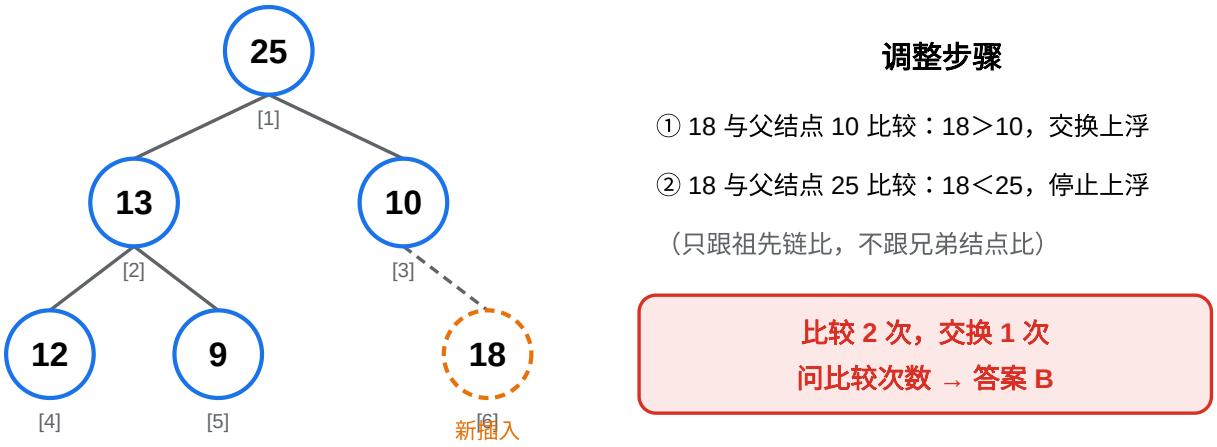
正确选项为什么对： 先和 10 比（交换），再和 25 比（不交换），一共 2 次，就是 B，选 B。

错误选项为什么错＋举例：

- 算成 1 次：只数了交换的那次，漏了和 25 比的那次。教训：不交换的那次比较也要数，比较次数包括最后停下来的那一次。
- 算成 3 次或更多：以为要比到根才停。教训：一次不交换就停，不用比到根。

笔记本一句话： 第 11 题＋堆插入比较：和 10 比换位置，和 25 比停下来，共 2 次。

【图像】



大根堆尾部插入18后的两次比较与上浮调整示意图

【定位】 数据结构 · 第9章 排序 · 堆的插入与调整

【易混对比】

- "比较次数"和"交换次数"的区别：比较是两个元素比大小（每上一层都要比一次），交换是比较之后才可能发生的动作；本题比较 2 次、交换 1 次，题目问的是比较次数。
- "尾部插入上浮"和"建堆下沉筛选"的区别：插入新元素是从尾部沿父链向上调整（sift-up），只跟祖先结点比；建堆是从下往上逐层向下筛选（sift-down）。本题走上浮路线。
- "大根堆"和"小根堆"的区别：只是比较方向反了（大根堆是子结点大于父结点才交换），比较次数的计算方法完全一样。

新元素 18 从尾部沿父链上浮：18 与父结点 10 比较（大，交换），再与 25 比较（小，停止），共比较 2 次，选 B。

【坑位】

比较次数不等于交换次数；停下来确认"不用换"的那一次比较也要计数。

误选 A（1 次）：只数了发生交换的那一次比较，漏了最后"比了但不换"的确认比较——上浮停止本身就是一次比较的结果。误选 C、D：把堆里不相关的比较都算了进来；新元素只跟自己祖先链上的结点比，不跟兄弟结点比。

【套路】

- 第一步，定位插入位置：新元素 18 放在数组尾部（第 6 个位置），其父结点是第 3 个位置的元素 10。
- 第二步，沿父链上浮比较：18 与 10 比（18 大，交换上浮）；18 到达第 3 个位置，再与父结点 25 比（18 小，停止）。
- 第三步，数比较不数交换：共进行了 2 次比较（1 次交换），题目问比较次数，对上选项 B。

尾部插入只跟父链比，每上一层计一次比较，数比较不数交换。

【类比】

像新兵入营按身高排队"向上插队"：新来的 18 先站在队尾，只跟自己正前方的组长比身高，赢了就换位，再跟上一级组长比，输了就地停下。每见一个组长比一次，一共比 2 次。术语贴回去：组长是"父结点"，换位是"交换"，比身高是"比较"。

【口诀】 尾部插入往上爬，父子比较定去留；比到父大停下脚，数清比较选 B 项。

【考频】 「堆」在 2009–2022 年解析中出现 19 次 · 高频

第 12 题 · 计算机性能指标 MIPS、CPI、IPC、MFLOPS 的含义（正确答案：D）

题干与选项：

下列选项中，描述浮点数操作速度指标的是( )

- A. MIPS
- B. CPI
- C. IPC
- D. MFLOPS

入手点： 四个指标各管一摊：MIPS 管每秒多少百万条指令，CPI 管每条指令几个周期，IPC 管每个周期几条指令，MFLOPS 管每秒多少百万次浮点运算。

专有名词从零解释：

- 先想“MIPS”：每秒执行多少百万条指令，用来衡量标量机（一次执行一条指令的机器）的性能。
- 再想“CPI”：平均每条指令要几个时钟周期；再想“IPC”：CPI 的倒数，每个时钟周期能执行几条指令。
- 再想“MFLOPS”：每秒执行多少百万次浮点运算，用来描述浮点运算速度，衡量向量机（一次能算一串数据的机器）的性能。
- 再想“标量机和向量机”：标量机一次处理一个数，向量机一次处理一串数；衡量浮点运算速度要用 MFLOPS，不是 MIPS。

考查知识点：

- 先想四个指标的定义：MIPS＝每秒百万条指令，衡量标量机；CPI＝每条指令的时钟周期数；IPC＝每个时钟周期的指令数，是 CPI 的倒数；MFLOPS＝每秒百万次浮点运算，衡量向量机浮点运算速度。
- 再想答案：说 MFLOPS 的那个选项就是 D（注：A、B、C 与 MIPS、CPI、IPC 的字母对应在源材料中未齐，这里只按四个指标的定义讲，不硬编对应关系）。
- 拿生活例子验算：MIPS 像“每秒写几个字”，CPI 像“写一个字要几秒”，IPC 像“一秒能写几个字”，MFLOPS 像“每秒算几道小数题”；问“每秒算几道小数题”的是哪个，当然是 MFLOPS。
- 边界情况：MIPS 高不等于浮点算得快——整数指令多、浮点指令少的机器，MIPS 可以很高，MFLOPS 却不高；两个指标衡量的东西不一样。
- 易混对子：不要把 MIPS 和 MFLOPS 混在一起——MIPS 数的是指令，MFLOPS 数的是浮点运算；不要把 CPI 和 IPC 弄反——CPI 是每条指令几个周期，IPC 是每个周期几条指令，两个互为倒数。

正确选项为什么对： MFLOPS 是每秒百万次浮点运算、衡量向量机浮点速度，说这个的就是 D，选 D。

错误选项为什么错＋举例：

- 选成 MIPS：MIPS 数的是指令条数，不是浮点运算次数。教训：问浮点运算速度，找 MFLOPS，不找 MIPS。
- 选成 CPI 或 IPC：CPI、IPC 说的是指令和周期的关系，不说浮点运算。教训：四个指标各管一摊，对号入座。

笔记本一句话： 第 12 题＋性能指标：MIPS 数指令，CPI 数周期，IPC 是倒数，MFLOPS 数浮点运算，选 MFLOPS。

【图像】

四个性能指标对比（题眼：浮点数操作速度）



题眼「浮点数操作速度」→ 后缀含 FLOP（浮点运算）的只有 D

MIPS 的 I 是指令，MFLOPS 的 FLOP 才是浮点

MIPS、CPI、IPC、MFLOPS四个性能指标含义对比卡，MFLOPS高亮为浮点速度指标

【定位】 计算机组成原理 · 第1章 计算机系统概述 · 计算机的性能指标

【易混对比】

- "MIPS"和"MFLOPS"的区别：MIPS 是每秒执行百万条指令（所有指令都算，衡量标量机整体执行速度）；MFLOPS 是每秒执行百万次浮点运算（只算浮点运算，衡量向量机/浮点部件）。题目问的是"浮点数操作速度"，只能是 MFLOPS。
- "CPI"和"IPC"的区别：CPI 是执行一条指令平均需要的时钟周期数，IPC 是一个时钟周期平均执行的指令条数，两者互为倒数；都是描述"指令—时钟周期"关系的指标，与浮点运算无关。

题眼是"浮点数操作速度"六个字：只有 MFLOPS 的 FLOP 是浮点运算，选 D。

【坑位】

MIPS 的 I 是 Instruction（指令），MFLOPS 的 FLOPS 是 Floating-point Operations Per Second（浮点运算）；别被两个词共有的首字母 M 迷惑，关键看后半截。

【套路】

- 第一步，读题眼："浮点数操作速度"——锁定"浮点"二字。
- 第二步，看后缀：四个选项里只有 D 含有 FLOP（浮点运算），直接锁定。
- 第三步，排除验证：MIPS 管指令条数，CPI/IPC 管指令与周期的关系，都不沾浮点。

见"浮点速度"找 FLOP 后缀，其他三个指标都不沾浮点。

【类比】

MIPS 像"每分钟敲键盘多少下"（所有按键都算），MFLOPS 像"每分钟算对多少道数学题"（只算数学题）。题目问的是"数学题算得快不快"，当然看后者。术语贴回去：敲键盘是"执行指令"，算数学题是"浮点运算"。

【口诀】 浮点速度看 FLOPS，MIPS 只管指令数；CPI、IPC 周期账，D 选项里找 MFLOPS。

【考频】 「性能指标」在 2009–2022 年解析中出现 9 次 · 中频

第 13 题 · IEEE 754 单精度浮点数表示（正确答案：A）

题干与选项：

float 型数据通常用 IEEE 754 单精度浮点数格式表示。若编译器将 float 型变量 x 分配在一个 32 位浮点寄存器 FR1 中，且 x=-8.25，则 FR1 的内容是( )

- A. C104 0000H
- B. C242 0000H
- C. C184 0000H
- D. C1C2 0000H

入手点： 先把 x 写成 1.小数×2 的几次方，定出阶码；再按“数符 1 位、阶码 8 位、尾数 23 位、最高位 1 藏起来”的格式往里填。

专有名词从零解释：

- 先想“IEEE 754 单精度”：存小数的一种标准格式，一共 32 位：数符 1 位（0 正 1 负）、阶码 8 位、尾数 23 位。
- 再想“阶码和偏置”：阶码存的不是真正的指数，而是真正的指数加上 127（偏置值）；真正的指数是 3，存进去就是 3+127＝130。
- 再想“隐藏的最高位 1”：二进制小数写成 1.小数×2 的几次方时，整数部分的 1 一定存在，不用存进尾数，尾数只存小数点后面的位；这样尾数 23 位实际表示了 24 位精度。

考查知识点：

- 先想 x 的规格化：x＝-1000.01＝-1.00001×2^3；数符是 1（负数），真正的指数是 3。
- 再想阶码：E-127＝3，E＝130，写成 8 位二进制是 10000010。
- 再想尾数：1.00001 的整数部分 1 藏起来，尾数只存小数点后面的 00001，再补零凑够 23 位，就是 0000 1000 0000 0000 0000 000。
- 再想完整结果：数符 1、阶码 10000010、尾数 0000 1000 0000 0000 0000 000，拼起来是 1；1000 0010；0000 1000 0000 0000 0000 000，写成十六进制是 C1040000H，就是 A。
- 拿生活例子验算：-1000.01 像“-8.25”这个数，先写成 -1.00001×2^3（小数点往左挪 3 位），符号位记负、阶码记 130、尾数记 00001 后面补零；三段拼起来就是寄存器的内容。
- 边界情况：指数是 0 时阶码存 127；尾数全零时表示的是 1.0×2 的几次方；最高位 1 藏起来这个规矩只在规格化数里用，非规格化数（阶码全零）不适用。
- 易混对子：不要把偏置值当成 128——单精度偏置是 127，不是 128；不要把隐藏的 1 也写进尾数——尾数只存小数点后面的位，整数部分的 1 不占位置。

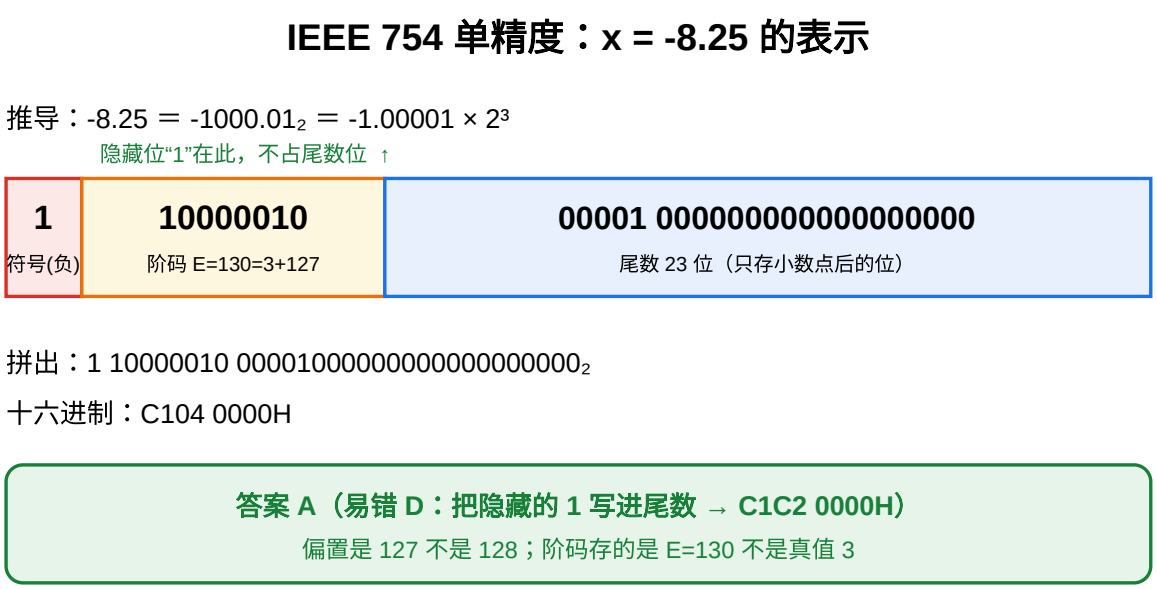
正确选项为什么对： 数符 1、阶码 10000010、尾数 0000 1000 0000 0000 0000 000，十六进制 C1040000H，就是 A，选 A。

错误选项为什么错＋举例：

- D（未考虑隐藏最高位 1、把偏置值当成 128）：尾数里多写了一个 1，阶码也算错了。教训：最高位 1 藏起来不存，偏置是 127 不是 128。
- 阶码算错的其他选项：指数 3 加偏置 127 是 130，不是 131 或 129。教训：阶码＝真正指数＋127，一步一步算。

笔记本一句话： 第 13 题＋IEEE 754 单精度：数符 1 位、阶码＝指数＋127、尾数藏起最高位 1，C1040000H。

【图像】



IEEE 754单精度32位格式划分：1位符号、8位阶码、23位尾数，标注-8.25的转换与隐藏位

【定位】 计算机组成原理 · 第2章 数据的表示与运算 · IEEE 754 浮点数标准

【易混对比】

- "隐藏位"存与不存的区别：IEEE 754 规格化数默认最高位是 1，这个 1 不占用尾数位（隐藏起来），尾数只存小数点后的位。本题尾数存的是 00001…，而不是 100001…。
- "偏置 127"和"偏置 128"的区别：单精度浮点数的偏置常数是 127（阶码 E＝真值 e＋127），不是 128；本题 e＝3，故 E＝130＝10000010。
- "阶码真值 e"和"移码 E"的区别：e＝3 是指数的真值，E＝130 才是真正存进格式字段的移码。

x＝-8.25＝-1000.01_2＝-1.00001×2^3，阶码 E＝3＋127＝130＝10000010，隐藏最高位 1，拼出 1|10000010|000010…＝C104 0000H，选 A。

【坑位】

规格化后隐藏的那个"1"不进尾数位；尾数只存小数点后面的位。

误选 D（C1C2 0000H）：没有隐藏最高位的 1，把 1.00001 的整数部分 1 也写进了尾数字段，导致尾数变成 100001…。教训：规格化形式 1.f 中，存的永远只是 f。

【套路】

- 第一步，转二进制小数：8.25＝1000.01_2，x 为负，数符为 1。
- 第二步，规格化并隐藏：1.00001×2^3，最高位的 1 隐藏，尾数取 00001（后补 0 至 23 位）。
- 第三步，算阶码：E＝3＋127＝130＝10000010_2。
- 第四步，拼 32 位：1|10000010|00001000000000000000000＝C104 0000H，对上选项 A。

转二进、规格化藏 1、阶码加 127、拼三段，对十六进制验算。

【类比】

隐藏位像"默认大家都知道的规则"：规格化数的整数部分永远是 1，就像默认每句话都有主语一样，不用浪费纸面再写一遍，省下来的位置多记一位小数。术语贴回去：不写出来的 1 是"隐藏位"，省下的位置是"尾数精度"。

【口诀】 负八点二五先转二，规格化后藏个一；阶码三加一百二七，拼成 C104 选 A 项。

【考频】 「IEEE 754」在 2009–2022 年解析中出现 15 次 · 高频

第 14 题 · 哪种存储器不是随机存取方式（正确答案：B）

题干与选项：

下列各类存储器中，不采用随机存取方式的是( )

- A. EPROM
- B. CD-ROM
- C. DRAM
- D. SRAM

入手点： 随机存取的意思是：不管要找的数据在哪个物理位置，找它的时间都一样；光盘找数据要转着找，时间看位置，就不是随机存取。

专有名词从零解释：

- 先想“随机存取”：CPU 想找哪个存储单元，直接按地址找到，找的时间和这个单元在哪个物理位置没关系。
- 再想“顺序存取”：要按顺序一个一个往后找，找的时间取决于数据在哪个位置，离得远就找得久。
- 再想四种存储器：EPROM（可擦可编程的只读存储器）、CD-ROM（只读型光盘存储器，原卷写作 CDROM）、DRAM（动态随机存取存储器）、SRAM（静态随机存取存储器）。

考查知识点：

- 先想 A、C、D：EPROM、DRAM、SRAM 都是随机存取——按地址直接找到，时间和物理位置没关系。
- 再想 B：CD-ROM 是光盘，读数据要靠光头转着找，找的时间取决于数据在盘上的位置，是顺序存取，不是随机存取，就是 B。
- 再想一个提醒：CD-ROM 是只读型光盘存储器，不是只读存储器 ROM；名字里有个 ROM，但它属于光盘那一类，不属于半导体 ROM 那一类。
- 拿生活例子验算：随机存取像从书架上按编号直接抽一本书，不管书在第几排，伸手就拿到；顺序存取像听磁带找一首歌，得从头转到那个位置，歌越靠后等得越久；光盘就是后者。
- 边界情况：光盘里也有按地址找的办法（比如按扇区号），但物理上还是要转盘、移光头，时间仍然和位置有关，所以还是顺序存取。
- 易混对子：不要因为 CD-ROM 名字里有 ROM 就把它当成半导体存储器——它是光盘，存取方式是顺序的；不要把 DRAM、SRAM 的“随机存取”四个字只当成名字——它们确实是随机存取的。

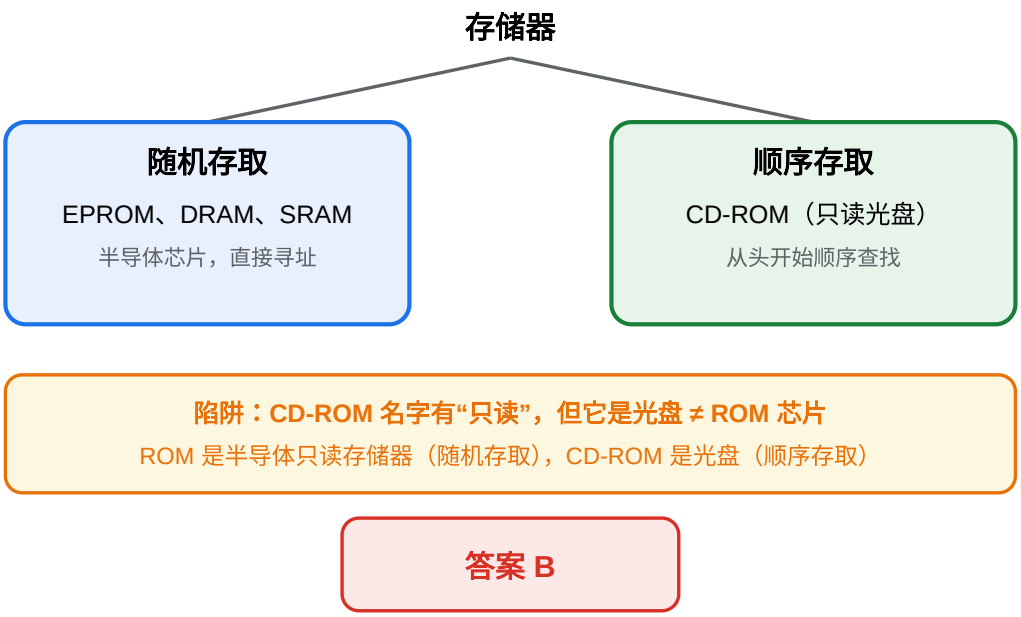
正确选项为什么对： CD-ROM 是光盘、顺序存取，四种里只有它不是随机存取，就是 B，选 B。

错误选项为什么错＋举例：

- A（EPROM）、C（DRAM）、D（SRAM）：这三个都是按地址直接找、时间和位置没关系的随机存取。教训：半导体存储器基本都是随机存取，光盘不是。

笔记本一句话： 第 14 题＋随机存取：EPROM、DRAM、SRAM 都是随机存取，CD-ROM 是光盘顺序存取。

【图像】



存储器按存取方式分类树：随机存取分支含EPROM、DRAM、SRAM，顺序存取分支含CD-ROM光盘，标注命名陷阱

【定位】 计算机组成原理 · 第3章 存储系统 · 存储器的分类

【易混对比】

- "CD-ROM"和"ROM"的区别：ROM 是半导体只读存储器芯片，采用随机存取；CD-ROM 是只读型光盘存储器，采用顺序存取。名字里都有"只读"，但一个是芯片、一个是光盘，存取方式完全不同——这是本题最大的陷阱。
- "EPROM"和"CD-ROM"的区别：EPROM 是可擦除可编程的半导体 ROM（随机存取），CD-ROM 是光盘（顺序存取）；"ROM"后缀相同，介质不同。

EPROM、DRAM、SRAM 都是半导体存储器，随机存取；CD-ROM 是光盘，顺序存取，选 B。

【坑位】

看到"只读"别直接归入 ROM 家族——先看介质：芯片还是光盘。CD-ROM 是光盘，不是 ROM 芯片。

【类比】

随机存取像翻书：想看第几页直接翻到；顺序存取像听磁带/转光盘：想听中间的歌，得从头转到那个位置。术语贴回去：直接翻页是"随机存取"，从头转是"顺序存取"。

【口诀】 EPROM、DRAM、SRAM，全是芯片随机盘；CD-ROM 是只读光碟，顺序存取不一样。

【考频】 「存储器分类」在 2009–2022 年解析中出现 8 次 · 中频

第 15 题 · 主存地址寄存器 MAR 的位数（正确答案：D）

题干与选项：

某计算机存储器按字节编址，主存地址空间大小为 64MB，现用 4M×8 位的 RAM 芯片组成 32MB 的主存储器，则存储器地址寄存器 MAR 的位数至少是( )

- A. 22 位
- B. 23 位
- C. 25 位
- D. 26 位

入手点： MAR 的位数看的是地址空间有多大，不看实际装了多少内存；64MB 的地址空间需要几位地址来编?

专有名词从零解释：

- 先想“MAR”：主存地址寄存器，CPU 要访问主存时，先把地址放进这个寄存器，再去主存找数据。
- 再想“地址空间”：地址能编到的最大范围；64MB 的地址空间，就是 64×1024×1024 个字节地址。
- 再想“按字节编址”：每个字节一个地址，地址空间 64MB 就有 64M 个地址。

考查知识点：

- 先想地址空间换算：64MB＝64×1024×1024 字节＝2^6×2^{20}＝2^{26} 个字节地址。
- 再想 MAR 位数：2^{26} 个地址需要 26 位二进制来编，所以 MAR 是 26 位，就是 D。
- 再想为什么不是 32MB：实际主存容量 32MB 只是现在装了多少内存；考虑以后扩展内存的需要，MAR 应该能访问整个主存地址空间；反过来，MAR 的位数决定了主存地址空间的大小，不是实际容量决定的。
- 拿生活例子验算：一个小区有 64M 个门牌号，门牌号需要 26 位才能编完；现在只住了 32M 户，不等于门牌号只要 25 位——空着的门牌号以后住人也要用，门牌号的位数看的是小区的规模，不是现在住了几户。
- 边界情况：按字编址时地址空间的算法不一样（一个地址对应一个字，不是对应一个字节），但本题是按字节编址，直接按字节数算。
- 易混对子：不要用 32MB 去算——32MB 是实际容量，不是地址空间；不要把 64MB 换算错——64MB＝2^{26} 字节，不是 2^{25} 也不是 2^{27}。

正确选项为什么对： 地址空间 64MB＝2^{26} 字节，需要 26 位地址，MAR 是 26 位，就是 D，选 D。

错误选项为什么错＋举例：

- 用 32MB 算出来的 25 位：把实际容量当成了地址空间。教训：MAR 看的是地址空间，不是现在装了多少内存。
- 换算错位数：64MB 换算成 2^{26} 字节，位数就是 26。教训：1MB＝2^{20} 字节，64＝2^6，合起来是 2^{26}。

笔记本一句话： 第 15 题＋MAR 位数：看地址空间不看实际容量，64MB＝2^{26}，MAR 是 26 位。

【图像】

MAR 位数由地址空间决定

地址空间 64MB ＝ 2^{26} B

需要 26 根地址线 → MAR 至少 26 位 → 答案 D

32MB ＝ 2^{25} 是实际装机容量（干扰项，对应 C）

实际插了多少芯片，不决定 MAR 位数

MAR 看“能寻多大”（地址空间），不看“装了多大”（实际容量）

对比：MDR 位数由数据总线宽度定（按字节编址 → 8 位）

64MB地址空间对应26根地址线决定MAR为26位的示意图，32MB实际容量标为干扰项

【定位】 计算机组成原理 · 第3章 存储系统 · 主存储器的基本组成

【易混对比】

- "地址空间大小（64MB）"和"实际装机容量（32MB）"的区别：MAR 的位数由地址空间决定——MAR 必须能寻址整个 64MB 地址空间，而不是只看实际插了多少内存芯片。64MB＝2^{26}B，故 MAR 至少 26 位；32MB＝2^{25} 是干扰项，对应选项 C。
- "MAR 位数"和"MDR 位数"的区别：MAR 位数由地址空间大小决定（地址线根数），MDR 位数由数据总线宽度决定（按字节编址时为 8 位）。

64MB＝2^{26}B，地址线需要 26 根，MAR 至少 26 位，选 D；32MB 只是实际装机容量，不决定 MAR 位数。

【坑位】

MAR 看的是"地址空间"（能寻多大），不是"实际装了多少芯片"（装了多大）。

误选 C（25 位）：拿 32MB 实际容量去算 2^{25}——把"装机容量"当成了"地址空间"。教训：题目给两个容量数字时，先问"哪个是地址空间"。

【套路】

- 第一步，找地址空间：题干"主存地址空间大小为 64MB"——这就是 MAR 的依据，32MB 实际容量先放一边。
- 第二步，换算成 2 的幂：64MB＝$2^6 \times 2^{20}$B＝2^{26}B。
- 第三步，指数即位数：需要 26 根地址线，MAR 至少 26 位，对上选项 D。

先锁定"地址空间"这个数字，换算成 2 的幂，指数就是 MAR 位数。

【类比】

MAR 像小区门牌号的位数：小区规划了 64M 个门牌号，门牌就得印 26 位宽，哪怕现在只住了 32M 户人家，门牌位数也不能缩水。术语贴回去：规划门牌数是"地址空间"，已入住户数是"实际容量"。

【口诀】 MAR 位数看空间，六四兆字节二二六；三十二兆是烟雾，实际容量别上当。

【考频】 「MAR」在 2009–2022 年解析中出现 22 次 · 高频

第 16 题 · 哪种寻址方式不需要寄存器（正确答案：A）

题干与选项：

不属于偏移寻址方式的是( )

- A. 间接寻址
- B. 基址寻址
- C. 相对寻址
- D. 变址寻址

入手点： 一种一种看有效地址怎么算：凡是“形式地址＋某个寄存器的内容”才算出来的，都要寄存器；只有一种是直接拿内存里的内容当地址。

专有名词从零解释：

- 先想“有效地址 EA”：真正要访问的那个内存单元的地址。
- 再想“形式地址 A”：指令里直接给出的那个地址数字，还不是最终的有效地址。
- 再想四种寻址：间接寻址是 EA＝(A)，也就是内存 A 号单元里装的那个数就是有效地址；基址寻址是 EA＝A＋基址寄存器 BR 的内容；相对寻址是 EA＝A＋程序计数器 PC 的内容；变址寻址是 EA＝A＋变址寄存器 IX 的内容。

考查知识点：

- 先想间接寻址：EA＝(A)，意思是到内存 A 号单元里取出那个数，那个数就是有效地址；整个过程没有用到任何寄存器，就是 A。
- 再想基址寻址：EA＝A＋BR，要用基址寄存器。
- 再想相对寻址：EA＝A＋PC，要用程序计数器。
- 再想变址寻址：EA＝A＋IX，要用变址寄存器。
- 再想规律：后三种都是“寄存器内容＋一个形式地址”相加才形成有效地址，只有间接寻址不是。
- 拿生活例子验算：间接寻址像信封里装着另一封信的地址，打开信封就知道往哪送，不用查任何登记本；基址、相对、变址像“登记本上的基数＋信封上的数字”加起来才是地址，离了登记本就算不出来。
- 边界情况：间接寻址可能要多访问一次内存（先到 A 号单元取地址，再到真正地址取数据），速度慢，但这是“要不要寄存器”的问题，不是“快不快”的问题。
- 易混对子：不要把“间接寻址”当成“不用访问内存”——间接寻址恰恰要多访问一次内存，取的是内存里的地址；不要把 PC 当成“不是寄存器”——PC 就是程序计数器寄存器，相对寻址用到了它。

正确选项为什么对： 间接寻址 EA＝(A)，不经过任何寄存器，就是 A，选 A。

错误选项为什么错＋举例：

- 基址寻址：EA＝A＋BR，用了基址寄存器。教训：只要公式里有寄存器，就不是“不需要寄存器”。
- 相对寻址：EA＝A＋PC，用了程序计数器。教训：PC 也是寄存器。
- 变址寻址：EA＝A＋IX，用了变址寄存器。教训：三种“寄存器＋形式地址”的都要排除。

笔记本一句话： 第 16 题＋不需寄存器的寻址：间接寻址 EA＝(A)，其余三种都是寄存器＋形式地址。

【图像】

四种寻址方式的有效地址 EA

间接寻址	EA = (A)	形式地址中直接存有效地址，无加法
基址寻址	EA = (BR) + A	基址寄存器内容 + 形式地址
变址寻址	EA = (IX) + A	变址寄存器内容 + 形式地址
相对寻址	EA = (PC) + A	程序计数器内容 + 形式地址

偏移寻址 = 寄存器内容 + 形式地址；间接寻址无“加” → 答案 A

间接、基址、变址、相对四种寻址方式有效地址公式对比，间接寻址无加法操作

【定位】 计算机组成原理 · 第4章 指令系统 · 寻址方式

【易混对比】

- "间接寻址"和"偏移寻址"的区别：间接寻址的有效地址 EA=(A)，形式地址 A 中直接存放的就是有效地址，一次访存取出即用，没有"加"这个动作；基址、变址、相对寻址都是"寄存器内容+形式地址"，这个"加"就是"偏移"。
- "偏移寻址的三种"之间的区别：基址寻址 EA=(BR)+A，变址寻址 EA=(IX)+A，相对寻址 EA=(PC)+A——加的寄存器不同，但都是"寄存器+位移"的结构。

EA=(A) 直接取地址，不加任何寄存器——间接寻址不是偏移寻址，选 A。

【坑位】

偏移=寄存器内容+形式地址；看到 EA 公式里没有加法，就不是偏移寻址。

【套路】

- 第一步，写 EA 公式：间接 EA=(A)；基址 EA=(BR)+A；变址 EA=(IX)+A；相对 EA=(PC)+A。
- 第二步，找没有加法的：只有间接寻址的公式里没有"+"，它不属于偏移寻址，对上选项 A。

默写四个 EA 公式，挑出没有加号的那个。

【类比】

偏移寻址像"导航+偏移"：先到寄存器这个地标，再按形式地址走 A 米；间接寻址像"纸条上直接写着门牌号"，照着去就行，不用再加路。术语贴回去：地标是"基址/变址寄存器或 PC"，纸条是"形式地址中直接存放有效地址"。

【口诀】 基址变址加相对，寄存器里加 A 值；间接寻址无偏移，纸条地址直接取。

【考频】 「寻址方式」在 2009–2022 年解析中出现 10 次 · 高频

第 17 题 · 无符号数大于分支时的标志位组合（正确答案：C）

题干与选项：

某机器有一个标志寄存器，其中有进位/借位标志 CF、零标志 ZF、符号标志 SF 和溢出标志 OF，条件转移指令 bgt（无符号整数比较大于时转移）的转移条件是( )

- A. $\overline{CF+OF}=1$
- B. $\overline{SF+ZF}=1$
- C. $\overline{CF+ZF}=1$
- D. $\overline{CF+SF}=1$

入手点： 无符号数比较只看两样：减法有没有借位（CF）、结果是不是零（ZF）；A 大于 B 时，这两样分别是什么？

专有名词从零解释：

- 先想“无符号数”：只有 0 和正数，没有负数；比较大小就是比数值本身。
- 再想“CF（进位／借位标志）”：做减法时，如果被减数不够减、要向高位借位，CF 就是 1；够减不用借，CF 就是 0。
- 再想“ZF（零标志）”：减法结果是 0，ZF 就是 1；结果不是 0，ZF 就是 0。
- 再想“SF 和 OF”：SF 是符号标志（结果为负时是 1），OF 是溢出标志（有符号数运算溢出时是 1）；这两个是给有符号数比较用的，无符号数比较用不上。

考查知识点：

- 先想 A＞B 时的减法：A-B 肯定够减，不用向高位借位，所以 CF＝0。
- 再想结果是不是零：A＞B，A-B 肯定不是 0（结果是 0 说明 A 和 B 相等），所以 ZF＝0。
- 再想组合：CF＝0 且 ZF＝0，就是 C。
- 拿生活例子验算：你有 10 块钱，买 7 块钱的东西，够付不用借（CF＝0），找零是 3 块不是 0（ZF＝0）；如果找零是 0，说明你花的和你有的一样多（A＝B），不是大于。
- 边界情况：A＝B 时，A-B＝0，CF＝0、ZF＝1；A＜B 时，A-B 不够减，CF＝1、ZF＝0；三组组合各不相同，正好把大于、等于、小于区分开。
- 易混对子：不要用 SF 和 OF 来判断无符号数大小——SF、OF 是有符号数的标志，无符号数比较只看 CF 和 ZF；不要把“CF＝0”理解成“没有进位”就等于“相等”——CF＝0 只说明够减，相等不相等要看 ZF。

正确选项为什么对： A＞B 时够减不用借（CF＝0）、结果不是零（ZF＝0），组合就是 C，选 C。

错误选项为什么错＋举例：

- 用到 SF 或 OF 的组合：SF、OF 是有符号数比较的标志，无符号数比较用不上。教训：无符号数比大小，只看 CF 和 ZF。
- CF＝1 的组合：CF＝1 说明不够减，那是 A＜B 的情况，不是 A＞B。教训：借位标志反着读就错了。
- ZF＝1 的组合：ZF＝1 说明结果是 0，那是 A＝B 的情况，不是 A＞B。教训：结果是零等于相等，不是大于。

笔记本一句话： 第 17 题＋无符号大于：够减 CF＝0、结果非零 ZF＝0，无符号比较只看 CF 和 ZF。

【图像】

无符号大于分支 bgt：A－B 的标志位逻辑



CF、ZF 先“或”：0 或 0 ＝ 0

再“非”：非 0 ＝ 1 → 满足转移条件

转移条件：非(CF＋ZF) ＝ 1 → 答案 C

含 SF、OF 的 B、D 是有符号数比较的，先排除

语义：A＞B ⟺ A－B 够减（CF=0）且有剩余（ZF=0）

无符号大于分支的标志位逻辑：CF与ZF为0经或非门输出1，SF和OF置灰标注为有符号数专用

【定位】 计算机组成原理 · 第2章 数据的表示与运算 · 定点数运算的标志位

【易混对比】

- "无符号数比较"和"有符号数比较"的区别：无符号数比较看 CF（借位标志）和 ZF（零标志）；有符号数比较看 SF（符号标志）、OF（溢出标志）和 ZF。本题条件转移指令 bgt 明确是"无符号整数比较大于"，所以凡是出现 SF、OF 的选项（B、D）直接排除。
- "非(CF＋ZF)＝1"和"CF＋ZF＝1"的区别：选项中的上划线是"非"（逻辑非），非(CF 或 ZF)＝1 意味着 CF＝0 且 ZF＝0，即"无借位且差不为零"，正好对应 A－B＞0。

无符号 A＞B 等价于 A－B 无借位（CF＝0）且结果非零（ZF＝0），即非(CF＋ZF)＝1，选 C。

【坑位】

先看"无符号"三个字——SF、OF 是有符号数比较的标志，出现在选项里就是来送分的排除项。

误选 B：把有符号数大于的条件（看 SF、OF）套到了无符号数上。教训：比较之前先定"有/无符号"，无符号只认 CF 和 ZF。

【套路】

- 第一步，定符号类型：题干"无符号整数"——排除含 SF、OF 的 B、D。
- 第二步，转减法语义：A＞B 等价于 A－B 结果为正，即"够减（无借位）且有剩余（非零）"。
- 第三步，写标志条件：无借位即 CF＝0，非零即 ZF＝0，合起来是非(CF＋ZF)＝1，对上选项 C。

无符号先排除 SF/OF，再把"大于"翻译成"无借位且非零"。

【类比】

无符号比较像数苹果：A 堆拿走 B 堆那么多，够拿（没借位）且还有剩（不为零），就说明 A 比 B 多。术语贴回去：够拿是"CF＝0"，有剩是"ZF＝0"。

【口诀】 无符号比大小，CF、ZF 来定调；无借位来非零值，非或为一选 C 项。

【考频】 「标志位」在 2009–2022 年解析中出现 17 次 · 高频

第 18 题 · RISC 的特征三句话是否成立（正确答案：D）

题干与选项：

下列给出的指令系统特点中，有利于实现指令流水线的是( )

Ⅰ. 指令格式规整且长度一致

Ⅱ. 指令和数据按边界对齐存放

Ⅲ. 只有 Load/Store 指令才能对操作数进行存储访问

- A. 仅Ⅰ、Ⅱ
- B. 仅Ⅱ、Ⅲ
- C. 仅Ⅰ、Ⅲ
- D. Ⅰ、Ⅱ、Ⅲ

入手点： RISC 的精髓是“简单、规整、好流水”；三句话分别说指令定长、对齐、只有 Load/Store 能访存，一句一句看是不是都对得上这个精髓。

专有名词从零解释：

- 先想“RISC”：精简指令集计算机，指令种类少、格式规整，目标是让每条指令都跑得快、好排流水线。
- 再想“指令定长”：每条指令的长度都一样长，译码的时候不用先猜这条指令有多长。
- 再想“对齐”：指令和数据都放在地址是某个倍数的位置上，一个存取周期就能取到，不用分两次取。
- 再想“Load/Store”：只有取数指令（Load）和存数指令（Store）才能访问内存，其他指令只能在寄存器里算。
- 再想“流水线”：像工厂流水线一样，让多条指令的不同步骤同时进行，提高速度。

考查知识点：

- 先想第Ⅰ句（对）：指令格式规整、长度一致，译码就简单——不用先判断指令多长，直接按固定格式拆开，有利于实现流水线。
- 再想第Ⅱ句（对）：指令和数据边界对齐，保证在一个存取周期内就能取到需要的数据和指令，不用多余的延迟等待，也利于流水线。
- 再想第Ⅲ句（对）：只有 Load/Store 指令才能对操作数进行存储访问，指令操作简化了，执行时间长度固定，能有效简化流水线复杂度。
- 再想组合：三句都成立，A 是仅Ⅰ、Ⅱ、B 是仅Ⅱ、Ⅲ、C 是仅Ⅰ、Ⅲ、D 是Ⅰ、Ⅱ、Ⅲ，就是 D。
- 拿生活例子验算：RISC 像标准化生产的零件——尺寸都一样（定长）、摆放都对齐（对齐）、只有专门的搬运工能进仓库（Load/Store），流水线转起来就顺；零件尺寸不一、摆放歪斜、谁都能进仓库，流水线就卡。
- 边界情况：RISC 不是“指令越少越好”——指令种类少是为了格式规整、执行时间固定，不是为了单纯追求数量少；现代处理器很多是 RISC 内核加翻译层，但这三条特征本身还是 RISC 的标志。
- 易混对子：不要把 RISC 和 CISC 的特征说反——指令不定长、种类多、访存方式多是 CISC 的样子，不是 RISC 的；不要以为“只有 Load/Store 能访存”会让程序变慢——访存指令少了，但每条指令都快、流水线顺，整体反而快。

正确选项为什么对： 定长、对齐、仅 Load/Store 访存三句都成立，组合是Ⅰ、Ⅱ、Ⅲ，就是 D，选 D。

错误选项为什么错＋举例：

- A（仅 Ⅰ、Ⅱ）：漏了Ⅲ——只有 Load/Store 能访存同样是 RISC 的特征。教训：三句要一句一句验，漏一句对的，组合就错。
- B（仅 Ⅱ、Ⅲ）：漏了 Ⅰ——指令定长同样是 RISC 的特征。教训：定长简化译码，是 RISC 的标志之一。
- C（仅 Ⅰ、Ⅲ）：漏了 Ⅱ——指令和数据对齐同样是 RISC 的特征。教训：对齐保证一个周期取到，是流水线顺畅的前提。

笔记本一句话： 第 18 题＋RISC 三特征：定长、对齐、仅 Load/Store 访存，三句都成立。

【图像】



RISC三项特征全部打勾：指令格式规整、边界对齐、Load/Store结构，均有利于指令流水线

【定位】 计算机组成原理 · 第4章 指令系统 · CISC 与 RISC

【易混对比】

- "RISC"和"CISC"的区别：RISC 是精简指令集——指令种类少、格式规整长度一致、只有 Load/Store 能访存、采用硬布线控制；CISC 是复杂指令集——指令多而格式不规整、多种寻址方式、多用微程序控制。本题三项都是 RISC 一侧的特征。
- "有利于流水线"和"RISC 设计目标"的区别：RISC 的设计目标之一就是让指令流水线易于实现，三项特征每一条都在为流水线铺路（规整格式让取指译码节拍一致，对齐存放让访存不跨边界，Load/Store 让访存时机确定）。

Ⅰ、Ⅱ、Ⅲ 都是 RISC 的标志性特征，三项全选，选 D。

【坑位】

"Ⅰ、Ⅱ、Ⅲ"全选型题目最容易在"仅Ⅰ、Ⅱ"这类缺一项的选项上丢分；RISC 的三条流水线友好特征要一条不漏地记住。

【类比】

RISC 像标准化流水线工厂：零件规格统一（格式规整长度一致）、货架摆放整齐（边界对齐）、只有专门的搬运工能进仓库（Load/Store 结构），流水线自然转得又快又稳。术语贴回去：搬运工是"Load/Store 指令"，仓库是"主存"。

【口诀】 格式规整长度齐，边界对齐存取易；访存只有 LoadStore，三项全对选 D 项。

【考频】 「RISC」在 2009–2022 年解析中出现 5 次 · 中频

第 19 题 · 指令周期相关叙述中错误的一项（正确答案：C）

题干与选项：

假定不采用 Cache 和指令预取技术，且机器处于"开中断"状态，则在下列有关指令执行的叙述中，错误的是( )

- A. 每个指令周期中 CPU 都至少访问内存一次
- B. 每个指令周期一定大于或等于一个 CPU 时钟周期
- C. 空操作指令的指令周期中任何寄存器的内容都不会被改变
- D. 当前程序在每条指令执行结束时都可能被外部中断打断

入手点： 指令周期是“取指令、执行指令”的一轮；一句一句验：不预取时是不是每轮都要取指、每轮是不是至少一个时钟周期、空操作指令取指后 PC 变不变、开中断时每轮结束会不会被打断。

专有名词从零解释：

- 先想“指令周期”：CPU 执行一条指令的一轮：先取指令，再执行指令。
- 再想“指令预取”：提前把后面的指令取出来放着的技术；不用预取时，每轮都要现取。
- 再想“Cache”：CPU 和内存之间的小块高速存储；不用 Cache 时，取指令要直接访问内存。
- 再想“PC（程序计数器）”：记着下一条要执行指令的地址的寄存器；取指之后，PC 会自动加 1（指向下一条指令）。
- 再想“空操作指令”：什么都不干的指令，执行它就是空转一轮。

考查知识点：

- 先想 A（对）：不采用指令预取技术时，每个指令周期都需要取指令——没有提前取好的，就得每轮现取。
- 再想 B（对）：时钟周期是 CPU 的最小时间单位，每个指令周期一定大于或等于一个 CPU 时钟周期——一轮里至少要花一个时钟周期；不采用 Cache 技术时，每次取指令都至少要访问内存一次（当指令字长与存储字长相等且按边界对齐时）。
- 再想 C（错）：空操作指令的指令周期内，任何寄存器的内容都不会被改变——这句是错的；即使是空操作指令，在取指操作之后 PC 也会自动加 1，PC 的内容变了；所以“任何寄存器都不变”不成立，就是 C。
- 再想开中断的情况：机器处于“开中断”状态时，在每条指令执行结束时都可能被外部中断打断——这句是对的，指令周期结束是响应中断的时机。
- 拿生活例子验算：空操作指令像“原地踏步”——脚是没往前走，但数数的牌子（PC）还是翻了一页；说“什么都没变”不对，牌子变了。
- 边界情况：PC 自动加 1 是取指阶段的动作，和指令本身干不干活没关系；任何指令（包括空操作）取指后 PC 都会变。
- 易混对子：不要把“空操作什么都不干”理解成“什么寄存器都不变”——PC 是取指阶段就变的，和指令的执行内容无关；不要把“指令周期”和“时钟周期”混在一起——指令周期由好几个时钟周期组成，至少是一个。

正确选项为什么对： C 说空操作周期内任何寄存器都不变，但取指后 PC 会自动加 1，所以 C 是错的那句，就是 C，选 C。

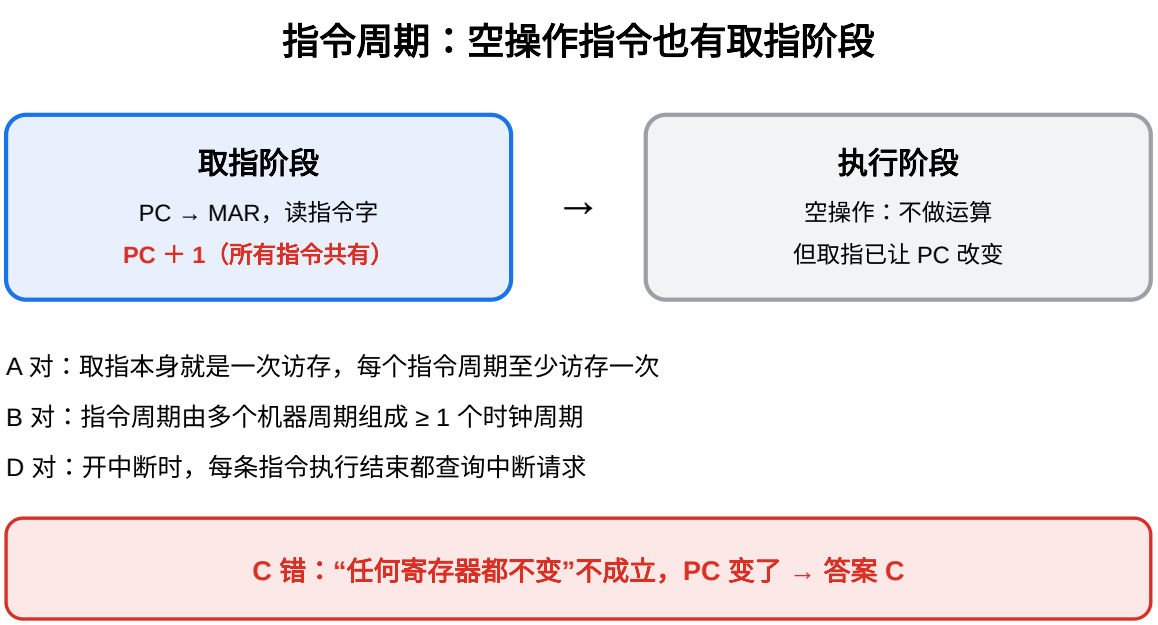
错误选项为什么错＋举例：

- A：不预取时每轮都要现取指令，是对的。教训：没有提前取好的，每轮都得取。

- B：时钟周期是最小时间单位，指令周期至少一个时钟周期，是对的。教训：指令周期不可能比一个时钟周期还短。
- 开中断时每轮结束可能被打断：这是对的，指令周期结束是响应中断的时机。教训：中断在指令之间响应，不在指令中间响应。

笔记本一句话： 第 19 题＋指令周期：空操作取指后 PC 也会加 1，“任何寄存器都不变”是错的。

【图像】



指令周期流程：取指阶段PC自动加1，空操作指令同样经过取指，C选项错误点标注

【定位】 计算机组成原理 · 第5章 中央处理器 · 指令周期

【易混对比】

- "空操作指令"和"什么都不做"的区别：空操作（NOP）只是执行阶段不做运算，但它同样要经过取指阶段——取指时 PC 指向下一条指令（PC＋1），所以"任何寄存器的内容都不会被改变"是错的，错在 PC 变了。
- "指令周期"和"时钟周期"的区别：指令周期是取出并执行一条指令的全部时间，由若干个机器周期（每个含若干时钟周期）组成，所以指令周期一定大于或等于一个时钟周期（B 对）。
- "至少访存一次"：任何指令的取指阶段都要从内存取指令字，所以每个指令周期 CPU 至少访问内存一次（A 对）；"开中断"状态下，CPU 在每条指令执行结束时查询中断，可能被外部中断打断（D 对）。

错误的是 C：空操作指令也要取指，取指后 PC 自动加 1，PC 的内容改变了。

【坑位】

空操作≠什么都不做——取指是所有指令共有的动作，PC＋1 跑不掉。

【套路】

- 第一步，逐项验证：A 取指必访存，对；B 指令周期由多个时钟周期组成，对；D 开中断时指令执行结束查中断请求，对。
- 第二步，盯住 C 的"任何"："任何寄存器都不变"说法太绝对——取指阶段 PC＋1，PC 变了，C 错。

- 第三步，锁定答案：题目问"错误的是"，选 C。

"至少""一定""任何""都"这类绝对化词语是首先要怀疑的对象。

【类比】

空操作像"原地踏步"：脚确实没往前走（执行阶段没做运算），但报数器照样加 1（PC＋1），不然下一次不知道该踏哪一步。术语贴回去：报数器是"程序计数器 PC"，原地踏步是"空操作指令"。

【口诀】 取指访存不能少，周期总比时钟大；空操作也加 PC，中断查询在指令末。

【考频】 「指令周期」在 2009–2022 年解析中出现 4 次 · 中频

第 20 题 · 哪种信号不在数据总线上传输（正确答案：C）

题干与选项：

在系统总线的数据线上，不可能传输的是( )

- A. 指令
- B. 操作数
- C. 握手（应答）信号
- D. 中断类型号

入手点： 数据总线是传“数据”的：指令、操作数、中断类型号都是数据；握手信号是管“什么时候传”的联络信号，走控制总线。

专有名词从零解释：

- 先想“三总线”：数据总线传数据，地址总线传地址，控制总线传控制信号和联络信号。
- 再想“握手（应答）信号”：通信双方确认“你发了吗”“我收到了”的联络信号，属于通信联络控制信号。
- 再想“中断类型号”：告诉 CPU 是哪种中断的编号；CPU 靠它查中断向量表，找到对应的中断处理程序入口。

考查知识点：

- 先想取指令：取指令时，指令本身是在数据线上传输的——指令就是一种数据。
- 再想操作数：操作数是数据，当然在数据线上传输。
- 再想中断类型号：CPU 响应中断请求后，把中断应答信号发回数据总线，再从数据总线读取中断类型号，查中断向量表找到处理程序入口；中断类型号也是在数据线上传输的。
- 再想握手信号：握手（应答）信号是通信联络控制信号，应该在控制总线上传输，不在数据总线上传输，就是 C。
- 拿生活例子验算：数据总线像运货的卡车，拉的是指令、操作数、中断类型号这些“货”；握手信号像打电话确认“货发了吗”“货到了吗”，是联络电话，不上卡车。
- 边界情况：中断应答信号本身也是控制信号，但 CPU 是从数据总线读取中断类型号的；题目问的是“哪种信号不在数据总线上传输”，握手信号是唯一走控制总线的那个。
- 易混对子：不要把“中断类型号”和“中断应答信号”混在一起——类型号是数据（走数据总线），应答是控制（走控制总线）；不要以为“控制总线只传时钟、复位”——联络用的握手信号也在控制总线上传。

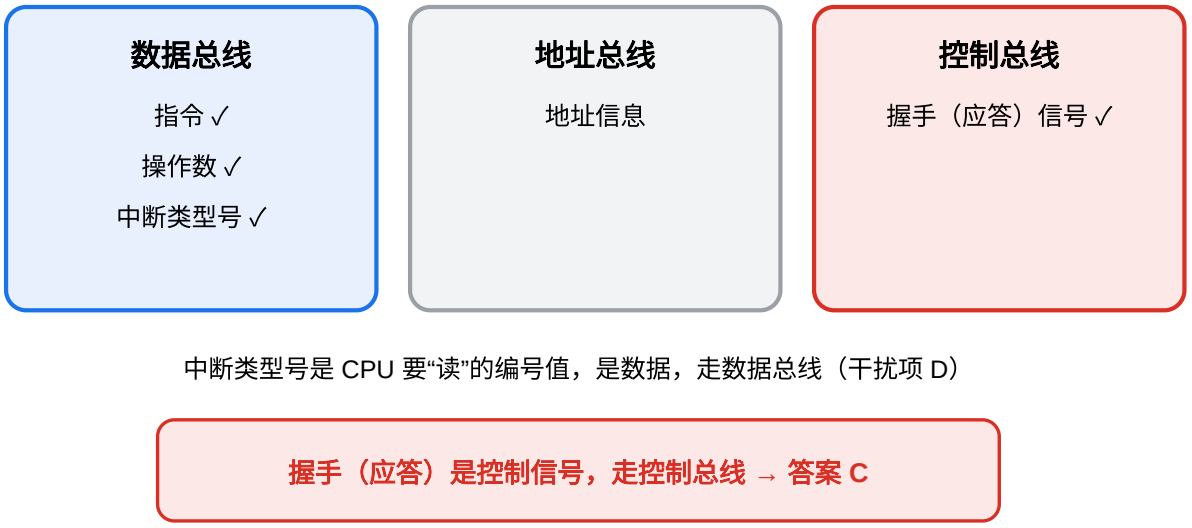
正确选项为什么对： 握手（应答）信号是通信联络控制信号，走控制总线，不在数据总线上传输，就是 C，选 C。

错误选项为什么错＋举例：

- 指令：取指令时指令在数据线上传输。教训：指令就是一种数据。
- 操作数：操作数是数据，在数据线上传输。教训：数据总线传的就是操作数这类东西。
- 中断类型号：CPU 从数据总线读取中断类型号查表。教训：类型号是编号数据，走数据总线。

笔记本一句话： 第 20 题＋三总线：指令、操作数、中断类型号走数据总线，握手信号走控制总线。

【图像】



系统三总线信号归属图：指令、操作数、中断类型号走数据总线，握手应答信号走控制总线

【定位】 计算机组成原理 · 第6章 总线 · 总线的分类

【易混对比】

- "数据总线"和"控制总线"的区别：数据总线双向传输数据类信息——指令、操作数都在其上传输；控制总线传输控制类信号——握手（应答）信号是典型的控制信号，走控制总线，不走数据总线。
- "中断类型号"的归属：CPU 通过数据总线读取中断类型号，再据此查中断向量表找到中断服务程序入口——所以中断类型号是走数据总线的，它是本题的干扰项。

数据总线上传的是"数据"（指令、操作数、中断类型号）；握手（应答）信号是控制信号，走控制总线，选 C。

【坑位】

"握手""应答"一听就是控制协调类信号——控制信号不坐数据总线这趟车。

误选 D：以为中断类型号也是控制信息。教训：中断类型号是 CPU 要"读"的一个编号值，是数据，走数据总线。

【套路】

- 第一步，分类：题目问"数据线上不可能传输的"——把四个选项分成"数据类"和"控制类"。
- 第二步，找控制信号：指令、操作数、中断类型号都是数据类；只有握手（应答）信号是控制类，对上选项 C。

先二分"数据类 vs 控制类"，控制类的就是答案。

【类比】

数据总线像货车的车厢（拉货：指令、操作数、中断类型号），控制总线像司机手里的对讲机（发指令：握手、应答）。"收到请回复"是握手信号，走对讲机，不会装进车厢里拉走。术语贴回去：车厢是"数据总线"，对讲机是"控制总线"。

【口诀】 数据总线拉数据，指令操作中断号；握手应答是控制，走错车道选 C 项。

【考频】 「总线」在 2009–2022 年解析中出现 4 次 · 中频

第 21 题 · 中断屏蔽字（正确答案：D）

题干与选项：

某计算机有五级中断 L4～L0，中断屏蔽字为 M4M3M2M1M0，Mi=1（0≤i≤4）表示对 Li 级中断进行屏蔽。若中断响应优先级从高到低的顺序是 L0→L1→L2→L3→L4，且要求中断处理优先级从高到低的顺序是 L4→L0→L2→L1→L3，则 L1 的中断处理程序中设置的中断屏蔽字是( )

- A. 11110
- B. 01101
- C. 00011
- D. 01010

入手点： 屏蔽字里的 0 表示“可以被中断”，1 表示“不可以被中断”；一个中断源只能屏蔽比它优先级低（或相等）的中断源，不能屏蔽比它优先级高的。

专有名词从零解释：

- 先想“中断屏蔽字”：一串 0 和 1，每一位对应一个中断源；0 表示这个中断源可以打断当前中断处理，1 表示不可以。
- 再想“优先级”：中断源排队处理时谁先谁后；高优先级的中断可以打断低优先级的中断处理，低优先级的不能打断高优先级的。
- 再想本题的规矩：高优先级 0 表示可被中断，比该中断优先级低（或相等）的置 1 表示不可被中断。

考查知识点：

- 先想 L1 能屏蔽谁：L1 只能屏蔽 L3 和其自身——L3 比 L1 优先级低，自身和自身优先级相等；比 L1 优先级高的，L1 屏蔽不了。
- 再想屏蔽字：M4M3M2M1M0 里，M3 和 M1 置 1（被屏蔽的 L3 和自身 L1），其余置 0（可以打断 L1 的高优先级中断源），所以屏蔽字是 01010，就是 D。
- 拿生活例子验算：L1 像一个正在开会的人，只有比他级别低（L3）和他自己（L1）不能打断他；级别比他高的人（M4、M2、M0 对应的中断源）随时可以打断；屏蔽字里 1 的位置就是“不能打断”的那两位。
- 边界情况：自身那一位总是 1——一个中断处理过程中，同级的中断不能打断自己，否则会无限嵌套；最高优先级的中断源屏蔽字全是 1（谁都不能打断它），最低优先级的中断源屏蔽字里只有自身那一位是 1。
- 易混对子：不要把 0 和 1 的意思弄反——本题规矩是 0 可被中断、1 不可被中断，和有些教材“1 表示开放”的规矩正好反着，看题目的规矩，不看习惯；不要以为低优先级能屏蔽高优先级——屏蔽只能往低处屏蔽，不能往高处屏蔽。

正确选项为什么对： L1 只能屏蔽 L3 和自身，M3 和 M1 置 1，其余置 0，屏蔽字是 01010，就是 D，选 D。

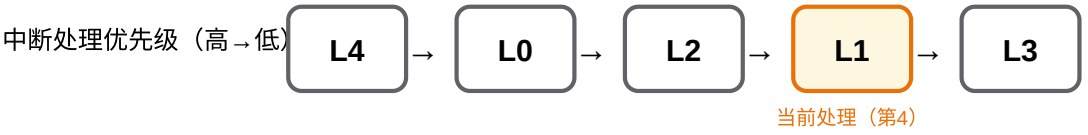
错误选项为什么错＋举例：

- 0 和 1 弄反的屏蔽字：把“可被中断”写成了 1。教训：先看题目给的规矩，0 是可被中断，1 是不可被中断。
- 把高优先级也屏蔽了的屏蔽字：低优先级屏蔽不了高优先级。教训：屏蔽只能屏蔽低（或相等）优先级的。

笔记本一句话： 第 21 题＋中断屏蔽字：0 可被中断、1 不可被中断，L1 只屏蔽 L3 和自身，01010。

【图像】

L1 中断处理程序中的屏蔽字推导



规则：排在 L1 之前的放行（屏蔽字置 0），L1 自身及之后的屏蔽（置 1）



M4M3M2M1M0 = 01010 → 答案 D

中断处理优先级链与L1屏蔽字推导：L4→L0→L2→L1→L3，屏蔽字M4M3M2M1M0=01010逐位标注放行或屏蔽

【定位】 计算机组成原理 · 第7章 输入/输出系统 · 程序中断方式

【易混对比】

- "中断响应优先级"和"中断处理优先级"的区别：响应优先级是多个中断同时到来时先响应谁（硬件排队顺序，题中 L0→L1→L2→L3→L4）；处理优先级是中断处理过程中谁能打断谁（由屏蔽字决定，题中 L4→L0→L2→L1→L3）。算屏蔽字必须用处理优先级，这是本题第一道坎。
- "屏蔽字 1"和"屏蔽字 0"的区别：Mi＝1 表示对 Li 级中断进行屏蔽（不许它打断当前处理），Mi＝0 表示开放（允许它打断）。1＝挡住，0＝放行，别记反。

处理优先级 L4→L0→L2→L1→L3，L1 排第 4：排在 L1 之前的 L4、L0、L2 放行置 0，L1 自身和排在之后的 L3 屏蔽置 1，得 M4M3M2M1M0＝01010，选 D。

【坑位】

屏蔽字看的是"处理优先级"，不是"响应优先级"；1＝屏蔽（挡住），0＝开放（放行）。

误用响应优先级 L0→L1→L2→L3→L4 去算：会得出 L1 只屏蔽 L2、L3、L4 的错误结论。教训：题干给两个顺序，算屏蔽字只认"处理优先级"。

【套路】

- 第一步，列处理优先级：L4→L0→L2→L1→L3（注意不是响应顺序）。
- 第二步，定 L1 的位置：L1 在这条链上排第 4。
- 第三步，高的放行、自己和低的屏蔽：L4、L0、L2 排在 L1 之前，置 0；L1 自身和 L3 置 1。
- 第四步，按 M4M3M2M1M0 组装：0、1、0、1、0＝01010，对上选项 D。

认准处理优先级链，排在前面的放行（0），自己和后面的屏蔽（1）。

【类比】

像开会时 L1 正在发言：比他级别高的（L4、L0、L2）可以插话（放行），他自己不能打断自己、级别更低的（L3）不许插话（屏蔽）。术语贴回去：插话是"中断嵌套"，发言顺序是"中断处理优先级"。

【口诀】 处理优先定屏蔽，高的放行低的挡；自身屏蔽不能少，一零一零对 D 项。

【考频】 「中断屏蔽」在 2009–2022 年解析中出现 5 次 · 中频

第 22 题 · 程序查询方式下 CPU 用于设备 I/O 的时间占比（正确答案：C）

题干与选项：

某计算机处理器主频为 50MHz，采用定时查询方式控制设备 A 的 I/O，查询程序运行一次所用的时钟周期至少为 500。在设备 A 工作期间，为保证数据不丢失，每秒需对其查询至少 200 次，则 CPU 用于设备 A 的 I/O 的时间占整个 CPU 时间的百分比至少是( )

- A. 0.02%
- B. 0.05%
- C. 0.20%
- D. 0.50%

入手点： 先算每秒花在查询上的时钟周期总数，再除以每秒总的时钟周期数。

专有名词从零解释：

- 先想“程序查询方式”：CPU 不停地问设备“你好了没有”，问到了就处理；问一次要花若干个时钟周期。
- 再想“主频 50MHz”：CPU 每秒有 50M 个时钟周期，也就是 50,000,000 个。
- 再想“时间占比”：花在 I/O 上的周期数除以总周期数。

考查知识点：

- 先想每秒查询花多少周期：每秒至少查询 200 次，每次至少 500 个周期，总共 200×500＝100000 个周期。
- 再想占比：100000 除以 50M（50,000,000），等于 0.002，也就是 0.20%，就是 C。
- 拿生活例子验算：CPU 每秒有 5000 万个时间单位，查询设备花了 10 万个单位；10 万占 5000 万的比例是千分之二，也就是百分之零点二。
- 边界情况：题目说的是“至少”查询 200 次、“至少”500 个周期，所以 0.20% 是下限；实际查询更频繁或每次更久，占比只会更大。
- 易混对子：不要把 50M 当成 50 万或 5000 万以外的数——50MHz 就是每秒 50,000,000 个周期；不要把 0.002 写成 0.2% 以外的数——0.002 换成百分数是 0.20%，不是 2%，也不是 0.002%。

正确选项为什么对： 200×500＝100000 个周期，占 50M 个周期的 0.20%，就是 C，选 C。

错误选项为什么错＋举例：

- 占比算成 2%：小数点错了一位，0.002 换成百分数是 0.20%，不是 2%。教训：小数换百分数要乘 100，0.002×100＝0.20。
- 主频换算错：50MHz 是 50,000,000，不是 50,000。教训：MHz 是每秒百万个周期，50MHz 就是 5000 万。

笔记本一句话： 第 22 题＋程序查询占比：200×500＝100000 个周期，占 50M 的 0.20%。

【图像】

程序查询 I/O 时间占比计算

每秒查询耗费周期 = 200 次 × 500 周期 = 100,000 周期/秒

（200 次、500 周期都是“至少”，算出的是占比下限）

↓

主频 50MHz = 每秒 50,000,000 个时钟周期

↓

占比 = 100,000 ÷ 50,000,000 = 0.002 = 0.20%

答案 C（注意不是 0.02%，小数点别错位）

程序查询I/O时间占比计算流程：每秒查询周期数除以主频周期数得0.20%

【定位】 计算机组成原理 · 第7章 输入/输出系统 · 程序查询方式

【易混对比】

- "至少查询 200 次"和"恰好 200 次"的区别：题目求的是时间占比"至少"是多少，所以 200 次和 500 周期都取下限值代入——算出来的是占比的下限 0.20%。
- "时钟周期数"和"时间"的区别：500 是"时钟周期个数"不是时间，必须先乘出每秒的总周期数，再除以主频（每秒的周期数），单位统一为"周期"才能相除。

每秒查询耗费 200×500＝100000 个周期，主频 50MHz 即每秒 50,000,000 个周期，100000÷50,000,000＝0.2%，选 C。

【坑位】

单位换算别掉链：MHz 是"每秒百万周期"，500 是"周期个数"；先把分子分母都化成"每秒周期数"再除。

误选 A（0.02%）：小数点错一位——100000÷50,000,000＝0.002＝0.2%，不是 0.02%。教训：除完先写成小数再化百分数，数零的个数。

【套路】

- 第一步，算每秒查询总周期：200 次×500 周期＝100000 周期/秒。
- 第二步，算每秒主频总周期：50MHz＝50,000,000 周期/秒。
- 第三步，相除化百分数：100000÷50,000,000＝0.002＝0.20%，对上选项 C。

分子"每秒查询周期数"，分母"每秒主频周期数"，相除即占比。

【类比】

像保安巡逻占全天时间的比例：每秒巡 200 圈（查询次数），每圈 500 步（周期数），一天能走 5000 万步（主频），巡逻步数占比就是 10 万除以 5000 万。术语贴回去：巡逻是"程序查询"，步数是"时钟周期"。

【口诀】 查询次数乘周期，除以主频化小数；十万除以五千万，百分之零点二零。

【考频】 「程序查询」在 2009–2022 年解析中出现 25 次 · 高频

第 23 题 · 既照顾短作业又不产生饥饿的调度算法（正确答案：B）

题干与选项：

下列选项中，满足短任务优先且不会发生饥饿现象的调度算法是( )

- A. 先来先服务
- B. 高响应比优先
- C. 时间片轮转
- D. 非抢占式短任务优先

入手点： 短作业优先照顾了短作业，但长作业会饿死；先来先服务不饿死，但不照顾短作业；要找一个把“等了多久”和“作业多短”一起算的算法。

专有名词从零解释：

- 先想“短作业优先”：先跑执行时间短的作业；短作业等得少，但长作业可能一直排在后面，永远轮不到，这叫饥饿。
- 再想“先来先服务”：按到达顺序排队，谁先到谁先跑；不会饿死，但短作业要排在长作业后面等，不照顾短作业。
- 再想“时间片轮转”：每个作业轮流跑一小段时间；不会饿死，但同样不优先照顾短作业。
- 再想“高响应比优先”：响应比＝(等待时间＋执行时间)/执行时间；每次挑响应比最高的作业跑。

考查知识点：

- 先想高响应比为什么照顾短作业：等待时间相同时，作业执行时间越短，响应比越高（分母小，比值大），所以短作业优先被挑中，满足短任务优先。
- 再想高响应比为什么不饿死：长作业等待时间越长，响应比也越大（分子变大），等得越久，响应比越高，迟早会被挑中，所以不会发生饥饿。
- 再想其他算法为什么不行：先来先服务和时间片轮转不符合短任务优先（短作业要排队等）；非抢占式短任务优先会产生饥饿（长作业一直被短作业插队）。
- 再想答案：高响应比优先算法就是 B。
- 拿生活例子验算：高响应比像排队时给每个人算一个“着急程度”＝(等了多久＋要办多久)/要办多久；要办的事越短、等得越久，着急程度越高；长事等得久了，着急程度也会涨上来，迟早轮到它。
- 边界情况：高响应比优先每次调度都要算所有作业的响应比，开销比先来先服务大；但这是调度算法本身的代价，不影响它“又照顾短作业又不饿死”的性质。
- 易混对子：不要把“短作业优先”和“高响应比优先”当成一个——短作业优先只看执行时间，会饿死长作业；高响应比还看等待时间，不会饿死；不要以为时间片轮转照顾短作业——轮转是大家轮流，不分长短。

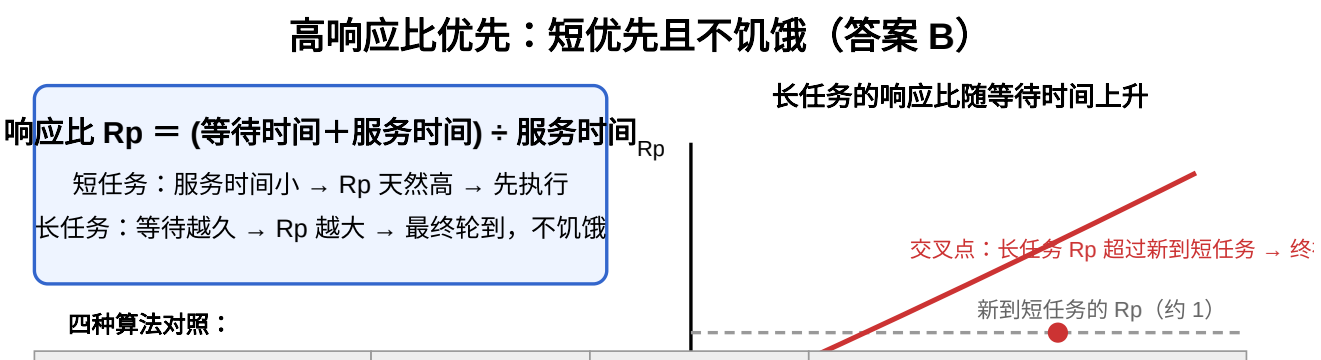
正确选项为什么对： 高响应比优先既让短作业响应比高、又让长作业等久了响应比涨上来，两头都照顾，就是 B，选 B。

错误选项为什么错＋举例：

- 先来先服务：不饿死，但短作业要排在长作业后面等，不照顾短作业。教训：先到先得不管作业长短。
- 时间片轮转：不饿死，但同样不优先照顾短作业。教训：轮流跑不分长短。
- 非抢占式短任务优先：照顾短作业，但长作业一直被插队，会饿死。教训：只看执行时间，不管等待时间，长作业就没出头之日。

笔记本一句话： 第 23 题＋高响应比优先：响应比＝(等待＋执行)/执行，短作业比值高，长作业等久了比值也涨，不饿死。

【图像】



四种算法对照：

算法	短优先	无饥饿	出局原因
A 先来先服务	✗	✓	不区分任务长短
B 高响应比优先	✓	✓	两条都满足 → 答案
C 时间片轮转	✗	✓	轮流分片，不分长短
D 非抢占短任务优先	✓	✗	长任务被插队，会饥饿

题干两个条件必须同时成立：D 只满足"短优先"，漏看"不饥饿"就会误选

高响应比优先公式与长任务响应比随等待时间上升、最终超过新到短任务的示意图

【定位】 操作系统 · 第2章 · 处理机调度算法

【易混对比】

- "高响应比优先"和"非抢占式短任务优先"的区别：短任务优先只看服务时间长短，长任务会被不断到来的短任务插队、可能永远等不到 CPU（饥饿）；高响应比优先的响应比里加了等待时间，等得越久响应比越大，长任务最终一定能轮到。
- "先来先服务"和"时间片轮转"的区别：前者按到达顺序排队、不看服务长短；后者把 CPU 时间切成片轮流分、不区分长短；两者都不满足"短任务优先"。

既要短任务优先、又要不饥饿，只有高响应比优先，选 B。

【坑位】

看到"短任务优先"别直接抓 D——非抢占式短任务优先满足短优先，但长任务会饥饿，题干两个条件必须同时成立。

误选 D：只看了"短任务优先"四个字，漏了"不会发生饥饿"的第二个条件。误选 A/C：先来先服务和时间片轮转都不区分任务长短，连第一个条件都不满足。

【套路】

- 第一步，把题干拆成两个条件：①短任务优先 ②不发生饥饿。
- 第二步，逐项代入验证：A 不满足①，出局；C 不满足①，出局；D 满足①但违背②，出局。
- 第三步，剩下 B：响应比=(等待+服务)/服务，短任务响应比天然高、先执行，长任务等待久了响应比变大、最终能被调度，两个条件都成立。

【类比】

像银行排队叫号：高响应比优先是"等得越久、号码权重越大"——刚来的短业务先办，但等了一上午的老客户权重会超过新来的短业务，不会永远等下去；非抢占式短任务优先是"只看业务长短"，老客户会被每一个新来的短业务插队，饿死在队伍里。

【口诀】 短优先还要防饥饿，高响应比最稳妥；等待写进分子里，长短任务都有路。

【考频】 「调度算法」在 2009–2022 年解析中出现 9 次 · 中频

第 24 题 · 哪项在用户态执行（正确答案：A）

题干与选项：

下列选项中，在用户态执行的是( )

- A. 命令解释程序
- B. 缺页处理程序
- C. 进程调度程序
- D. 时钟中断处理程序

入手点： 用户态和核心态的分界线是：操作系统核心干的事在核心态，用户程序干的事在用户态；中断、调度都是核心的事，命令解释程序是面向用户的。

专有名词从零解释：

- 先想“用户态”：用户程序运行的状态，只能执行普通指令，不能直接碰硬件、不能改系统关键数据。
- 再想“核心态”：操作系统核心运行的状态，什么指令都能执行，能直接管硬件。
- 再想“中断”：设备或程序出了需要操作系统处理的事，CPU 从用户态陷入核心态去处理。
- 再想“命令解释程序”：用户敲命令、操作系统把命令翻译成操作的那层程序，是命令接口，面向用户。

考查知识点：

- 先想缺页处理：缺页是中断的一种，中断处理在核心态执行。
- 再想时钟中断：时钟中断也是中断，在核心态执行。
- 再想进程调度：进程调度是操作系统核心程序，不用用户干预，在核心态执行。
- 再想命令解释程序：命令解释程序属于命令接口，是四个选项中唯一面向用户的，在用户态执行，就是 A。
- 拿生活例子验算：用户态像大厅，核心态像后台机房；缺页、时钟中断、进程调度都是机房里的活，只能在机房干；命令解释程序是前台接待，站在大厅里接用户的命令。
- 边界情况：命令解释程序有时候也叫 shell，它本身是用户程序；它要干系统级的事时，会通过系统调用陷入核心态，但它自己运行在用户态。
- 易混对子：不要把“命令解释程序”当成操作系统核心——它是命令接口，是用户程序；不要把“进程调度”当成用户能干的事——调度是核心程序，用户干预不了。

正确选项为什么对： 命令解释程序是唯一面向用户的命令接口，在用户态执行，就是 A，选 A。

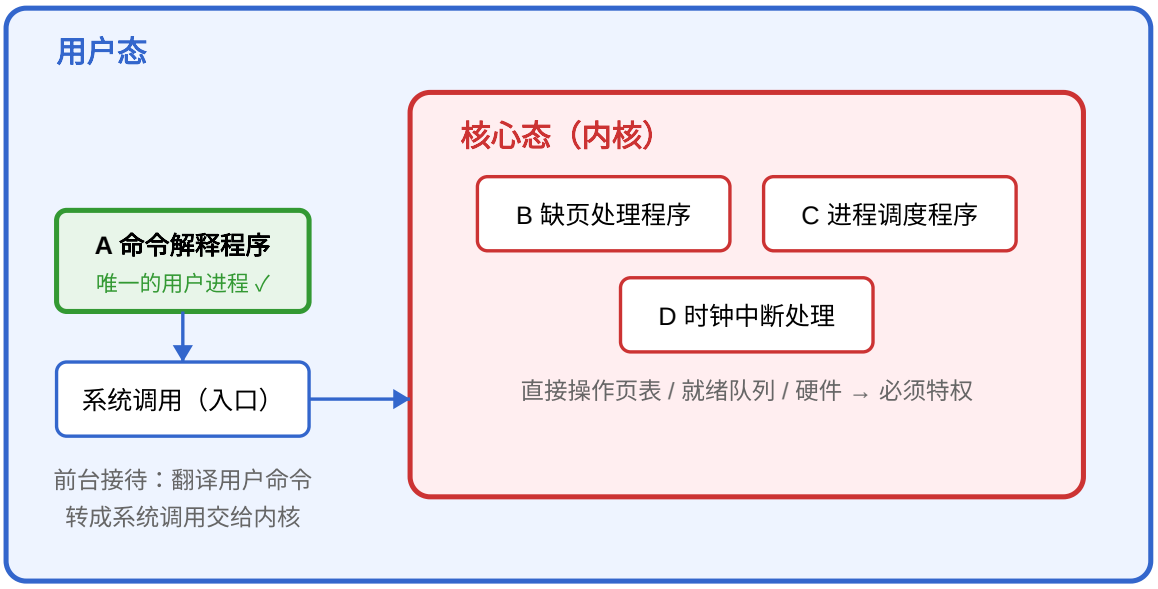
错误选项为什么错＋举例：

- 缺页处理：缺页是中断，在核心态执行。教训：中断处理都在核心态。
- 时钟中断：时钟中断是中断，在核心态执行。教训：中断处理都在核心态。
- 进程调度：调度是操作系统核心程序，在核心态执行。教训：核心程序不用用户干预，都在核心态。

笔记本一句话： 第 24 题＋用户态：缺页、时钟中断、调度都在核心态，命令解释程序在用户态。

【图像】

用户态 vs 核心态：谁在执行？（答案 A）



判用户态还是核心态，看的是"有没有特权、归不归内核管"，不是名字像不像系统组件

用户态与核心态分层图：命令解释程序在用户态，缺页处理、进程调度和时钟中断处理在核心态

【定位】 操作系统 · 第1章 · 操作系统的运行机制

【易混对比】

- "命令解释程序"和"系统调用"的区别：命令解释程序是跑在用户态的用户进程，负责把用户命令翻译成一条条系统调用；系统调用是进入内核的入口，真正的特权操作在核心态完成。
- "缺页处理、进程调度、时钟中断处理"三者的共同点：都要直接操作内核数据结构或硬件（页表、就绪队列、时钟），只能在核心态执行。

四个选项里只有命令解释程序是用户进程，选 A。

【坑位】

"命令解释程序"名字里带"程序"、天天和系统打交道，不等于它跑在核心态——判用户态还是核心态，看的是它有没有特权、归不归内核管。

【套路】

- 第一步，抓"谁在执行"：四个选项中只有一个是面向用户的普通进程。
- 第二步，用特权操作反推：缺页要改页表、调度要操作就绪队列、时钟中断要处理硬件中断，都离不开特权，只能是核心态。

【类比】

像公司前台：命令解释程序是前台接待，听懂客户的话、翻译成内部工单（系统调用）转交给各部门；但前台自己不进机房拔服务器——机房（核心态）只有运维（缺页处理、进程调度、时钟中断处理）能进。

【口诀】 命令解释用户态，缺页调度时钟中断全在核；前台不进机房门，用户态里它一个。

【考频】 「用户态」在 2009–2022 年解析中出现 15 次 · 高频

第 25 题 · 线程独占、进程可共享的资源（正确答案：D）

题干与选项：

在支持多线程的系统中，进程 P 创建的若干个线程不能共享的是( )

- A. 进程 P 的代码段
- B. 进程 P 中打开的文件
- C. 进程 P 的全局变量
- D. 进程 P 中某线程的栈指针

入手点： 进程是分资源的基本单位，线程是跑程序的基本单位；进程里的东西大部分是大家共用的，只有一小部分是每个线程自己私有的。

专有名词从零解释：

- 先想“进程”：一个正在跑的程序，操作系统分给它一整套资源（内存、文件、代码）。
- 再想“线程”：进程里的一条执行路线；同一个进程里的线程共用进程的资源，但每个线程自己跑自己的。
- 再想“栈”：每个线程自己用的、记着函数调用和局部变量的一小块内存；每个线程的栈是分开的。

考查知识点：

- 先想进程的资源：进程的代码段、打开的文件、全局变量等，都是进程资源，同一个进程里的线程都可以共享。
- 再想线程私有的东西：进程中某线程的栈指针属于线程自己——每个线程有自己的栈，栈指针指着自己的栈顶；线程的栈是独享的，对其他线程透明（其他线程看不到、用不了）。
- 再想答案：线程的栈指针就是 D。
- 拿生活例子验算：进程像一个办公室，代码段、文件、全局变量是办公室的公共设备，大家共用；每个线程像办公室里的一个人，每个人有自己的记事本（栈），记事本翻到哪一页（栈指针）是自己的事，别人不碰。
- 边界情况：线程除了栈和栈指针，还有自己的程序计数器和寄存器组，也是私有的；但本题的四个选项里，只有栈指针是那个“线程独占”的答案。
- 易混对子：不要把“代码段”当成线程私有的——代码段是进程的，线程共用；不要把“打开的文件”当成线程私有的——文件是进程打开的，线程共用；不要把“全局变量”当成线程私有的——全局变量是进程的，线程共用。

正确选项为什么对： 线程的栈指针属于线程自己、独享，其他线程透明，就是 D，选 D。

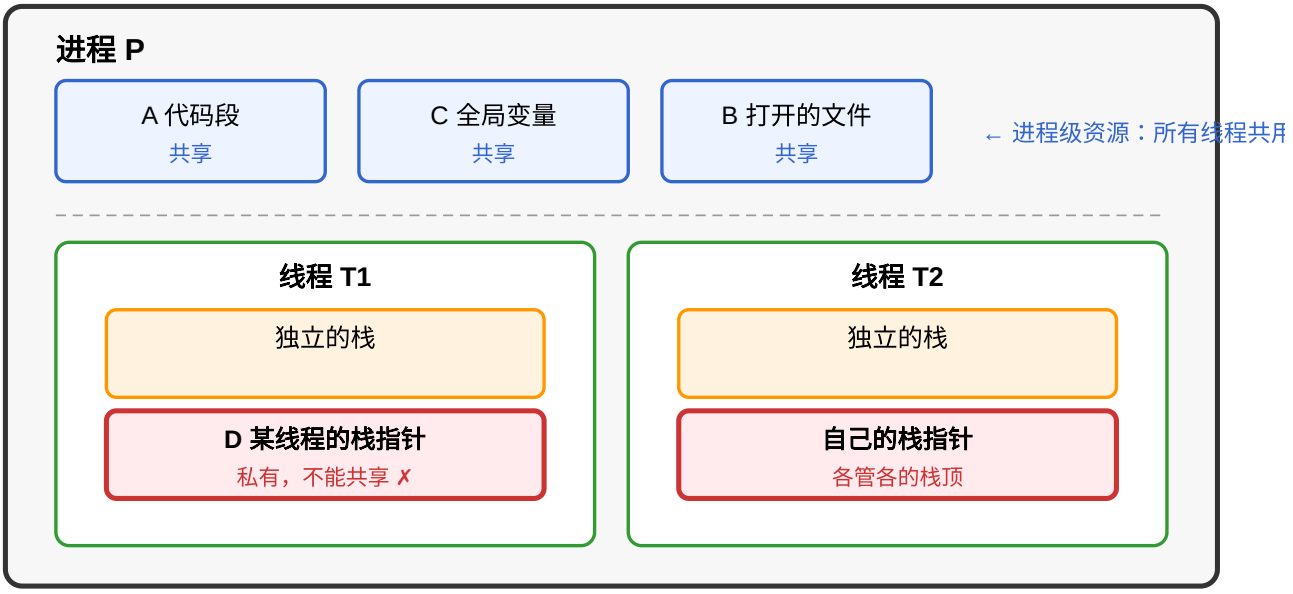
错误选项为什么错＋举例：

- 代码段：进程的资源，线程共用。教训：代码是大家一起读的，不是某个线程私有的。
- 打开的文件：进程打开的，线程共用。教训：文件描述符是进程的，线程都能用。
- 全局变量：进程的，线程共用。教训：全局变量在进程地址空间里，所有线程都看得到。

笔记本一句话： 第 25 题＋线程独占：代码、文件、全局变量是进程共用的，线程的栈指针是独享的。

【图像】

进程 P 的资源：共享 vs 线程私有（答案 D）



线程独享的是"执行现场"（栈、栈指针、寄存器）；进程级资源全共享

进程 P 的地址空间图：代码段、全局变量、打开文件表为进程级共享资源，各线程独享自己的栈与栈指针

【定位】 操作系统 · 第2章 · 线程

【易混对比】

- "栈指针"和"全局变量"的区别：栈指针是线程私有的执行现场，每个线程有自己独立的调用栈、栈指针各管各的；全局变量是进程级资源，所有线程看到的是同一份。
- "打开的文件"和"线程私有资源"的区别：文件描述符表归进程所有，线程共用同一张表；线程私有的只有线程ID、一组寄存器、栈和栈指针这类"正在执行"的现场。

线程独享的是执行现场（栈、栈指针、寄存器），进程级资源全共享，选 D。

【坑位】

别把"栈"当成共享内存——每个线程的函数调用栈是独立的，栈指针指向的是自家栈顶，别的线程够不着。

【套路】

- 第一步，画两栏表：左边"进程级（共享）"、右边"线程级（私有）"。
- 第二步，逐项归类：代码段、打开的文件、全局变量进左边；栈指针进右边，答案就是它。

【类比】

像合租公寓：客厅、厨房、水电表（代码段、全局变量、打开的文件）是整套房子的公共资源，大家共用；但每个人的卧室钥匙（栈指针）只能开自己的门，借不出去。

【口诀】 线程共享进程物，代码文件全局数；栈和指针各一份，私房钥匙不外借。

【考频】 「线程」在 2009–2022 年解析中出现 31 次 · 高频

第 26 题 · 输入输出软件的层次顺序（正确答案：B）

题干与选项：

用户程序发出磁盘 I/O 请求后，系统的正确处理流程是( )

- A. 用户程序→系统调用处理程序→中断处理程序→设备驱动程序
- B. 用户程序→系统调用处理程序→设备驱动程序→中断处理程序
- C. 用户程序→设备驱动程序→系统调用处理程序→中断处理程序
- D. 用户程序→设备驱动程序→中断处理程序→系统调用处理程序

入手点： 用户程序最先发话，操作系统一层一层往下传，最后到硬件；硬件有动静了，再一层一层往上传回来。

专有名词从零解释：

- 先想“用户层”：用户程序自己，发出“我要用设备”的请求。
- 再想“与设备无关的软件层”：也就是系统调用的处理程序；不管用的是什么设备，这一层的处理都一样（比如缓冲、错误报告）。
- 再想“设备驱动程序层”：针对某一种具体设备的程序，知道这个设备的寄存器怎么读写。
- 再想“中断处理程序”：设备有动静（比如数据到了）时，硬件发中断，操作系统跑来处理的那段程序。

考查知识点：

- 先想自上而下的顺序：用户层、与设备无关的软件层、设备驱动程序层、中断处理程序，就是 B。
- 再想请求的走向：用户使用设备时，首先在用户程序中发出一次系统调用；操作系统内核收到请求后，请求调用处理程序处理；再转到相应的设备驱动程序；当设备准备好或所需数据到达后，设备硬件发出中断。
- 再想数据的回路：数据按上述调用顺序逆向传到用户程序中——中断处理程序把数据交给驱动程序，驱动程序交给系统调用处理程序，最后回到用户程序。
- 拿生活例子验算：用户层像顾客点菜，与设备无关的软件层像总服务台（不管点什么菜，流程一样），设备驱动程序层像某个菜系的厨师（只会做自己那类菜），中断处理程序像上菜的服务员（菜好了喊一声）；点菜从上往下传，上菜从下往上传。
- 边界情况：中断处理程序在最底层，但它不是“用户调用”的那一层——它是硬件有动静时才跑的；层次顺序说的是软件结构，不是调用时间。
- 易混对子：不要把中断处理程序放到最上面——它在最底层，离硬件最近；不要把设备驱动程序和与设备无关的软件层顺序弄反——先是与设备无关的统一处理，再是针对具体设备的驱动。

正确选项为什么对： 自上而下是用户层、与设备无关的软件层、设备驱动程序层、中断处理程序，就是 B，选 B。

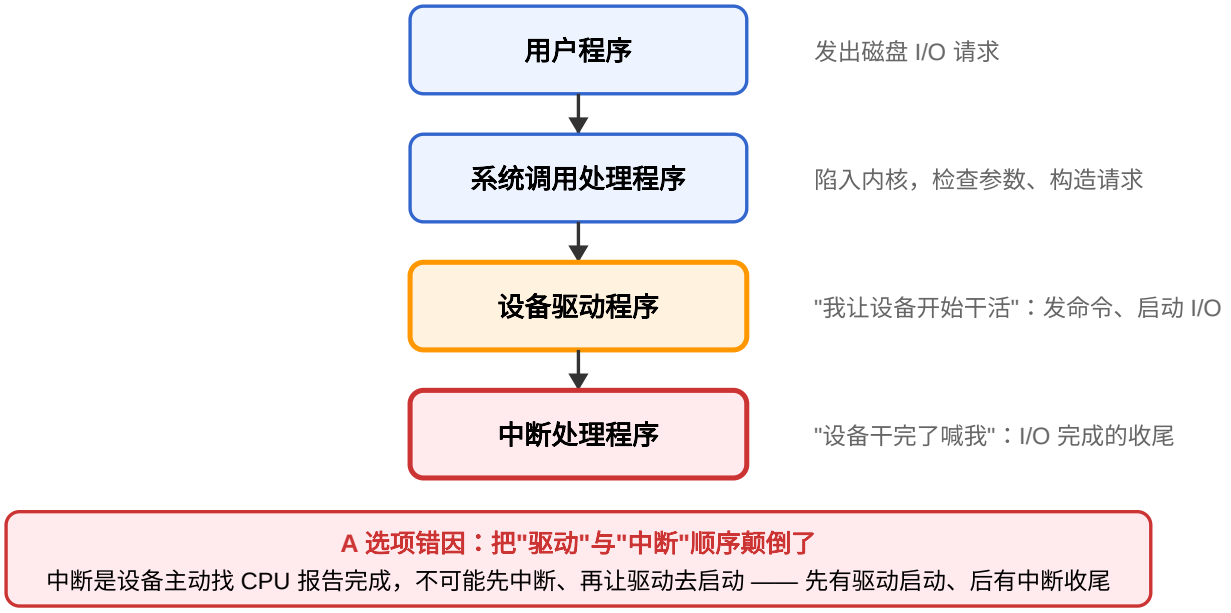
错误选项为什么错＋举例：

- 把中断处理程序放错位置的顺序：中断处理程序在最底层，离硬件最近。教训：层次从上到下是用户、统一处理、驱动、中断。
- 把驱动层和无关层顺序弄反的顺序：先是与设备无关的统一处理，再是具体设备的驱动。教训：统一处理在上，具体驱动在下。

笔记本一句话： 第 26 题＋I/O 软件四层：用户层、与设备无关层、驱动层、中断处理程序，自上而下。

【图像】

I/O 软件层次：自上而下的处理流程（答案 B）



I/O 软件四层处理流程图：用户程序→系统调用处理程序→设备驱动程序→中断处理程序，标注 A 选项错在驱动与中断顺序颠倒

【定位】 操作系统 · 第5章 · I/O 软件的层次结构

【易混对比】

- "设备驱动程序"和"中断处理程序"的区别：驱动是"我让设备开始干活"——向控制器发命令、启动 I/O；中断处理是"设备干完了喊我"——I/O 完成后的收尾工作；启动一定在完成之前，所以驱动在前、中断在后。
- "系统调用处理程序"和"用户程序"的区别：用户程序在用户态发出 I/O 请求，系统调用处理程序是陷入内核后的第一站，负责检查参数、构造 I/O 请求，再往下交给驱动。

自上而下：用户程序→系统调用处理程序→设备驱动程序→中断处理程序，选 B。

【坑位】

A 和 B 只差后两项的顺序——记住"先有驱动启动、后有中断收尾"：中断是设备主动找 CPU 报告完成，不可能先中断、再让驱动去启动。

【套路】

- 第一步，按时间线走一遍：用户发起请求→陷入内核（系统调用）→驱动给设备下命令→设备做完发中断→中断处理收尾。
- 第二步，卡住易反的两项：驱动启动在前、中断收尾在后，顺序反了的选项直接划掉。

【类比】

像点外卖：你下单（用户程序）→平台接单转给厨房（系统调用）→厨师开始炒菜（驱动启动设备）→菜好了服务员喊你取餐（中断）→你取餐走人（中断处理收尾）。不可能是服务员先喊你、厨师再开始炒菜。

【口诀】 用户发起系统调，驱动启动中断收；B 选项顺序对，A 颠倒要划掉。

【考频】 「I/O软件层次」在 2009–2022 年解析中出现 14 次 · 高频

第 27 题 · 银行家算法／试分配后能否找到安全序列（正确答案：D）

题干与选项：

某时刻进程的资源使用情况如下表所示。

|进程|已分配资源 R1|已分配资源 R2|已分配资源 R3|尚需 R1|尚需 R2|尚需 R3|

|---|---|---|---|---|---|---|

|P1|2|0|0|0|0|1|

|P2|1|2|0|1|3|2|

|P3|0|1|1|1|3|1|

|P4|0|0|1|2|0|0|

可用资源：R1=0，R2=2，R3=1。此时的安全序列是( )

- A. P1，P2，P3，P4
- B. P1，P3，P2，P4
- C. P1，P4，P3，P2
- D. 不存在

> 注：资源表格转录自原卷，A、B、C 各序列与首个数的细节以原卷为准

入手点： 安全序列的意思是：按某个顺序跑，每个进程轮到时手里的资源都够它跑完，跑完还把资源还回来；试分配之后如果怎么排都卡住，就是没有安全序列。

专有名词从零解释：

- 先想“银行家算法”：操作系统放贷的规矩——只有按某个安全顺序能让所有进程都跑完，才敢把资源借出去；借出去之后系统还必须是安全的。
- 再想“安全序列”：一个进程排队顺序，按这个顺序跑，每个进程轮到时资源都够，跑完还把资源还回来，后面越跑越顺。
- 再想“试分配”：先假装把资源分出去，看看分完之后系统还安不安全；不安全就不真分。

（注：本节标注部分核对——A、B、C 各自的序列与部分数字在源材料中未齐，这里只按排除逻辑讲，不拼凑各选项的序列。）

考查知识点：

- 先想排除 A、B 的路：当剩余资源分配给 P1，待 P1 执行完后，可用资源数变成（首个数未看清，2，1）；这时仅能满足 P3 的需求；按这条路走下去，后面会卡住，所以以 P1 开头的试分配方案走不通，排除 A、B。
- 再想排除 C 的路：接着分配给 P4，待 P4 执行完后，可用资源数变成 (2，2，2)；这时已经无法满足任何进程的需求，同样卡住，所以 C 的方案也走不通，排除 C。
- 再想结论：A、B、C 的方案都走不通，试分配之后不存在安全序列，就是 D（D 表示不存在，即无安全序列，不是另一条试分配序列）。
- 拿生活例子验算：银行手里剩的钱先借给 P1，P1 还回来之后只够 P3 用；P3 用完再借给 P4，P4 还回来之后谁都不够用——这条路借到一半就断了，说明这笔贷款不能放；其他几条路同样断在半路，所以干脆不放。

- 边界情况：安全序列不要求每个进程一开始就够——只要按某个顺序，每个进程轮到时都够就行；但如果每条路都卡住，那就确实没有安全序列。
- 易混对子：不要以为“第一个进程够”就等于“有安全序列”——第一个够了，后面卡住也不算；不要把“试分配”和“真分配”混在一起——试分配只是假装分一次看看，分完不安全就不真分。

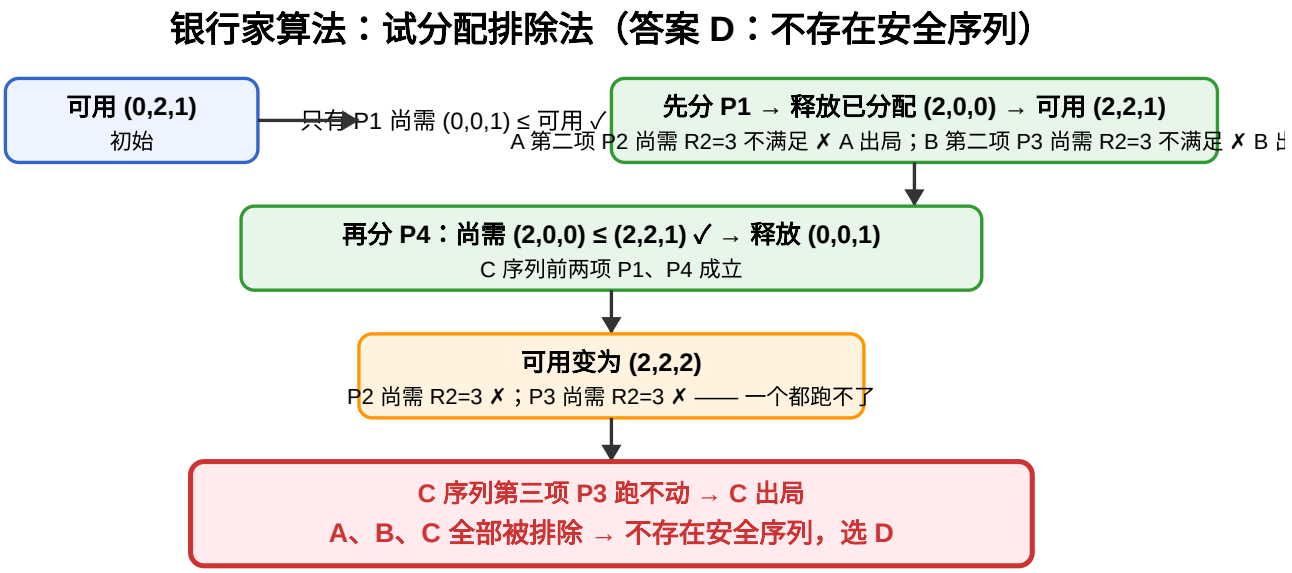
正确选项为什么对： A、B、C 的试分配路都卡住，不存在安全序列，D 表示不存在，就是 D，选 D。

错误选项为什么错＋举例：

- 以 P1 开头的方案（A、B）：P1 跑完之后只剩 P3 能跑，后面走不通。教训：安全序列要整条验，不能只看开头。
- 分到 P4 之后卡住的方案（C）：P4 跑完之后可用资源是 (2，2，2)，哪个进程都不够。教训：中途卡住，整条路就不安全。

笔记本一句话： 第 27 题＋银行家试分配：A、B、C 的路都卡住，不存在安全序列，选“不存在”。

【图像】



口诀：尚需 ≤ 可用才能跑；跑完一定要把"已分配"加回可用再往下判

资源表：P1 已分配(2,0,0)尚需(0,0,1)；P2 已分配(1,2,0)尚需(1,3,2)；P3 已分配(0,1,1)尚需(1,3,1)；P4 已分配(0,0,1)尚需(2,0,0)

注：A、B、C 序列细节以原卷为准，此处只做排除逻辑演示

银行家算法试分配排除流程图：从可用(0,2,1)出发试分配 P1、P4，最终 P2、P3 都跑不动，不存在安全序列

【定位】 操作系统 · 第2章 · 死锁避免（银行家算法）

【易混对比】

- "试分配排除法"和"从头拼安全序列"的区别：前者拿选项当剪刀——选项的每一项都必须在当时可用资源下"跑得动"，第一处跑不动整个选项出局；后者自己找一条完整序列，费时且容易在分支上绕晕。本题用排除法更快。
- "尚需资源"和"已分配资源"的区别：判断能不能跑看的是"尚需≤当前可用"；跑完之后要加回的是"已分配"，两列别用反。

三个候选序列在试分配中全部被排除，不存在安全序列，选 D。

【坑位】

跑完一个进程，一定要把它的"已分配"加回可用资源再往下判断——忘了加回，后面的进程会误判为跑不动。

【套路】

- 第一步，找起点：初始可用 (0,2,1)，只有 P1 的尚需 (0,0,1) 满足"尚需≤可用"，先分 P1；A 序列第二项 P2 尚需 R2=3 不满足，A 出局；B 序列第二项 P3 尚需 R2=3 不满足，B 出局。
- 第二步，加回再判：P1 跑完释放已分配 (2,0,0)，可用变为 (2,2,1)；此时 P4 的尚需 (2,0,0) 满足，再分 P4（C 序列前两项 P1、P4 成立）。
- 第三步，收尾验证：P4 跑完释放 (0,0,1)，可用变为 (2,2,2)；P2、P3 的尚需 R2=3 都不满足，C 序列第三项 P3 跑不动，C 出局——三个选项全灭，选 D。

【类比】

像银行放贷审查：银行家手里只有 (0,2,1) 的现金，先借给唯一还得起的 P1；P1 还款后手头变成 (2,2,1)，再借给 P4；P4 还款后手头 (2,2,2)，但 P2、P3 要的 R2 都是 3，借不起——没有一种放款顺序能让所有人都还上款，系统不安全。

【口诀】 银行家算法防死锁，试分配来逐个过；尚需不大于可用，跑完加回再抉择；三路全灭无序列，D 选项不存在。

【考频】 「银行家算法」在 2009–2022 年解析中出现 8 次 · 中频

第 28 题 · 缺页中断处理过程（正确答案：D）

题干与选项：

在缺页处理过程中，操作系统执行的操作可能是( )

Ⅰ. 修改页表

Ⅱ. 磁盘 I/O

Ⅲ. 分配页框

- A. 仅Ⅰ、Ⅱ
- B. 仅Ⅱ
- C. 仅Ⅲ
- D. Ⅰ、Ⅱ和Ⅲ

入手点： 缺页就是“要的数据不在内存里”；处理就是三步：找个空位、把数据从外存搬进来、把页表改对。

专有名词从零解释：

- 先想“缺页中断”：程序要访问的数据不在内存里，访问时触发的中断；操作系统要把数据从外存调进来，程序才能接着跑。
- 再想“页框”：内存里放一页数据的位置；空闲页框就是还没放数据的空位置。
- 再想“页面置换”：内存里没有空闲页框时，把某页换出去、腾出位置。
- 再想“页表”：记着每一页在不在内存、在哪个页框的表；有效位是页表里表示“这一页在不在内存”的标志位。

考查知识点：

- 先想第一步：缺页中断产生后，需要在内存中找到空闲页框，分配给需要访问的页；如果没有空闲页框，就要做页面置换，腾出一个。
- 再想第二步：缺页中断处理程序调用设备驱动程序做磁盘 I/O，把位于外存上的页面调入内存。
- 再想第三步：调入之后修改页表——把页表中代表该页是否在内存的标志位（有效位）置为 1，并把物理页框号填入相应位置；如果必要，还要修改其他相关表项。
- 再想答案：这三步都对，D 是Ⅰ、Ⅱ、Ⅲ三项均包括在内（D 本身不是一条独立陈述），就是 D。
- 拿生活例子验算：缺页像要看的书不在书架上——先在书架上找个空位（没有空位就把一本不常用的书先收起来），再去仓库把书搬回来，最后在目录卡上记下“这本书在书架第几格”。
- 边界情况：页面置换不是每次缺页都做——有空闲页框时直接分配，只有没空位时才置换；页表改完之后，缺页的指令要重新执行一次，这次数据已经在内存里了，就能接着跑。
- 易混对子：不要把“调入内存”和“修改页表”的顺序弄反——先把数据搬进来，再改页表记下它在哪；不要以为缺页时进程继续跑——数据没调进来之前，进程只能阻塞等着。

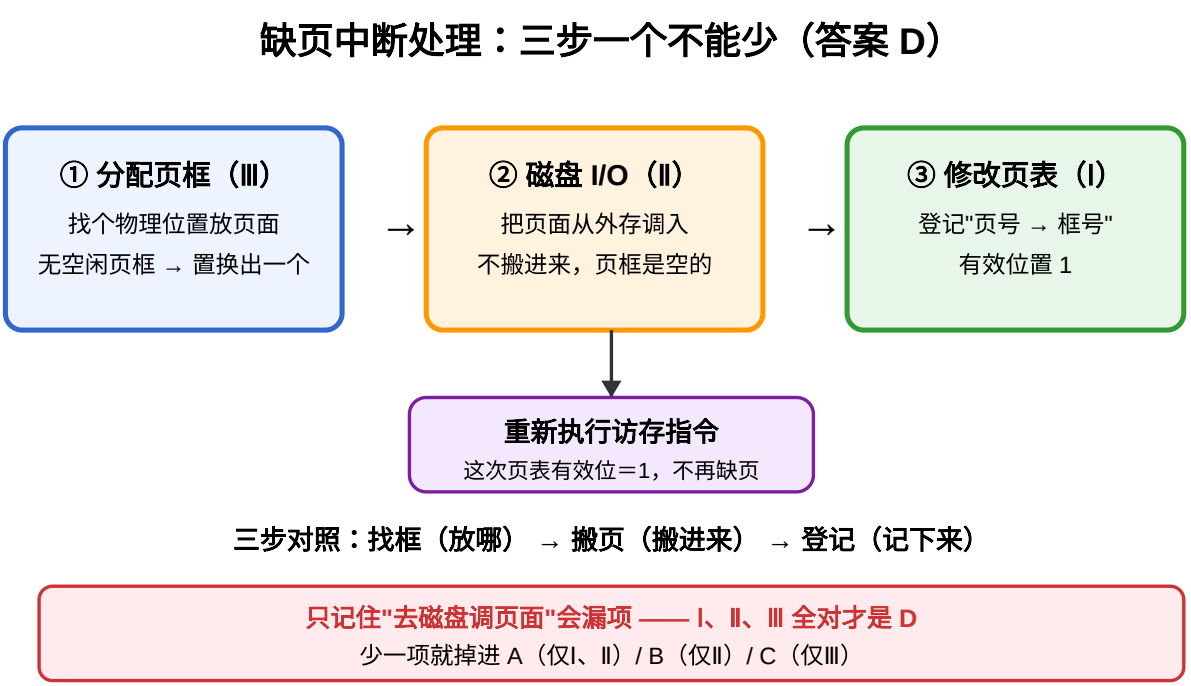
正确选项为什么对： 找空闲页框、调设备驱动程序做磁盘 I/O 调入、改页表有效位和页框号，三步都对，D 是三项全包括，就是 D，选 D。

错误选项为什么错＋举例：

- 只包括其中一步或两步的选项：三步缺一不可，少任何一步，缺页处理都不完整。教训：找位、搬数据、改页表，三步都要有。

笔记本一句话： 第 28 题＋缺页处理三步：找空闲页框、调入内存、改页表有效位和页框号。

【图像】



缺页中断处理三步流程图：分配页框→磁盘 I/O 调入页面→修改页表，最后重新执行指令

【定位】 操作系统 · 第3章 · 虚拟内存管理（缺页中断处理）

【易混对比】

- "分配页框"和"修改页表"的区别：分配页框是找个物理位置放页面（没有空闲就置换出一个）；修改页表是把"页号→框号"的映射登记进页表、把有效位置 1；前者是找地方，后者是登记。
- "磁盘 I/O"在这三步里的位置：页面内容在外存，不经过磁盘 I/O 搬进来，页框是空的、页表登记了也没用，所以它是中间必经的一步。

找框、搬页、登记，三步一个不能少， Ⅰ 、 Ⅱ 、Ⅲ全选，选 D。

【坑位】

只记住"缺页就是去磁盘调页面"，会漏掉分配页框和修改页表——三项全对才是 D，少一项就掉进 A/B/C。

【套路】

- 按时间线数动作：页框（页面放哪）→磁盘 I/O（把页面搬进来）→改页表（登记框号、有效位置 1）→重新执行访存指令。

【类比】

像图书馆借书发现书不在架上：先找个空书架格（分配页框，格子满了就腾一格）→去仓库把书搬来（磁盘 I/O）→在目录卡上登记这本书的位置（修改页表）→回去重新借书。

【口诀】 缺页处理有三步，找框搬盘改页表； Ⅰ Ⅱ Ⅲ全都要，D 选项别漏掉。

【考频】 「缺页中断」在 2009–2022 年解析中出现 25 次 · 高频

第 29 题 · 抖动现象的防止方法（正确答案：A）

题干与选项：

当系统发生抖动（thrashing）时，可以采取的有效措施是( )

Ⅰ. 撤销部分进程

Ⅱ. 增加磁盘交换区的容量

Ⅲ. 提高用户进程的优先级

- A. 仅 Ⅰ
- B. 仅 Ⅱ
- C. 仅Ⅲ
- D. 仅 Ⅰ、 Ⅱ

入手点： 抖动就是页面换来换去、CPU 光顾着换页不干活；要治它，就得让内存里同时跑的进程少一点、用的页面少一点。

专有名词从零解释：

- 先想“对换”：把内存里的进程暂时换到外存、腾出内存，之后再换回来。
- 再想“文件区和对换区”：外存分成两块——文件区存文件，对换区存从内存换出的进程。
- 再想“抖动”：刚刚被换出的页很快又要被访问，为此又要换出其他页，而那页又很快被访问；如此频繁地置换页面，大部分时间都花在页面置换上，系统性能下降。

考查知识点：

- 先想抖动是怎么来的：内存里同时跑的进程太多，每个进程分到的页框太少，刚换出去的页马上又要用，换进来又要换出去别的页，CPU 的时间全花在换页上，正事没干多少。
- 再想撤销部分进程为什么管用：撤销部分进程，减少了同时跑的进程数，也就减少了所用到的页面数，剩下的进程能分到更多页框，换页次数下来了，抖动就防止住了，就是 A。
- 再想为什么对换区大小和进程优先级不管用：对换区大小只决定外存能存多少换出的进程，不决定内存里换页的频繁程度；进程优先级只决定谁先跑，不决定页面够不够用；这两个都和抖动无关。
- 拿生活例子验算：抖动像一张小桌子挤了太多人写作业，每个人只有一小块地方，书换来换去，时间全花在换书上；赶走几个人，剩下的人地方大了，书不用老换，作业就写得顺了。
- 边界情况：撤销进程是比较狠的办法，实际系统里还会用工作集、缺页率控制等更精细的办法；但本题的选项里，只有撤销部分进程是防止抖动的有效办法。
- 易混对子：不要把“对换区”和“内存”混在一起——对换区在外存，抖动是内存页框不够闹的，改外存分区大小治不了；不要以为“提高优先级”能治抖动——优先级管的是谁先跑，不是页面够不够。

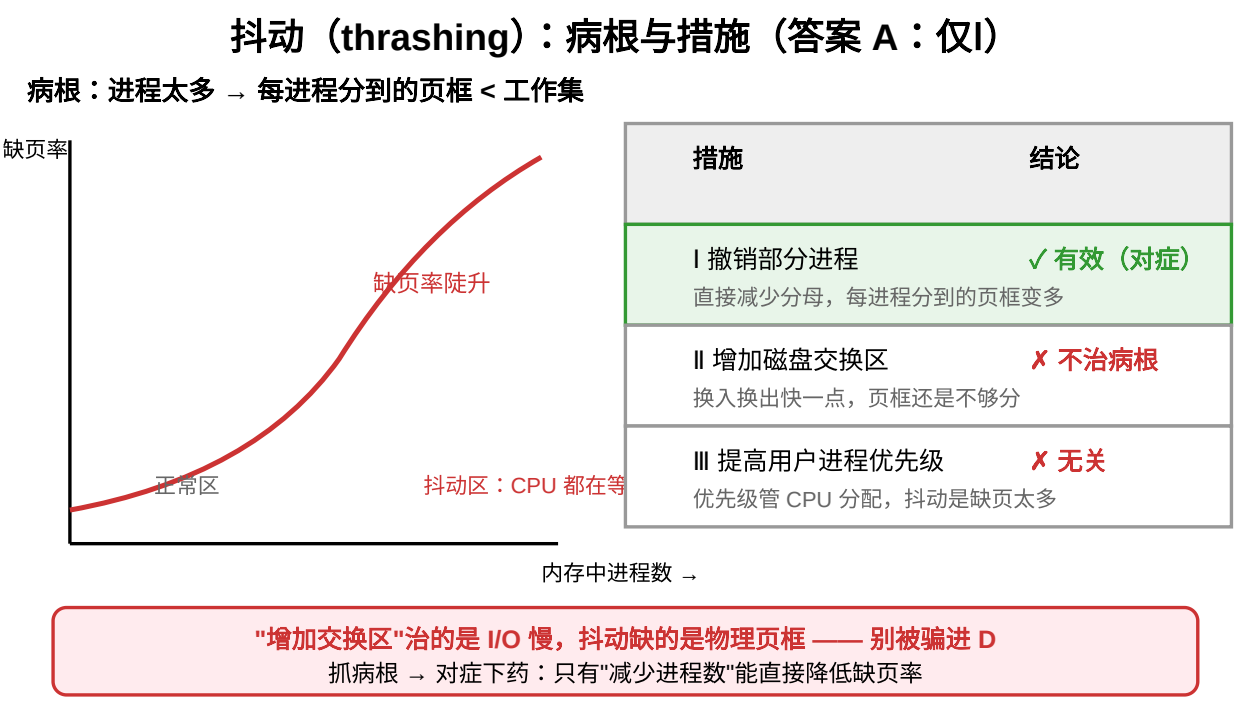
正确选项为什么对： 撤销部分进程减少所用页面数，剩下的进程页框够了，抖动就防止住了，就是 A，选 A。

错误选项为什么错＋举例：

- 改对换区大小：对换区在外存，只决定能存多少换出的进程，不决定内存换页的频繁程度。教训：抖动是内存页框不够的问题，不是外存分区的问题。
- 改进程优先级：优先级只决定谁先跑，不决定页面够不够用。教训：谁先跑和页面够不够是两回事。

笔记本一句话： 第 29 题＋防抖动：撤销部分进程，减少所用页面数，剩下的进程页框够了。

【图像】



抖动示意图：内存中进程数过多导致缺页率陡升，三种措施中只有撤销部分进程对症

【定位】 操作系统 · 第3章 · 虚拟内存管理（抖动）

【易混对比】

- "撤销部分进程"和"增加磁盘交换区"的区别：抖动的病根是内存中进程太多、每进程分到的页框不够工作集；撤销进程是直接减少分母、对症下药；增加交换区只是让换入换出快一点，不改变"页框不够分"的病根。
- "提高优先级"和抖动的关系：优先级决定 CPU 先给谁用，抖动是缺页太多、CPU 都在等 I/O，两者不在一个维度上，提高优先级治不了抖动。

抖动的病根是页框不够分，只有撤销部分进程对症，选 A。

【坑位】

"增加交换区"听起来像在治 I/O 慢，但抖动缺的不是交换区、是物理页框——加交换区治标不治本，别被它骗进 D。

【套路】

- 第一步，抓病根：抖动=缺页率过高=进程太多、每进程分到的页框装不下工作集。
- 第二步，对症下药：只有减少进程数能直接降低缺页率，II、III都不碰病根。

【类比】

像教室座位不够、学生太多，大家不停起身换座位（缺页）：解决办法是让一部分学生先出去（撤销进程），而不是把走廊加宽（增加交换区）或给某些学生发 VIP 牌（提高优先级）——走廊再宽，座位还是不够。

【口诀】 抖动病根框不够，撤销进程才对症；加区提权都不治，仅 I 能选 A 落定。

【考频】 「抖动」在 2009–2022 年解析中出现 6 次 · 中频

第 30 题 · 逻辑地址在哪一阶段产生（正确答案：C）

题干与选项：

在虚拟内存管理中，地址变换机构将逻辑地址变换为物理地址，形成该逻辑地址的阶段是( )

- A. 编辑
- B. 编译
- C. 链接
- D. 装载

入手点： 关键是先分清本题说的“逻辑地址”指什么：它指的是各模块里的段内偏移量，不是链接之后整个程序的全局地址空间；段内偏移量在编译时就定下来了。

专有名词从零解释：

- 先想“逻辑地址”：程序里用的地址；本题里指的是各模块中的段内偏移量，也就是相对于当前模块首址的偏移。
- 再想“线性地址”：分页存储管理中，单个程序所有模块集合在一起构成的地址；和“段内偏移量”不是同一个东西。
- 再想“物理地址”：CPU 外部地址总线上寻址物理内存的地址信号，是地址变换的最终结果。
- 再想 C 语言的编译流程：预处理（cpp）→编译（cc1）→汇编（as）→链接（ld）产生可执行文件；链接前一步产生的是可重定位的二进制目标文件。

考查知识点：

- 先想各模块独立编译：C 语言采用源文件独立编译的方法；main.c、file1.c、file2.c 等在链接前一步生成了 main.o、file1.o、file2.o；这些目标模块采用的逻辑地址都从 0 开始，但只是相对于该模块的逻辑地址（段内偏移量）。
- 再想链接干什么：链接器把这三个文件、libc 和其他库文件链接成一个可执行文件；链接阶段主要完成了重定位，形成整个程序的完整逻辑地址空间；比如 file1.o 逻辑地址为 0～1023，main.o 逻辑地址为 0～1023，假设链接时将 file1.o 链接在 main.o 之后，则重定位之后 file1.o 对应的逻辑地址就应为 1024～2047。
- 再想本题的答案：链接后形成的那个“整个程序的完整逻辑地址空间”不是本题说的逻辑地址；本题的逻辑地址指的是段内的偏移量，而不是链接后生成的整个程序全局的逻辑地址空间；所以逻辑地址是编译时产生的，就是 C。
- 再想区分三个地址：逻辑地址是各模块中的偏移地址（相对于当前模块首址）；线性地址是分页存储管理中单个程序所有模块集合在一起构成的地址；物理地址是 CPU 外部地址总线上寻址物理内存的地址信号，是地址变换的最终结果。
- 拿生活例子验算：每个文件独立编译时，自己的页码都从第 1 页编起（段内偏移量，编译时就有）；装订成一本书（链接）之后，整本书的页码重新排过（全局逻辑地址空间）；题目问的是“每个文件自己的页码什么时候编的”，答案是编译时，不是装订时。
- 边界情况：如果题目问的是“整个程序的完整逻辑地址空间什么时候形成”，答案就是链接时；本题问的是“逻辑地址”在段内偏移量意义下的产生阶段，答案是编译时；两个问法答案不同，审题时要分清。
- 易混对子：不要把“链接后形成完整逻辑地址空间”和“逻辑地址在编译时产生”当成矛盾——前者说的是全局地址空间，后者说的是段内偏移量，两个说的不是同一个东西；不要把逻辑地址、线性地址、物理地址混在一起——逻辑地址是模块内偏移，线性地址是模块集合，物理地址是总线上的最终地址。

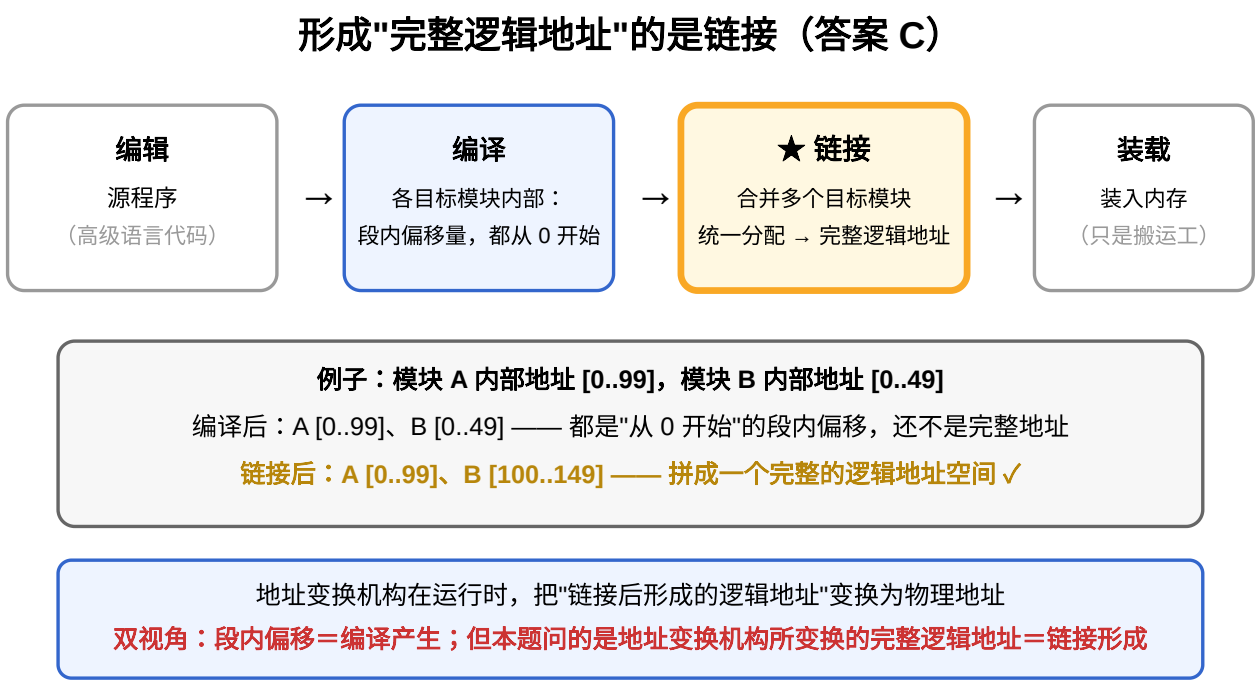
正确选项为什么对： 本题的逻辑地址指段内偏移量，各模块编译时就从 0 开始编了，所以是编译时产生，就是 C，选 C。

错误选项为什么错＋举例：

- 选链接时：如果问的是“整个程序的完整逻辑地址空间”，链接时是对的；但本题的逻辑地址指段内偏移量，编译时就产生了。教训：先看题目说的逻辑地址是哪种意义。
- 选汇编时：汇编只是把汇编语言变成机器语言，地址（段内偏移量）在编译阶段就已经定下来了。教训：编译产生偏移量，汇编不产生新的地址。

笔记本一句话： 第 30 题＋逻辑地址阶段：段内偏移量是编译时产生的，链接形成的是全局逻辑地址空间，两回事。

【图像】



程序装入链接三阶段地址变化图：编译产生各模块段内偏移，链接合并模块形成完整逻辑地址，装载后地址变换机构在运行时变换为物理地址

【定位】 操作系统 · 第3章 · 程序的装入和链接

【易混对比】

- "编译产生的段内偏移量"和"链接形成的逻辑地址"的区别：编译只给每个目标模块内部"从 0 开始"的相对地址；地址变换机构要变换的是整个程序的完整逻辑地址——这个完整地址是链接把多个模块拼起来、统一分配后才形成的。
- "装载"和"链接"的区别：装载是把链接好的程序放进内存（决定装入时还是运行时的地址变换方式），但它不产生逻辑地址，只是搬运工。

地址变换机构变换的是链接后形成的完整逻辑地址，选 C。

【坑位】

只记住"编译产生地址"会误选 B——编译产生的是各模块内部的段内偏移量；本题问的是地址变换机构所变换的那个完整逻辑地址，那是链接形成的。双视角缺一不可。

【套路】

- 第一步，定位"谁用这个地址"：地址变换机构变换的是完整的逻辑地址空间。
- 第二步，倒推谁生成它：完整的逻辑地址空间由链接合并多个目标模块、统一分配地址后形成。

【类比】

像盖小区：编译是各施工队在自己的图纸上从 1 号编房间号（段内偏移）；链接是总包把几栋楼拼成一个小区、重新统一编门牌号（完整逻辑地址）；住户（地址变换机构）拿着统一门牌号找房间——门牌号是总包编的，不是各施工队编的。

【口诀】 编译只给段内偏，链接拼出整地址；变换机构认整址，C 选项链接是正解。

【考频】 「逻辑地址」在 2009–2022 年解析中出现 21 次 · 高频

第 31 题 · 单缓冲与双缓冲完成磁盘块传输的总时间（正确答案：B）

题干与选项：

某文件占 10 个磁盘块，现要把该文件磁盘块逐个读入主存缓冲区，并送用户区进行分析。假设一个缓冲区与一个磁盘块大小相同，把一个磁盘块读入缓冲区的时间为 100μs，将缓冲区的数据传送到用户区的时间是 50μs，CPU 对一块数据进行分析的时间为 50μs。在单缓冲区和双缓冲区结构下，读入并分析完该文件的时间分别是( )

- A. 1500μs，1000μs
- B. 1550μs，1100μs
- C. 1550μs，1550μs
- D. 2000μs，2000μs

入手点： 单缓冲是“一块一块串着来”，双缓冲是“读盘和传输处理并行”；算总时间时，单缓冲每块都要等，双缓冲只算读盘时间加最后的收尾。

专有名词从零解释：

- 先想“单缓冲”：只有一个缓冲区；上一个磁盘块从缓冲区读入用户区完成之后，下一个磁盘块才能开始读入。
- 再想“双缓冲”：有两个缓冲区；一个缓冲区在往用户区送数据的时候，另一个缓冲区可以同时从外存读下一个磁盘块，两件事并行。
- 再想本题的时间：一块从外存读入缓冲区要 100μs，从缓冲区送到用户区要 50μs，处理一块要 50μs；一共 10 块。

考查知识点：

- 先想单缓冲的总时间：每块都要等前一块从缓冲区读入用户区完成，下一块才能开始读入；每块是读盘 100μs 加送用户区 50μs，一共 150μs；10 块就是 150×10＝1500μs；再加上处理最后一个磁盘块的时间 50μs，共 1550μs。
- 再想双缓冲的总时间：不存在等待磁盘块从缓冲区读入用户区的问题，10 个磁盘块可以连续从外存读入主存缓冲区，读盘时间是 100×10＝1000μs；再加上将最后一个磁盘块从缓冲区送到用户区的传输时间 50μs，以及处理最后一个磁盘块的时间 50μs，共 100×10＋50＋50＝1100μs，就是 B。
- 拿生活例子验算：单缓冲像只有一个篮子，装一篮菜（读盘）再送一篮菜（送用户区），送完才能装下一篮；双缓冲像两个篮子轮换，一个篮子在送菜的时候，另一个篮子已经在装下一篮菜，装菜的时间连起来了，最后只多算最后一次送菜和处理。
- 边界情况：双缓冲省的是“等待送用户区”的时间，不是省读盘时间——读盘 10 次一共 1000μs 一次都省不了；省下来的是中间 9 次等待送用户区的时间。
- 易混对子：不要把单缓冲的 1550μs 当成双缓冲的答案——双缓冲并行了，时间更短；不要漏了最后一次送用户区和处理的时间——双缓冲最后那块还是要送、要处理，50＋50 不能漏。

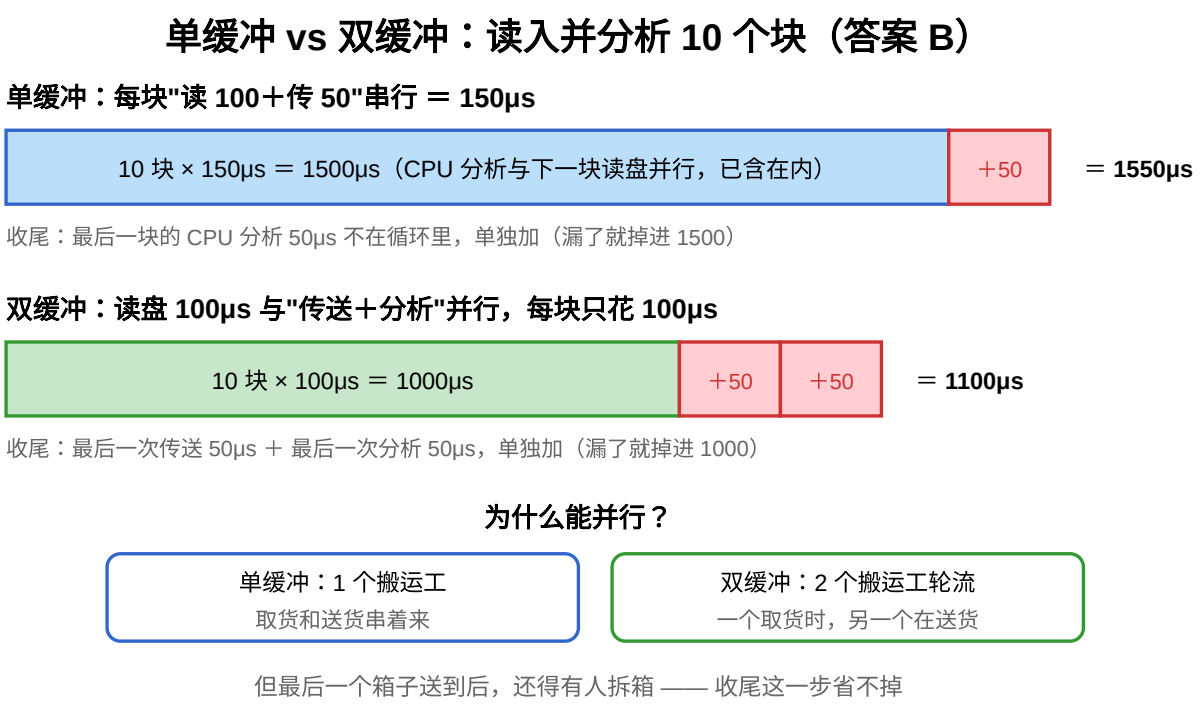
正确选项为什么对： 双缓冲读盘连续 1000μs，加最后送用户区 50μs、处理 50μs，共 1100μs，就是 B，选 B。

错误选项为什么错＋举例：

- 1550μs：那是单缓冲的总时间，不是双缓冲的。教训：单缓冲每块都等，双缓冲并行，时间不一样。
- 只算 1000μs：漏了最后一个磁盘块送用户区和处理的时间。教训：最后那块的 50＋50 也要算。

笔记本一句话： 第 31 题＋单双缓冲：单缓冲 1550μs，双缓冲读盘连续 1000μs＋收尾 100μs＝1100μs。

【图像】



单缓冲与双缓冲读入分析时间线对比图：单缓冲每块 150μs 串行、双缓冲每块 100μs 并行，尾部分别加收尾时间

【定位】 操作系统 · 第5章 · 缓冲管理

【易混对比】

- "单缓冲"和"双缓冲"的区别：单缓冲下每块的"读盘 100μs＋传送 50μs"是串行的 150μs，CPU 分析只能和下一块的读盘并行；双缓冲下读盘和"传送＋分析"并行，每块只花读盘的 100μs。
- "循环体"和"收尾"的区别：循环体内算的是 max 并行后的每块时间，最后一块的尾巴（单缓冲的最后一次分析、双缓冲的最后一次传送＋分析）不在循环里，要单独加。

单缓冲 10×150＋50＝1550μs，双缓冲 10×100＋50＋50＝1100μs，选 B。

【坑位】

最后一块的收尾别漏：单缓冲要加最后一次 CPU 分析 50μs，双缓冲要加最后一次传送 50μs 和分析 50μs——漏了就会掉进 A 选项的 1500/1000。

【套路】

- 第一步，画时间线定每块时间：单缓冲每块串行 100＋50＝150μs；双缓冲读盘 100μs 与传送＋分析并行，每块 100μs。
- 第二步，收尾单独加：单缓冲加最后一次分析 50μs，双缓冲加最后一次传送 50μs 和分析 50μs。

【类比】

像搬箱子进仓库再拆箱：单缓冲是一个搬运工，取货（读盘）和送货（传送）只能串着来；双缓冲是两个搬运工轮流，一个去取货时另一个在送货。但最后一个箱子送到后，还得有人拆箱（收尾）——这一步省不掉。

【口诀】 单缓百五乘十块，尾加分析五十来；双缓一百乘十块，传送分析尾再排；一五五零一一零零，B 选项里对得来。

【考频】 「缓冲管理」在 2009–2022 年解析中出现 2 次 · 低频

第 32 题 · 并发执行交错后不可能出现的结果（正确答案：C）

题干与选项：

有两个并发执行的进程 P1 和 P2，共享初值为 1 的变量 x。P1 对 x 加 1，P2 对 x 减 1。加 1 和减 1 操作的指令序列分别如下所示。

//加 1 操作：①load R1,x（取 x 到寄存器 R1 中）②inc R1 ③store x,R1（将 R1 的内容存入 x）

//减 1 操作：④load R2,x ⑤dec R2 ⑥store x,R2

两个操作完成后，x 的值( )

- A. 可能为-1 或 3
- B. 只能为 1
- C. 可能为 0、1 或 2
- D. 可能为-1、0、1 或 2

入手点： 把 6 条语句编号，按各种交错顺序跑一遍，看哪些结果能跑出来、哪个跑不出来。

专有名词从零解释：

- 先想“并发执行”：两个进程轮流用 CPU，它们的语句可以交错着执行，顺序不固定。
- 再想“交错”：P1 的语句和 P2 的语句可以按任何顺序穿插，但每个进程自己的 3 条语句相对顺序不能变（1 在 2 前面，2 在 3 前面；4 在 5 前面，5 在 6 前面）。

考查知识点：

- 先想编号：P1 的 3 条语句依次编号为 1、2、3，P2 的 3 条语句依次编号为 4、5、6。
- 再想几种交错的结果：依次执行 1、2、3、4、5、6，得结果 1；依次执行 1、2、4、5、6、3，得结果 2；执行 4、5、1、2、3、6，得结果 0。
- 再想不可能的结果：结果 1、2、0 都能跑出来，结果 -1 在任何交错顺序下都跑不出来，就是 C。
- 拿生活例子验算：两个人轮流往一个数上加减，P1 的三步和 P2 的三步各有自己的先后顺序；把 6 步按各种穿插法排一遍，能排出的结果有 1、2、0，-1 怎么排都排不出来。
- 边界情况：交错顺序的总数是 6 条语句里选 3 个位置给 P1（其余给 P2），一共 20 种；但结果只有几种，因为很多交错顺序算出来是同一个结果；判断“不可能”要所有交错顺序都跑不出来才算。
- 易混对子：不要只试一两种交错顺序就下结论——“不可能”要所有顺序都试过才算；不要把“结果 -1”想成“某条语句单独跑出来的”——结果是 6 条语句交错之后共同算出来的，不是某一条语句的结果。

正确选项为什么对： 交错顺序能跑出 1、2、0，-1 怎么排都跑不出来，就是 C，选 C。

错误选项为什么错＋举例：

- 结果 1：按 1、2、3、4、5、6 的顺序能跑出来。教训：P1 先跑完、P2 再跑完是一种交错顺序。
- 结果 2：按 1、2、4、5、6、3 的顺序能跑出来。教训：P2 插在 P1 中间也是一种交错顺序。
- 结果 0：按 4、5、1、2、3、6 的顺序能跑出来。教训：P2 先跑两步、P1 再跑也是一种交错顺序。

笔记本一句话： 第 32 题＋并发交错：6 条语句按各自顺序穿插，能出 1、2、0，-1 出不来。

【图像】

指令交错的三种结果（答案 C：0、1、2）

P1（加1）：**①load R1,x ②inc R1 ③store x,R1**　　P2（减1）：**④load R2,x ⑤dec R2 ⑥store x,R2**　　x 初值＝1

交错 1：①②③④⑤⑥

顺序执行：P1 先把 x 写成 2，P2 再写成 1 → **x＝1**

交错 2：①②④⑤⑥③

P2 整体插进 P1 的 load 与 store 之间 → P1 的"＋1"被覆盖丢失 → **x＝2**

交错 3：④⑤①②③⑥

P1 整体插进 P2 的 load 与 store 之间 → P2 的"－1"被覆盖丢失 → **x＝0**

核心：load / store 不是原子的 —— 谁后写，x 就是谁算出来的数

-1 不可能：每条序列"读-改-写"配对，加 1 和减 1 各做一次，不会出现"加两次/减两次"

别被"加1减1抵消、只能是1"误导 —— 中间插入对方的完整操作会把我方的更新覆盖掉

P1、P2 六条指令三种交错执行时间线：顺序执行得 1，P2 插进 P1 的读与写之间得 2，P1 插进 P2 的读与写之间得 0

【定位】 操作系统 · 第2章 · 进程同步

【易混对比】

- "结果 2"和"结果 0"的区别：2 是 P2 的"读-改-写"整体插进 P1 的 load 和 store 之间，P1 的加 1 被覆盖丢失；0 是反过来，P1 插进 P2 的 load 和 store 之间，P2 的减 1 被覆盖丢失——谁后写，x 就是谁算出来的数。
- "可能为 -1"为什么不可能：每条指令序列内部是"读-改-写"配对的，一次交错里加 1 和减 1 各做一次，不会出现"加两次"或"减两次"的效果。

x 的值取决于谁最后写：可能为 0、1 或 2，选 C。

【坑位】

别被"加 1 减 1 抵消、只能是 1"误导——load 和 store 不是原子的，对方的完整"读-改-写"插进中间，会把我方的更新覆盖掉。

【套路】

- 第一步，编号列交错：把 6 条指令编号，按"谁的 store 最后执行"分类。
- 第二步，找三种典型：顺序交错 1,2,3,4,5,6 得 1；1,2,4,5,6,3 得 2；4,5,1,2,3,6 得 0。

【类比】

像两人同时往黑板上写最终数字：都先看了一眼黑板（load，看到 x＝1），各自在草稿纸上算好（一个算出 2，一个算出 0），最后写上去的覆盖了先写的——黑板上留下的，就是后写的那个人算出来的数。

【口诀】 读写并非原子性，交错结果三种值；后写覆盖先写数，零一二中 C 是对。

【考频】 「并发执行」在 2009–2022 年解析中出现 11 次 · 高频

第 33 题 · TCP/IP 网络层向上提供的服务性质（正确答案：A）

题干与选项：

TCP/IP 参考模型的网络层提供的是( )

- A. 无连接不可靠的数据报服务
- B. 无连接可靠的数据报服务
- C. 有连接不可靠的虚电路服务
- D. 有连接可靠的虚电路服务

入手点： 看 IP 首部里有什么、没有什么：没有建连接的字段，就不是面向连接的；没有序号和校验和字段，就不是可靠的；剩下就是无连接、尽最大努力交付的数据报服务。

专有名词从零解释：

- 先想“网络层”：TCP/IP 四层里的第三层（从下往上数），负责把分组从一台主机送到另一台主机；代表协议是 IP。
- 再想“数据报服务”：每个分组自己找路、独立发送，不用先建连接；分组可能丢、可能乱序、可能重复。
- 再想“尽最大努力交付”：网络层会尽量把分组送到，但不保证一定送到、送到的顺序也不保证；丢了、错了不管，靠上层去处理。
- 再想“面向连接和可靠服务”：面向连接是先建连接再传数据；可靠是保证数据不丢、不错、不乱序；这两个都是运输层 TCP 干的事，不是网络层 IP 干的事。

考查知识点：

- 先想为什么是无连接的：TCP/IP 的网络层向上只提供简单灵活的、无连接的、尽最大努力交付的数据报服务；考查 IP 首部，如果是面向连接的，那么应有用于建立连接的字段，但是没有。
- 再想为什么是不可靠的：如果提供可靠的服务，那么至少应有序号和校验和两个字段，但是 IP 分组首部也没有（IP 首部中只有首部校验和，校验的只是首部本身，不是整个分组的数据）。
- 再想可靠是谁的活：通常有连接、可靠的应用是由运输层的 TCP 实现的；TCP 在 IP 的数据报服务之上，用序号、确认、重传做出可靠的效果。
- 拿生活例子验算：IP 像平信——写上地址就寄，不用先打电话建连接，寄丢了也不负责；TCP 像挂号信——先确认对方能收、每封信编号、丢了重寄，保证送到；平信的活是 IP 干的，挂号信的活是 TCP 干的。
- 边界情况：IP 首部有首部校验和，但它只校验首部有没有错，不保证数据不错、更不保证送到；不能因为有个“校验和”就说 IP 是可靠的。
- 易混对子：不要把“首部校验和”当成“数据校验和”——首部校验和只管首部，数据部分 IP 不管；不要把 TCP 的可靠当成 IP 的可靠——可靠是 TCP 在 IP 之上做出来的，不是 IP 本身提供的。

正确选项为什么对： 网络层向上只提供无连接的、尽最大努力交付的数据报服务，就是 A，选 A。

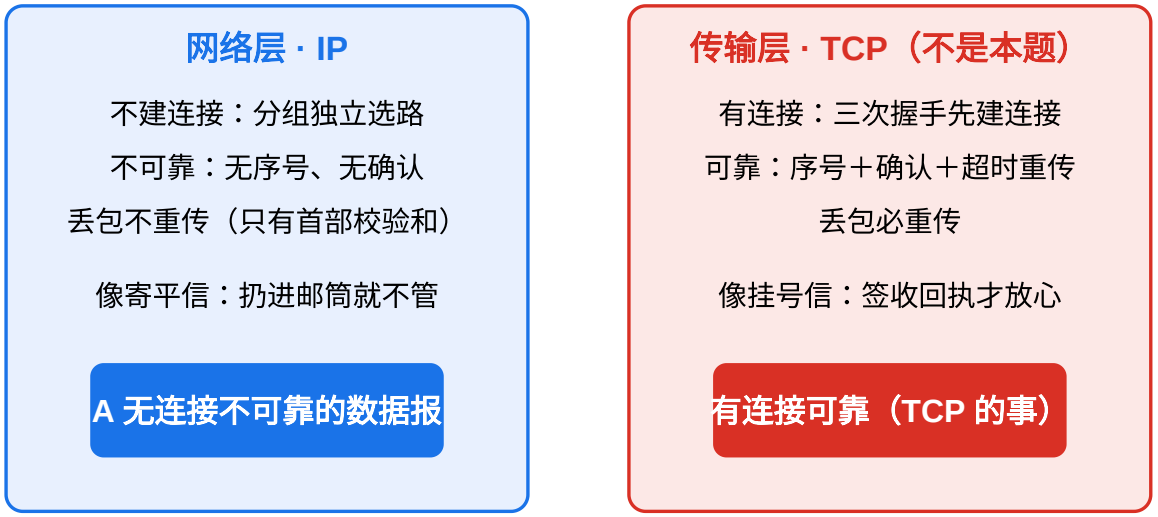
错误选项为什么错＋举例：

- 面向连接的服务：IP 首部没有建连接的字段，不是面向连接的。教训：建连接是 TCP 的事。
- 可靠的服务：IP 首部没有序号和数据校验和字段，不保证可靠。教训：可靠是 TCP 在 IP 之上做出来的。

笔记本一句话： 第 33 题＋IP 服务：无连接、尽最大努力交付的数据报服务，可靠是 TCP 的活。

【图像】

TCP/IP 网络层：无连接不可靠的数据报服务（A）



题干问的是网络层 → 只能选 A；虚电路（C、D）是有连接的，IP 不建连接

网络层 IP 无连接不可靠与传输层 TCP 有连接可靠的对比示意图

【定位】 计算机网络 · 第4章 网络层 · IP 数据报服务

【易混对比】

- "网络层的无连接不可靠"和"传输层的有连接可靠"的区别：前者是 IP 做的事，分组独立选路、不建连接、丢了不管；后者是 TCP 做的事，先三次握手建连接，再用序号加确认加超时重传保证可靠。题干问网络层，只能选前者。
- "数据报服务"和"虚电路服务"的区别：数据报是无连接的，每个分组独立选路、可能走不同路径；虚电路是有连接的，先建立一条逻辑连接，分组沿固定路径走。本题 A 是数据报口径，C、D 是虚电路口径。

TCP/IP 网络层只提供无连接不可靠的数据报服务，选 A。

【坑位】

题目问的是哪一层，别把 TCP 的可靠和有连接安到网络层头上。

误选 B（无连接可靠）：把 TCP 的可靠搬给了 IP。教训：IP 首部只有首部校验和，没有序号和确认机制，丢包不重传。误选 C、D（虚电路）：虚电路是有连接的，IP 根本不建连接；有连接可靠是 TCP 的事。

【套路】

- 第一步，先看层：题干说网络层，锁定 IP。
- 第二步，连问两个"有没有"：IP 建不建连接？不建。丢包重不重传？不重传。
- 第三步，对号入座：无连接加不可靠，只有 A。

看层，问建不建连接、问丢包重不重传，无连接不可靠选 A。

【类比】

IP 像寄平信：信扔进邮筒就完了，不挂号、不保送，丢了邮局不赔。TCP 像挂号信加回执：先确认对方在（建连接），每封信都要对方签收（确认），没签收就再寄一次（重传）。术语贴回去：平信是"无连接不可靠的数据报"，挂号信是"有连接可靠的虚电路式服务"。

【口诀】 网络层上 IP 跑，平信一封不管到；可靠有连接别认错，那是 TCP 的活要记牢。

【考频】 「IP 数据报服务」在 2009–2022 年解析中出现 2 次 · 低频

第 34 题 · 波特率与比特率换算（正确答案：B）

题干与选项：

若某通信链路的数据传输速率为 2400bit/s，采用 4 相位调制，则该链路的波特率是( )

- A. 600 波特
- B. 1200 波特
- C. 4800 波特
- D. 9600 波特

入手点： 一个码元能带几个比特，看它有几种变化；4 种相位就是 4 种变化，一个码元带 2 个比特；波特率＝比特率除以每个码元的比特数。

专有名词从零解释：

- 先想“码元”：信号的一个基本变化单位；一个码元可以表示几种不同的状态。
- 再想“波特率 B”：每秒传几个码元，单位是波特。
- 再想“比特率 C”：每秒传几个比特，单位是 bit/s。
- 再想换算公式：C＝B log2 N，N 是一个码元所取的离散值个数（也就是一个码元有几种变化）。

考查知识点：

- 先想一个码元带几个比特：采用 4 种相位，即可表示 4 种变化，故一个码元可携带 log2 4＝2bit 信息。
- 再想波特率：波特率＝比特率/2；按题目给的比特率算出来是 1200 波特，就是 B。
- 拿生活例子验算：一个信号灯一次能亮 4 种颜色（4 种变化），一次就传了 2 个比特的信息（00、01、10、11）；每秒传 2400 个比特，信号灯每秒只要变 1200 次。
- 边界情况：N 必须是 2 的幂次才好算 log2 N；4 种相位对应 2bit，8 种相位对应 3bit，2 种相位对应 1bit；波特率永远小于等于比特率（一个码元至少带 1 个比特）。
- 易混对子：不要把波特率和比特率当成一个东西——波特率数的是码元，比特率数的是比特；不要把 log2 N 算错——4 种变化是 log2 4＝2，不是 4。

正确选项为什么对： 一个码元带 2bit，波特率＝比特率/2＝1200 波特，就是 B，选 B。

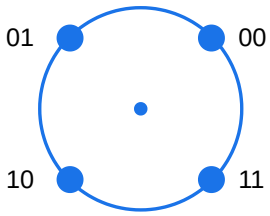
错误选项为什么错＋举例：

- 波特率算成比特率本身：漏除了每个码元的比特数。教训：一个码元带 2bit，波特率是比特率的一半。
- 波特率算成比特率的 2 倍：把除法做成了乘法。教训：C＝B log2 N，求 B 要除以 log2 N，不是乘。

笔记本一句话： 第 34 题＋波特率：C＝B log2 N，4 种相位一个码元 2bit，波特率＝比特率/2＝1200。

【图像】

4 相位调制：波特率 = 1200 波特（B）



4 种相位（离散值）

每码元比特数 = log2(4) = 2

即 1 个码元携带 2 比特

波特率 = 2400 ÷ 2 = 1200 波特

（码元数比比特数少，方向是除）

答案 B：1200 波特

误选 D（9600）：把相位数 4 直接乘了；误选 A（600）：除以了 4；先取对数再除

4 相位调制下 1 码元携带 2 比特、比特率除以 2 得到 1200 波特的计算示意图

【定位】 计算机网络 · 第2章 物理层 · 编码与调制

【易混对比】

- "比特率"和"波特率"的区别：比特率是单位时间传多少比特，波特率是单位时间传多少码元；1 码元可以携带多个比特，两者通过"每码元比特数"换算。
- "4 相位"和"1 码元 4 比特"的区别：4 相位是码元有 4 种离散取值，每码元携带的比特数是 log2(4)=2，不是 4。相位数要先取对数才能变成比特数。

每码元 2 比特，波特率=2400/2=1200 波特，选 B。

【坑位】

相位数不是每码元比特数，先取 log2 再除，别直接乘除相位数。

误选 D（9600）：把 2400 乘了 4。教训：相位数要先变成"每码元比特数"。误选 A（600）：把 2400 除以了 4。教训：除数是 log2(4)=2，不是 4。误选 C（4800）：把 2400 乘了 2。教训：波特率比比特率小，是除不是乘。

【套路】

- 第一步，相位数取对数：4 相位→log2(4)=2，即 1 码元携带 2 比特。
- 第二步，比特率除以每码元比特数：2400/2=1200 波特。
- 第三步，核对方向：1 码元顶 2 比特，码元数比比特数少，1200 小于 2400，方向对。

相位取对数得每码元比特数，比特率除以它得波特率。

【类比】

像用箱子装苹果运货：比特率是每秒运多少个苹果，波特率是每秒运多少个箱子。4 相位就是每箱装 2 个苹果（4 种状态对应 00、01、10、11）。每秒 2400 个苹果，每箱 2 个，就是每秒 1200 箱。术语贴回去：苹果是"比特"，箱子是"码元"，每秒箱数是"波特率"。

【口诀】 四相位来两比特，比特除二得波特；方向别搞反，一千二百选 B 项。

【考频】 「波特率」在 2009–2022 年解析中出现 3 次 · 中频

第 35 题 · 选择重传协议超时后重传哪些帧（正确答案：B）

题干与选项：

数据链路层采用选择重传协议（SR）传输数据，发送方已发送了 0～3 号数据帧，现已收到 1 号帧的确认，而 0、2 号帧依次超时，则此时需要重传的帧数是( )

- A. 1
- B. 2
- C. 3
- D. 4

入手点： 选择重传的确认是“逐个确认”，不是“累积确认”；1 号帧的确认只说明 1 号到了，不说明 0、2 号到了；超时的 0、2 号要重传，没超时的 3 号不用重传。

专有名词从零解释：

- 先想“选择重传协议”：接收方逐个确认正确接收的分组，不管收到的分组是否有序，只要正确接收就发一个选择 ACK 确认。
- 再想“累积确认”：GBN 协议用的办法——确认 n 号帧，就表示 n 号之前的所有帧都收到了；选择重传的 ACK 没有这个意思。
- 再想“超时重传”：发送方给每帧设一个计时器，计时器超时还没收到确认，就把这帧重发一遍。

考查知识点：

- 先想 1 号帧的确认是什么意思：在选择重传协议中，接收方逐个确认正确接收的分组；对 1 号帧的确认只说明 1 号帧收到了，不具累积确认的作用，不能推出 0 号帧也收到了。
- 再想 0、2 号帧：0、2 号帧超时，发送方认为接收方没有收到 0、2 号帧，于是重传这两帧。
- 再想 3 号帧：3 号帧计时器并无超时，所以暂时不重传 3 号帧。
- 再想答案：重传 0 号和 2 号帧、暂不重传 3 号帧，就是 B。
- 拿生活例子验算：选择重传像点名——老师点到 1 号，1 号答“到”，只说明 1 号在，不说明 0 号、2 号在；0 号、2 号点名没人答（超时），就再点一次；3 号还没到点名时间，先不点。
- 边界情况：如果 3 号帧的计时器也超时了，3 号也要重传；本题 3 号计时器没超时，所以不重传；如果用的是 GBN 协议，收到 1 号确认会累积确认 0 号，那就不用重传 0 号——但本题是选择重传，不是 GBN。
- 易混对子：不要把选择重传的 ACK 当成累积确认——选择 ACK 只认自己那一帧；不要把 GBN 的规则套到选择重传上——GBN 是“确认 n 号等于 n 号之前全收到”，选择重传不是。

正确选项为什么对： 选择 ACK 不累积，0、2 号超时重传，3 号没超时不重传，就是 B，选 B。

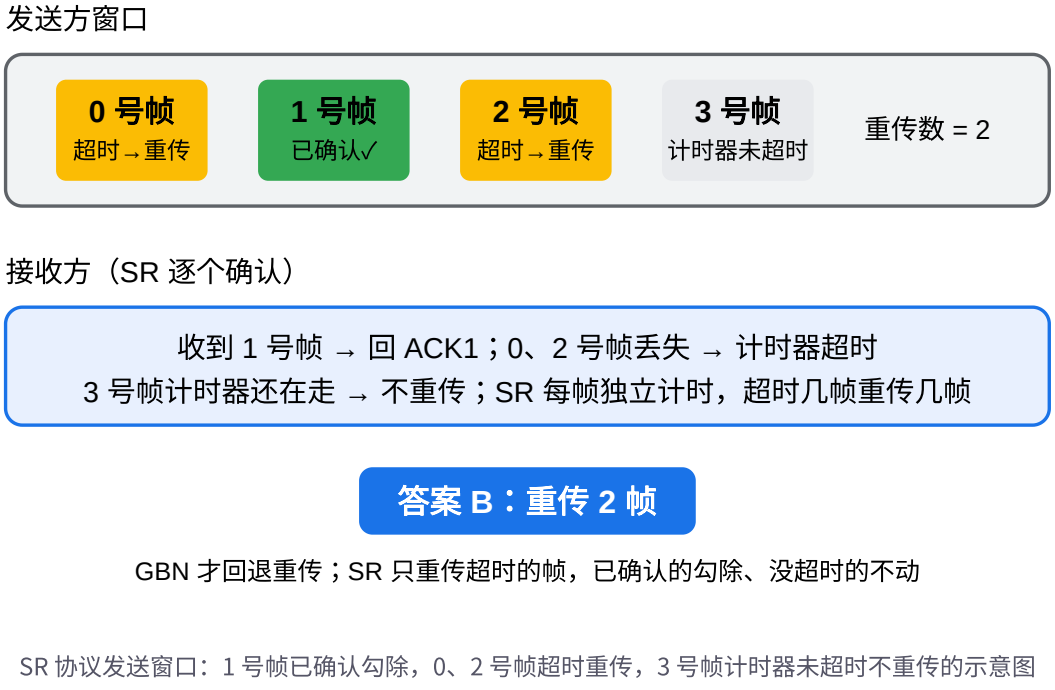
错误选项为什么错＋举例：

- 只重传 0 号或只重传 2 号：0、2 号都超时了，两帧都要重传。教训：超时的帧一帧都不能漏。
- 连 3 号也重传：3 号计时器没超时，不用重传。教训：没超时的帧先放着，等超时了再说。
- 不重传 0 号（以为 1 号确认累积了 0 号）：选择 ACK 不具累积确认的作用。教训：选择重传和 GBN 的确认规则不一样。

笔记本一句话： 第 35 题＋选择重传：ACK 逐个确认不累积，0、2 号超时重传，3 号没超时不重传。

【图像】

SR 协议：只重传超时的 0、2 号帧（B）



SR 协议发送窗口：1 号帧已确认勾除，0、2 号帧超时重传，3 号帧计时器未超时不重传的示意图

【定位】 计算机网络 · 第3章 数据链路层 · 选择重传协议 SR

【易混对比】

- "SR"和"GBN"的区别：SR 对每一帧单独确认，只重传超时的那一帧；GBN 用累积确认，一帧超时就回退，把后面已发的全重传。本题是 SR，所以只重传 0、2 号帧。
- "已发送未确认"和"需要重传"的区别：前者包括还在等计时器的帧，后者只包括计时器已经超时的帧。3 号帧已发送未确认，但计时器没超时，不用重传。

SR 逐个确认，只重传超时的 0、2 号帧，共 2 帧，选 B。

【坑位】

3 号帧的计时器还没超时，别把"已发送未确认"全部算成重传。

误选 C（3）：把 0、2、3 全算上了。教训：3 号帧只是还没收到确认，计时器还在走，不到超时不重传。误选 D（4）：连已确认的 1 号帧也算上了。教训：收到确认的帧直接勾除。误选 A（1）：按 GBN 的思路只重传最早超时的 1 帧。教训：SR 是每帧独立计时，超时几帧重传几帧。

【套路】

- 第一步，勾掉已确认的：收到 1 号帧确认，1 号出局。
- 第二步，挑出超时的：0、2 号帧依次超时，重传这两帧。
- 第三步，放过没超时的：3 号帧计时器还在走，不动。

确认的勾除，超时的重传，没超时的不动，数超时帧数。

【类比】

像老师收四份作业：1 号交了打勾；0 号、2 号过了截止时间还没交，点名重交；3 号还没到截止时间，再等等。SR 就是每份作业单独设一个截止时间（计时器），过期才催。术语贴回去：作业是"数据帧"，打勾是"确认"，截止时间是"超时计时器"，点名重交是"重传"。

【口诀】 选择重传各管各，确认勾除超时拖；三号计时还在走，重传只要两帧多。

【考频】 「选择重传」在 2009–2022 年解析中出现 5 次 · 中频

第 36 题 · 用 ACK 帧确认数据帧正确到达目的地的无线局域网协议（正确答案：D）

题干与选项：

下列选项中，对正确接收到的数据帧进行确认的 MAC 协议是( )

- A. CSMA
- B. CDMA
- C. CSMA/CD
- D. CSMA/CA

入手点： 无线网里冲突不好检测，所以要用“发完等确认”的办法；哪个协议有 ACK 机制，哪个就是答案。

专有名词从零解释：

- 先想“CSMA/CA”：无线局域网标准 802.11 中的协议，在 CSMA（载波监听多路访问）的基础上增加了冲突避免的功能。
- 再想“ACK 帧”：接收方收到数据帧后发回的确认帧；发送方收到 ACK 帧，才能确认发出的数据帧已正确到达目的地。
- 再想“CSMA/CD”：带冲突检测的载波监听多路访问，以太网用的办法——边发边听，听到冲突就停下来重发；接收方并不需要确认。
- 再想“CDMA”：码分多址，是物理层的内容，用不同的码来区分用户，不是介质访问控制协议。

考查知识点：

- 先想 CSMA/CA 的 ACK 机制：ACK 帧是 CSMA/CA 避免冲突的机制之一；只有当发送方收到接收方发出的 ACK 帧后，才能确认发出的数据帧已正确到达目的地。
- 再想排除法：CDMA 是物理层的内容，先排除；CSMA/CD 是带冲突检测的办法，接收方并不需要确认，排除；CSMA 是 CSMA/CD 的超集所没有的内容——既然 CSMA/CD 没有 ACK 确认，CSMA 自然也没有，排除；剩下就是 CSMA/CA，就是 D。
- 拿生活例子验算：无线网像隔着墙喊话，喊的人听不见自己是不是撞了别人的话（冲突不好检测），所以规矩是“喊完等对方回一句‘收到了’（ACK）”；以太网像在一个房间里喊话，喊的时候能听见撞没撞上，就不用等对方回话。
- 边界情况：CSMA/CA 除了 ACK，还有 RTS/CTS（请求发送／允许发送）等避免冲突的办法；但本题问的是“用 ACK 帧确认到达”的协议，答案就是 CSMA/CA。
- 易混对子：不要把 CSMA/CD 和 CSMA/CA 弄反——CD 是冲突检测（以太网），CA 是冲突避免（无线网）；不要把 CDMA 当成介质访问控制协议——CDMA 是物理层的码分多址。

正确选项为什么对： CSMA/CA 用 ACK 帧确认数据帧正确到达目的地，就是 D，选 D。

错误选项为什么错＋举例：

- CDMA：物理层的内容，不是介质访问控制协议。教训：CDMA 是码分多址，在物理层。
- CSMA/CD：带冲突检测的以太网办法，接收方不需要确认。教训：能检测冲突的，不用 ACK。
- CSMA：CSMA/CD 都没有 ACK 确认，CSMA 自然也没有。教训：CSMA 是 CSMA/CD 的基础，没有 ACK 机制。

笔记本一句话： 第 36 题＋无线网确认：CSMA/CA 用 ACK 确认到达，CSMA/CD 靠冲突检测不用确认。

【图像】

对正确接收的数据帧进行确认：CSMA/CA（D）

无线 · CSMA/CA

发送方 → 数据帧 → 接收方

接收方 → ACK 确认帧 → 发送方

冲突避免：发送方检测不到冲突

靠接收方 ACK 确认收到

D：有确认机制 ✓

有线 · CSMA/CD

发送方边发边听

自己能检测到冲突

冲突了自己重发

不需要接收方确认

无确认机制 ✗

CDMA 是物理层码分多址技术，不是 MAC 协议；纯 CSMA 也没有确认机制

题眼"对正确接收到的数据帧进行确认" → 只有无线局域网的 CSMA/CA 用 ACK

802.11 CSMA/CA 用 ACK 确认数据帧、与 CSMA/CD 发送方自检冲突无需确认的对比示意图

【定位】 计算机网络 · 第3章 数据链路层 · 无线局域网 MAC 协议

【易混对比】

- "CSMA/CA"和"CSMA/CD"的区别：CA 用于无线局域网，冲突避免，发送方检测不到冲突，靠接收方回 ACK 来确认收到；CD 用于有线以太网，冲突检测，发送方边发边听，自己能发现冲突，不需要接收方确认。
- "CDMA"和"CSMA"的区别：CDMA 是物理层的码分多址技术，用正交码区分用户；CSMA 是 MAC 层的载波监听多路访问。题目问 MAC 协议对数据帧的确认，CDMA 不在讨论范围内。

只有 CSMA/CA（802.11 无线局域网）用 ACK 确认正确接收的数据帧，选 D。

【坑位】

无线环境下发送方检测不到冲突，所以必须靠接收方的 ACK 确认，这是 CA 和 CD 的分水岭。

误选 C（CSMA/CD）：把有线以太网的协议套到"确认"上。教训：CD 靠发送方自己边发边听检测冲突，不需要接收方确认。误选 B（CDMA）：看到"MA"就当 MAC 协议。教训：CDMA 是物理层多址技术。误选 A（CSMA）：纯载波监听，没有确认机制。

【套路】

- 第一步，抓题眼："对正确接收到的数据帧进行确认"→找谁有 ACK 机制。
- 第二步，定位场景：无线局域网用 CSMA/CA，冲突避免加 ACK 确认。
- 第三步，排除：CSMA/CD 无需确认，CDMA 是物理层，CSMA 无确认机制→D。

题眼是"确认"，无线用 CA 加 ACK，有线 CD 自己检测不需要确认。

【类比】

像在嘈杂的广场上喊话：无线（CA）是你喊完听不见回声，只能让对方举手示意"收到了"（ACK）；有线（CD）像两个人拉一根绳子说话，一边说一边能感觉到绳子被拽（冲突），自己就知道重说，不用对方确认。术语贴回去：举手示意是"ACK 确认帧"，感觉绳子被拽是"冲突检测"。

【口诀】 无线难测靠确认，CA 回 ACK 来兜底；有线自检 CD 行，CDMA 物理别乱提。

【考频】 「CSMA/CA」在 2009–2022 年解析中出现 5 次 · 中频

第 37 题 · 路由聚合与下一跳（正确答案：D）

题干与选项：

某网络拓扑如下图所示，路由器 R1 只有到达子网 192.168.1.0/24 的路由。为使 R1 可以将 IP 分组正确地路由到图中所有子网，则在 R1 中需要增加的一条路由（目的网络，子网掩码，下一跳）是( )

- A. 192.168.2.0，255.255.255.128，192.168.1.1
- B. 192.168.2.0，255.255.255.0，192.168.1.1
- C. 192.168.2.0，255.255.255.128，192.168.1.2
- D. 192.168.2.0，255.255.255.0，192.168.1.2

> 注：网络拓扑图以原卷为准

入手点： 先把两个子网的地址写成二进制，看前面几位相同；相同的部分就是聚合后的超网，掩码就是相同部分的位数；下一跳是和 R1 直接相连的下一个路由器的地址。

专有名词从零解释：

- 先想“路由聚合”：把几个地址相邻、前面几位相同的子网合成一个更大的网（超网），路由表里只要记一条，表就小了。
- 再想“子网掩码”：前面是 1、后面是 0 的一串；1 的部分是网络号，0 的部分是主机号；聚合后的掩码就是相同部分的位数。
- 再想“下一跳”：分组从这台路由器出去，下一个要去的路由器的地址；下一跳一定是和这台路由器直接相连的。

考查知识点：

- 先想 R1 需要的路由：要使 R1 能够正确将分组路由到所有子网，R1 中需要有到 192.168.2.0/25 和 192.168.2.128/25 的路由。
- 再想写成二进制：192.168.2.0 是 11000000 10101000 00000010 00000000；192.168.2.128 是 11000000 10101000 00000010 10000000。
- 再想聚合：前 24 位都相同，于是可以聚合成超网 192.168.2.0/24，子网掩码为前 24 位，即 255.255.255.0。
- 再想下一跳：下一跳是与 R1 直接相连的 R2 的地址，因此是 192.168.1.2。
- 再想答案：目的网络 192.168.2.0/24、掩码 255.255.255.0、下一跳 192.168.1.2，就是 D。
- 拿生活例子验算：两个门牌号 192.168.2.0 和 192.168.2.128，前 24 位一样，就像同一个小区的两栋楼；邮局只要记“这个小区的信都往 R2 那边送”，不用两栋楼各记一条。
- 边界情况：聚合的前提是两个子网地址相邻、前面几位相同；如果前面几位不同，就不能聚合；聚合后的掩码位数就是相同部分的位数，不是原来两个掩码的平均数。
- 易混对子：不要把聚合后的掩码写成 /25——聚合之后网络变大了，掩码位数变小了，是 /24；不要把下一跳写成目的网络里的地址——下一跳是和 R1 直接相连的 R2 的地址，不是目的子网里的地址。

正确选项为什么对： 前 24 位相同聚合成 192.168.2.0/24，掩码 255.255.255.0，下一跳是直接相连的 R2 的 192.168.1.2，就是 D，选 D。

错误选项为什么错＋举例：

- 掩码写成 /25：那是聚合前单个子网的掩码，不是聚合后的。教训：聚合之后网络变大，掩码位数变小。

- 目的网络写成两个子网之一：聚合之后只要记一条超网，不用各记一条。教训：聚合就是为了把两条合成一条。
- 下一跳写成目的子网里的地址：下一跳是和 R1 直接相连的 R2 的地址。教训：下一跳看的是“下一步去哪”，不是“最终去哪”。

笔记本一句话： 第 37 题＋路由聚合：前 24 位相同聚合成 /24，掩码 255.255.255.0，下一跳是直接相连的 R2。

【图像】



192.168.2.0/25 与 192.168.2.128/25 前 24 位相同聚合成 192.168.2.0/24、下一跳取直连地址 192.168.1.2 的示意图

【定位】 计算机网络 · 第4章 网络层 · 路由聚合与 CIDR

【易混对比】

- "路由聚合"和"划分子网"的区别：聚合是把多个小网络合并成一个大网络，网络前缀变短、掩码变短；划分子网是把一个大网络切成多个小网络，掩码变长。本题是把两个 /25 合成一个 /24，属于聚合。
- "下一跳"和"目的网络"的区别：目的网络写聚合后的大网络地址，下一跳写去往该网络方向上与本路由器直连的邻居接口地址。本题下一跳是 R2 上与 R1 直连的接口 192.168.1.2，不是 R1 自己的 192.168.1.1。

聚合成 192.168.2.0/24，掩码 255.255.255.0，下一跳 192.168.1.2，选 D。

【坑位】

下一跳必须是与 R1 直连的邻居接口地址，别写成 R1 自己的接口地址。

误选 B：下一跳写成 192.168.1.1。教训：192.168.1.1 是 R1 自己的接口，下一跳要写对端 R2 的接口 192.168.1.2。误选 A、C（掩码 255.255.255.128）：只写了一条 /25，没做聚合。教训：两条路由前 24 位相同，可以合成一条 /24，一条路由顶两条。

【套路】

- 第一步，比前缀：192.168.2.0/25 和 192.168.2.128/25 前 24 位相同→聚合成 192.168.2.0/24。
- 第二步，写掩码：/24 对应 255.255.255.0。

- 第三步，找直连邻居：看拓扑，R1 去往 192.168.2.0/24 要经过 R2，下一跳写 R2 上与 R1 直连的接口 192.168.1.2。

前缀相同就聚合，掩码跟着变短，下一跳写直连邻居的接口地址。

【类比】

像快递分拣：原来要贴两张面单（一张到 2.0/25，一张到 2.128/25），发现前 24 位都是"北京市朝阳区"，就合成一张大面单"北京市朝阳区"（/24），下一站写直接相连的中转站地址。术语贴回去：面单是"路由表项"，大面单是"聚合路由"，中转站地址是"下一跳"。

【口诀】 两条路由前段同，聚合一条掩码松；下一跳找直连口，一点二的地址别选错。

【考频】 「路由聚合」在 2009–2022 年解析中出现 3 次 · 中频

第 38 题 · /30 子网可容纳的主机数（正确答案：C）

题干与选项：

在子网 192.168.4.0/30 中，能接收目的地址为 192.168.4.3 的 IP 分组的最大主机数是( )

- A. 0
- B. 1
- C. 2
- D. 4

入手点： /30 就是主机号只剩 2 位；2 位一共 4 个地址，再去掉全 0 和全 1 两个特殊地址，剩下的就是能用的主机地址。

专有名词从零解释：

- 先想“/30”：子网掩码里前面 30 位是网络号，后面 2 位是主机号。
- 再想“主机号全 0”：表示这个网络本身，不作为主机地址使用。
- 再想“主机号全 1”：广播地址，表示这个网络里的所有主机，不作为某台主机的地址使用。

考查知识点：

- 先想地址范围：192.168.4.0/30，主机号只占 2 位，地址范围为 192.168.4.0～192.168.4.3，一共 4 个地址。
- 再想去掉特殊地址：主机号全 1 时即 192.168.4.3 是广播地址，主机号全 0 即 192.168.4.0 表示本网络本身、不作为主机地址使用。
- 再想可容纳主机数：4-2＝2 个主机，就是 C。
- 拿生活例子验算：一个小院只有 2 位门牌号（00、01、10、11），00 是院子本身的编号，11 是广播用的“全院喊话”编号，真正能住人的只有 01 和 10 两间房。
- 边界情况：/31 的网络主机号只有 1 位，按公式 2-2＝0，但实际点对点链路里 /31 可以住 2 台（两个地址都当主机用，不设广播）；本题是 /30，按常规公式算就是 2 台。
- 易混对子：不要把 4 个地址都当成主机地址——全 0 和全 1 是特殊地址，要去掉；不要把广播地址当成某台主机的地址——广播地址是喊全网用的，不是某台主机的。

正确选项为什么对： 4 个地址去掉网络地址和广播地址，剩 2 个主机地址，就是 C，选 C。

错误选项为什么错＋举例：

- 算成 4 台：没去掉全 0 和全 1 两个特殊地址。教训：主机号全 0 是网络本身，全 1 是广播，都不能当主机地址。
- 算成 3 台：只去掉了一个特殊地址。教训：全 0 和全 1 两个都要去掉。

笔记本一句话： 第 38 题＋/30 主机数：2 位主机号 4 个地址，去掉网络和广播，剩 2 台。

【图像】

192.168.4.0/30：可用主机 2 个（C）

/30 → 主机号 2 位 → 共 4 个地址（192.168.4.0 ～ 192.168.4.3）

192.168.4.0	192.168.4.1	192.168.4.2	192.168.4.3
主机号全 0	可用	可用	主机号全 1
网络地址 ✗	主机 ✓	主机 ✓	广播地址 ✗

全 0（网络地址）和全 1（广播地址）都不能分配给主机 → 4 - 2 = 2

目的地址为 192.168.4.3 的分组是广播 → 网内 2 台主机都能收到

答案 C：2

误选 D（4）：忘了掐头去尾；误选 A（0）：广播是全网都收，不是没人收

/30 子网 4 个地址：全 0 网络地址、全 1 广播地址不可作主机地址，只剩 2 个可用主机地址的示意图

【定位】 计算机网络 · 第4章 网络层 · 划分子网

【易混对比】

- "地址总数"和"可用主机数"的区别：/30 的地址总数是 4 个，但全 0 的是网络地址、全 1 的是广播地址，都不能分配给主机，可用主机数是 4-2=2。
- "192.168.4.3"和"主机地址"的区别：4.3 是主机号全 1 的广播地址，不是某个主机的地址；以它为目的地址的分组是广播，网内所有主机都能收到。

/30 共 4 个地址，去掉网络地址和广播地址，可用主机 2 个，选 C。

【坑位】

192.168.4.3 是本网广播地址，谁也不能拿它当自己的地址，但网内所有主机都能收到发往它的分组。

误选 D（4）：把地址总数当成了主机数。教训：全 0 网络地址和全 1 广播地址不能分配给主机。误选 A（0）：以为广播地址发出去没人收。教训：题目问"能接收目的地址为 4.3 的分组的最大主机数"，广播是网内所有主机都收，2 台主机都能收到。误选 B（1）：把广播当成单播只算一台。教训：广播地址的分组是发给全网的。

【套路】

- 第一步，算地址总数：/30 主机号 2 位→2 的 2 次方=4 个地址（4.0～4.3）。
- 第二步，掐头去尾：4.0 是网络地址，4.3 是广播地址，都不能作主机地址。
- 第三步，看题问什么：问能收到目的地址为 4.3 的分组的主机数→广播全网收→剩下 2 台主机→2。

总数减网络减广播，广播发往全网都收得到。

【类比】

像一栋楼只有 4 个门牌号：4.0 是楼栋号（整栋楼的代号），4.3 是大喇叭（对全楼广播），真正能住人的只有 4.1 和 4.2。大喇叭一喊，全楼住户都听得见，所以能收到"喊给 4.3 的话"的住户是 2 户。术语贴回去：楼栋号是"网络地址"，大喇叭是"广播地址"，住户是"主机"。

【口诀】 三十掩码两位主机，四个地址掐头去尾；广播人人能收到，可用主机只剩俩。

【考频】 「子网划分」在 2009–2022 年解析中出现 9 次 · 中频

第 39 题 · TCP 第二次握手报文段的 SYN、ACK、确认号（正确答案：C）

题干与选项：

主机甲向主机乙发送一个（SYN=1，seq=11220）的 TCP 段，期望与主机乙建立 TCP 连接，若主机乙接受该连接请求，则主机乙向主机甲发送的正确的 TCP 段可能是( )

- A.（SYN=0，ACK=0，seq=11221，ack=11221）
- B.（SYN=1，ACK=1，seq=11220，ack=11220）
- C.（SYN=1，ACK=1，seq=11221，ack=11221）
- D.（SYN=0，ACK=0，seq=11220，ack=11220）

入手点： 第二次握手是乙的回应：SYN 和 ACK 都要置 1；确认号是甲的初始序号加 1；乙自己的初始序号是乙自己随便选的，和甲的序号没关系。

专有名词从零解释：

- 先想“三次握手”：TCP 建连接要握三次手——甲发请求（SYN），乙回请求加确认（SYN＋ACK），甲再回确认（ACK）。
- 再想“SYN（同步位）”：表示“我想建连接，顺便告诉你我的初始序号”；再想“ACK（确认位）”：表示“这是一个确认报文段，确认号字段有效”。
- 再想“序号 seq”：本报文段数据部分的第一个字节的序号；建连接时，双方各选一个初始序号，握手报文段要消耗一个序号。
- 再想“确认号 ack”：期待收到对方下一个报文段的第一个字节的序号；收到对方的初始序号是 n，确认号就是 n+1。

考查知识点：

- 先想 SYN 和 ACK：在确认报文段中，同步位 SYN 和确认位 ACK 必须都是 1——乙既要回应甲的请求（SYN），又要确认甲的序号（ACK）。
- 再想确认号：返回的确认号 ack 是甲发送的初始序号 seq＝11220 加 1，即 ack＝11221。
- 再想乙的序号：乙也要选择并消耗一个初始序号 seq，seq 值由乙的 TCP 进程任意给定，它与确认号、请求报文段的序号没有任何关系（注：乙的 seq 具体数值在源材料中未给出，这里不编造）。
- 再想答案：SYN＝1、ACK＝1、ack＝11221，就是 C。
- 拿生活例子验算：甲打电话说“我从 11220 号开始说”（SYN）；乙回电话说“我听到了，你下次从 11221 号开始说”（ACK＝11221），同时说“我从我自己选的号开始说”（乙的 SYN，号码乙自己定）。
- 边界情况：第一次握手的 SYN＝1、ACK＝0；第二次握手的 SYN＝1、ACK＝1；第三次握手的 SYN＝0、ACK＝1；三次握手里 SYN 和 ACK 的组合每次都不一样，记混了就错。
- 易混对子：不要把确认号写成甲的序号本身——确认号是“期待收到的下一个序号”，是甲的序号加 1；不要以为乙的序号和甲的序号有关系——乙的初始序号是乙自己选的，和甲的没关系。

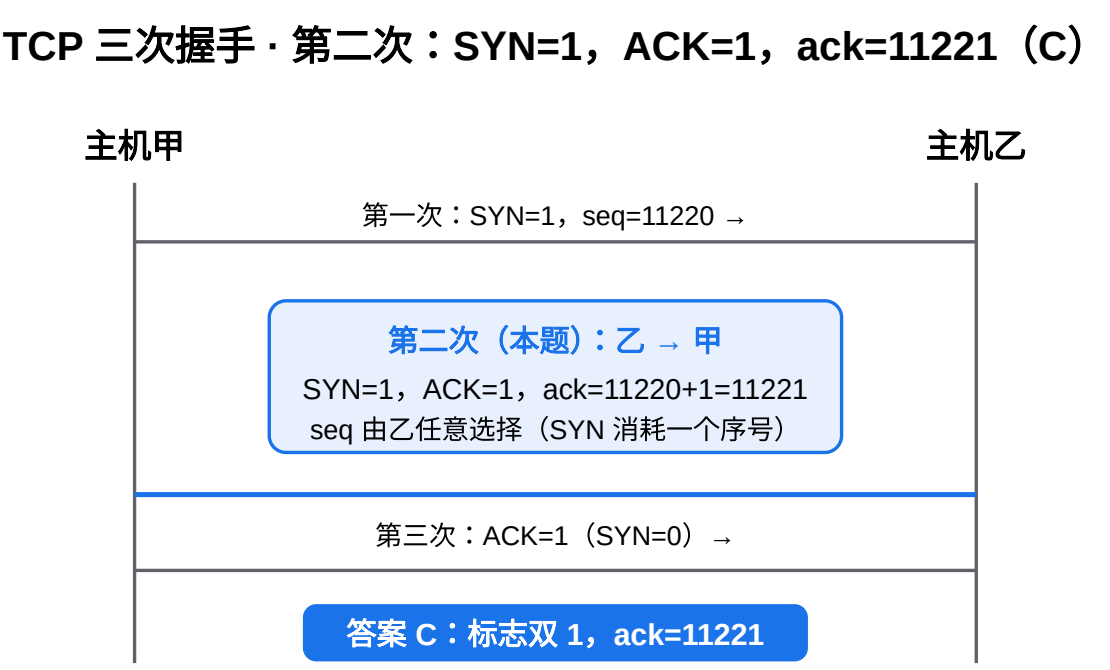
正确选项为什么对： 第二次握手 SYN＝1、ACK＝1，ack＝甲的 11220 加 1＝11221，就是 C，选 C。

错误选项为什么错＋举例：

- SYN 或 ACK 写成 0：第二次握手既要同步又要确认，两个位都必须是 1。教训：第一次是 SYN＝1、ACK＝0，第二次是两个都是 1，第三次是 SYN＝0、ACK＝1。
- 确认号写成 11220：确认号是期待的下一个序号，是 11220＋1＝11221。教训：确认号永远是对方序号加 1。

笔记本一句话： 第 39 题＋第二次握手：SYN＝1、ACK＝1，ack＝对方序号＋1＝11221，乙的序号自己选。

【图像】



C 的 seq=11221 只是"可能"之一；判题依据是 SYN=1、ACK=1、ack=11221，乙的 seq 不得编造

TCP 三次握手第二次：SYN=1、ACK=1、ack=11221、seq 由乙任意选择且 SYN 消耗一个序号的示意图

【定位】 计算机网络 · 第5章 传输层 · TCP 连接建立

【易混对比】

- "第二次握手"和"第三次握手"的区别：第二次是乙发给甲的，SYN=1 且 ACK=1，既确认甲的 SYN 又发出自己的 SYN；第三次是甲发给乙的，SYN=0、ACK=1，只是确认。
- "ack"和"seq"的区别：ack 是确认号，必须等于对方 seq 加 1（本题 11220+1=11221）；seq 是自己选的初始序号，由乙任意选择，不能从甲的 seq 推出来。

第二次握手 SYN=1、ACK=1、ack=11221，seq 由乙自选，选 C。

【坑位】

C 的 seq=11221 只是"可能"的正确形式之一，千万别记成乙的 seq 必然等于 11221；判题依据是 SYN=1、ACK=1、ack=11221。

误选 A、D：SYN=0、ACK=0。教训：第二次握手既要发自己的 SYN 又要确认对方，两个标志位都是 1。误选 B：ack=11220。教训：ack 是期望收到的下一个序号，甲的 SYN 消耗了一个序号，ack 必须是 11220+1=11221。

【套路】

- 第一步，看标志位：第二次握手 SYN=1 且 ACK=1→排除 A、D。
- 第二步，看 ack：ack=11220+1=11221→排除 B。
- 第三步，看 seq：乙的 seq 由乙任意选择，不判对错→剩 C。

先看双 1 标志位，再看 ack 加 1，seq 任意不纠结。

【类比】

像两个人约见面：甲打电话说"我是甲，暗号 11220，想约你"（SYN=1，seq=11220）。乙回电话说"收到你的暗号，下一个暗号该 11221 了（ack=11221），我也要约你，我的暗号我自己定"（SYN=1，ACK=1，seq 自选）。乙的暗号是多少，电话里没规定，乙说了算。术语贴回去：打电话约见面是"建立连接"，暗号是"序号"，回"我收到了"加确认是"ACK"。

【口诀】 二次握手双旗举，ack 加一要记住；seq 随乙心意定，C 项三条全对路。

【考频】 「TCP 三次握手」在 2009–2022 年解析中出现 3 次 · 中频

第 40 题 · TCP 序号与确认号的计算（正确答案：B）

题干与选项：

主机甲与主机乙之间已建立一个 TCP 连接，主机甲向主机乙发送了 3 个连续的 TCP 段，分别包含 300 字节、400 字节和 500 字节的有效载荷，第 3 个段的序号为 900。若主机乙仅正确接收到第 1 和第 3 个段，则主机乙发送给主机甲的确认序号是( )

- A. 300
- B. 500
- C. 1200
- D. 1400

入手点： 序号是“本段第一个字节的序号”，确认号是“期待收到的下一个段的第一个字节的序号”；先算第二个段的序号，再看乙期待的是哪个段。

专有名词从零解释：

- 先想“序号”：TCP 首部的序号字段，是指本报文段数据部分的第一个字节的序号。
- 再想“确认号”：TCP 首部的确认号字段，是指期待收到对方下一个报文段的第一个字节的序号。
- 再想“段长 400 字节”：一个段里有 400 个字节的数据，序号从第一个字节开始，往后数 400 个字节就是下一个段的第一个字节。

考查知识点：

- 先想第二个段的序号：第三个段的序号为 900，每个段 400 个字节，所以第二个段的序号为 900-400＝500。
- 再想乙期待的是哪个段：现在主机乙期待收到第二个段（第二个段还没收到，第三个段已经到了），所以乙发给甲的确认号是第二个段的第一个字节的序号，也就是 500，就是 B。
- 拿生活例子验算：甲寄了三包书，每包 400 页；第三包从第 900 页开始数，往前推一包，第二包就是从第 500 页开始；乙收到了第一包和第三包，还缺第二包，就告诉甲“你下次从第 500 页开始寄”。
- 边界情况：确认号是“期待收到的下一个”，不是“已经收到的最后一个”；已经收到的最后一个字节是 499，期待的下一个是 500，确认号写 500，不写 499。
- 易混对子：不要把确认号写成 900——900 是第三个段的序号，乙期待的是第二个段；不要把确认号写成 499——499 是已经收到的最后一个字节，确认号是期待的下一个字节 500。

正确选项为什么对： 第二个段序号是 900-400＝500，乙期待第二个段，确认号就是 500，就是 B，选 B。

错误选项为什么错＋举例：

- 确认号写成 900：900 是第三个段的序号，不是乙期待的第二个段。教训：先看乙缺的是哪个段，再算那个段的序号。
- 确认号写成 499：499 是已收到的最后一个字节，确认号是期待的下一个字节 500。教训：确认号永远是“下一个”，不是“最后一个”。

笔记本一句话： 第 40 题＋TCP 确认号：序号是本段首字节，确认号是期待的下段首字节，900-400＝500。

【图像】

TCP 确认号指向缺口：500（B）

字节流上的三段（序号 = 本段第一个字节的编号）：



乙的确认（累积确认，指向第一个缺口）：

乙期待收到的下一个字节 = 第 2 段起点 → 确认号 = 500

答案 B：500

确认号卡在缺口上，别被第 3 段的 900 带偏；确认号 = 期待收到的下一个字节序号

TCP 字节流上三段位置：第 2 段序号 500 缺失，乙期待第 2 段，确认号指向缺口 500 的示意图

【定位】 计算机网络 · 第5章 传输层 · TCP 可靠传输

【易混对比】

- "序号"和"确认号"的区别：序号是本段第一个字节的编号，确认号是期望收到的下一个字节的编号；一个指向自己，一个指向对方。
- "确认号从缺口算"和"确认号从最后收到的算"的区别：TCP 的确认是累积的，确认号指向第一个没收到的字节。乙收到了第 1、3 段，缺的是第 2 段，确认号指向第 2 段的起点 500，而不是第 3 段的 900。

第 2 段序号=900-400=500，乙缺第 2 段，确认号=500，选 B。

【坑位】

乙收到了第 3 段不代表确认号往后走，确认号卡在缺口上，别被第 3 段的序号 900 带偏。

误选 C（1200）：按"收到第 3 段就确认到第 3 段末尾"算的。教训：中间缺了一段，确认号只能停在缺口起点。误选 A（300）：把第 1 段的长度当成了确认号。教训：确认号是"期待收到的下一个字节"，第 1 段已经收完，下一个是第 2 段的起点。误选 D（1400）：把三段长度全加了。教训：确认号看的是"缺口"，不是"总量"。

【套路】

- 第一步，倒推缺失段的序号：第 3 段序号 900 减去第 2 段长度 400→第 2 段序号 500。
- 第二步，找缺口：乙只收到第 1、3 段，缺第 2 段。
- 第三步，确认号指向缺口：期待收到的下一个字节就是第 2 段起点→500。

先倒推缺失段序号，确认号永远指向第一个缺口。

【类比】

像收快递：三件包裹编号按字节排，甲发了 1、2、3 号，乙只收到 1 号和 3 号。乙给甲回条写"2 号还没到，请补发 2 号"——回条上写的就是 2 号的起始编号 500，而不是"3 号我收到了"。术语贴回去：回条是"确认"，回条上的编号是"确认号"，没收到的 2 号是"缺口"。

【口诀】 序号指段首字节，确认指望下一位；缺口在哪确认哪，二段五百选 B 对。

【考频】 「TCP 序号」在 2009–2022 年解析中出现 5 次 · 中频

附录 · 答案速查表

- 1–8：A、B、B、C、C、D、A、C
- 9–16：D、A、B、D、A、B、D、A
- 17–24：C、D、C、C、D、C、B、A
- 25–32：D、B、D、D、A、C、B、C
- 33–40：A、B、B、D、D、C、C、B